

tasks

Workshop Curriculum Platform Implementation Plan

TASK IDS ARE PER-FEATURE AND ARE NOT GLOBALLY UNIQUE — CITE THEM QUALIFIED. This file and `specs/002-knowledge-areas-deep-linking/tasks.md` number their tasks independently from T001, so the same id names two unrelated pieces of work in the two documents. **Measured 2026-09-03, the overlap is total, not an edge case: every one of this file's 120 task ids — T001 through T120 — also exists in 002 meaning something else** (002 runs T001–T143). T115 here is the workshop/scripts/index.sh wrapper; T115 in 002 is the answer-against-question verifier. T116 here is the crossref.sh wrapper; T116 in 002 is the verifier that could not decide.

The requirement ids collide the same way, and by the same construction. Measured the same day against the two spec.md files, which define their own ids independently from 001: SC-001–SC-021 (21 ids) and FR-001–FR-048 (48 ids — every FR this feature has) are defined in both specs with different meanings. This feature's SC-009 is citation support over generated answers; 002's SC-009 is a media-backed citation landing inside its cited span. Only SC-016a is unique to this file; 002's SC-015a–SC-015e, SC-017a–SC-017c, SC-022–SC-036, FR-049–FR-066 and T121–T143 are unique to that one.

Rule — prefix the feature number: 001:T115 vs 002:T115, 001:SC-009 vs 002:FR-030. A bare T115 in a brief, a commit message, a review or a source comment is ambiguous by construction. An unqualified citation of exactly this form already exists in the platform backend's own source at `workshop/platform/backend/pkg/answer/outcome.go`, where the pair it names is 002's; and two

briefs written from these files are **reported** to have resolved a bare id to the wrong task — that report is recorded here as received and was not independently measured.

Ids are NOT renumbered to fix this. Every existing cross-reference — in both `spec.md` files, in both `contracts/` trees, in `plan.md`, and in the implementation's own source comments — resolves against the current numbering, and renumbering would invalidate all of them at once. Qualification at the point of citation is the remedy; renumbering is not.

For agentic workers: REQUIRED SUB-SKILL: Use `superpowers:subagent-driven-development` (recommended) or `superpowers:executing-plans` to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Turn a 1h55m workshop recording into a browsable, semantically searchable curriculum whose every claim is traceable to a timestamped passage.

Architecture: An offline pipeline transcribes the recording into timestamped passages, each carrying a minted ULID anchored into its source artifact. A Go service serves the curriculum and a two-path search (lexical FTS5 for type-ahead, embeddings on submit) over a passage registry; an Angular client renders it. Answering is a pluggable provider seam that refuses rather than fabricates. Container orchestration is consumed from `submodules/containers`, never reimplemented.

Tech Stack: Go 1.26.2 · Angular (Karma/Jasmine) · SQLite + FTS5 (`modernc.org/sqlite`) · `faster-whisper` / `CTranslate2` CPU int8 · Lumen + ollama embeddings · podman (rootless) via `digital.vasic.containers` · Python 3.14.6 + Bash for the pipeline.

Global Constraints

Project-wide requirements. Every task's requirements implicitly include this section. Values are copied verbatim from `spec.md` and `research.md` — do not round them, do not “improve” them.

- **Suggestions $\leq$ 200 ms p95** (SC-005). Semantic embedding measured **18.2–21.0 s** under load, so type-ahead **MUST** be lexical; FTS5 prefix measured **p95 9.58 ms**.
- **Search results $\leq$ 2 s p95** (SC-006).

- **≥ 90% of a ≥20-query benchmark returns the expected passage in the top 5 (SC-007).**
- **≥ 80% success on queries sharing NO literal words with the target (SC-008).**
- **Transcript accuracy measured over ≥ 30 sampled 30-second AUDIO-TIMELINE windows (SC-002).**
- **Timestamp lands within 5 seconds of the spoken content (SC-003).**
- **100% of citations genuinely support their claim, verified over ≥ 20 answers (SC-009).**
- **≥ 10 unanswerable questions declined 100%, 0 fabricated (SC-010).**
- **Fresh clone to running curriculum in under 15 minutes (SC-004).**
- **A new chapter added in under 30 minutes hands-on, zero code or config change (SC-011).**
- **100% of checks carry a paired failure proof (SC-012) and 100% distinguish could-not-determine from pass and fail (SC-013).**
- **Zero active server-side CI workflows (SC-014, §11.4.156 — “No escape hatch”).**
- **WCAG 2.1 Level AA, zero Level A/AA violations, search fully keyboard-operable (SC-017).**
- **No chapter content is exported, published or served until that chapter’s redaction review has been recorded (FR-039; contracts/passage-contract.md §7.3 R7; contracts/pipeline-cli.md §4.6, §4.7 B5). Recording “none required” is an explicit, valid decision; skipping the review is not. FR-039 is measured by **no success criterion** — it is enforced only by gate G-PID-5 and publish precondition B5, so dropping either leaves the obligation decorative. This is a live obligation and not a hypothetical: the Chapter 1 recording features an identifiable third party.**
- **~~The symbol matching key that code passage identity rests on has NO CONFIRMED PRODUCER P-U1 IS SETTLED AND THIS CONSTRAINT IS DISCHARGED — measured 2026-09-03. The heading is struck rather than deleted so a reader can see it was closed by work, not by editing. (contracts/passage-contract.md §10, P-U1). §6.2 keys code identity on a symbol path, and Lumen exposes no symbol table through its CLI or MCP surface (measured: no tool returns chunk identifiers or symbols), so the 58,726-symbol figure comes from its internal store rather than from a supported interface. T015, T016 and T017 are conditional on T014 settling P-U1 and MUST NOT be started on the assumption that a producer exists. If T014 exits 1 — both candidate producers tested, neither yields a stable symbol path — then SC-016a is not implementable as specified, and that is an operator decision to record, not something~~**

to work around: `bash workshop/pipeline/detect_symbols.sh` now exits 0 and NAMES the producer: `pipeline/symbols`, a go/parser extractor OWNED BY THIS FEATURE at `workshop/pipeline/symbols/` — T014's option (b). Lumen's store was not merely unsupported, it was measured UNFIT and is REJECTED: 988 duplicate (`file_path`, `symbol`) keys across 15,108 Go rows (the key does not identify a passage) and 0 of 3,104 Go method rows carry a \$6.2-shaped receiver, 381 rows fusing several declarations under one label. The chosen producer's key is unique across 2,329 symbols of a real tree, byte-stable across reruns, survives a body edit / an insertion / a reorder, and MOVES on a rename so \$8 R4 can see it; --prove-failure catches 5 of 5 seeded defects. Evidence is tracked at `workshop/evidence/p-u1/`. The "SC-016a is not implementable as specified" branch DID NOT HAPPEN and must never be cited as though it had. T015, T016 and T017 are ordinary unwritten work today. The one outcome forbidden either way is a heuristic that re-points a citation quietly.

- **Container orchestration MUST consume `git@github.com:vasic-digital/containers.git` as a git submodule** and MUST extend rather than reimplement it (§11.4.76(1),(4)).
- **Rootless container runtime** (§11.4.161). Host has podman 5.7.1; **docker is absent**.
- **Local/internal only** (decision D1). No content leaves the machine when a local provider is configured (FR-024).
- **Corpus is the workshop/ module plus the vasic monorepo** (decision D2). Nothing outside this working tree.
- **EVERY component and service MUST be fully decoupled and reusable** (operator directive, 2026-09-01; FR-043..FR-045, SC-019). Concretely: no Go `internal/` package for anything reusable — `internal/` is importable only from within its own module, so it forecloses reuse by language rule. Reusable units go in public packages, and where the reuse is genuine they become their own module, following the in-repo precedent `digital.vasic.containers` (submodules/containers, consumed via `replace` in development and a pinned SHA in production). The test is concrete: a component that must import a curriculum type to function is not decoupled.

File Structure

Decomposition decisions locked in here. Files that change together live together; split by responsibility, not by technical layer.

Path	Responsibility	Where it actually is — measured 2026-09-02
submodules/passage/ pkg/passage/	pid minting, source anchors, registry, resolution, redaction log, code-symbol identity. The contract every other unit depends on , and reusable on its own — it knows nothing about curricula	AS NAMED. Holds pid.go, anchor.go, registry.go, decoupling_test.go + tests.resolve.go, redaction.go, symbol.go, ingest_match.go, symbol_test.go are ABSENT — resolution landed inside registry.go; the other three are unbuilt (T012, T015, T016, T017)
submodules/verdict/ pkg/verdict/	the 0/1/2 three-valued result type, used by every command and endpoint	AS NAMED (verdict.go)
workshop/platform/ backend/pkg/ transcript/	immutable machine layer + append-only correction overlay	[PATH NOT BUILT] — the directory does not exist and no equivalent landed anywhere (T031)
workshop/platform/ backend/internal/ store/	chapter/material persistence. Deliberately internal — this is the one genuinely workshop-specific unit; reusing it would mean reusing this curriculum's schema, which no other consumer wants (FR-043 asks for decoupling, not for exporting everything)	[PATH CORRECTED] → workshop/platform/ backend/pkg/ curriculum/ curriculum.go (type Store at :168), with its HTTP layer at internal/api/ chapters.go. Note the consequence for this row's own argument: the unit shipped in pkg/, not internal/

Path	Responsibility	Where it actually is — measured 2026-09-02
workshop/platform/ backend/pkg/media/	local range-serve with seek	[PATH CORRECTED] → workshop/platform/ backend/internal/api/ recording.go (http.ServeContent at :786, Accept- Ranges :769, 416 Content-Range :724) AS NAMED.
workshop/platform/ backend/pkg/search/	lexical leg, semantic leg, and the three- state verdict that fuses them	lexical.go, semantic.go, service.go present; the three-state type is in envelope.go, not the verdict.go T055 names
workshop/platform/ backend/pkg/index/	generations, verification gate, atomic swap	AS NAMED (generation.go)
workshop/platform/ backend/pkg/ crossref/	passage relationships, cycle- safe traversal	AS NAMED
workshop/platform/ backend/pkg/answer/	provider seam + the four grounding layers	AS NAMED (provider.go, extractive.go, ollama.go, schema.go, verify.go, pipeline.go)
workshop/pipeline/ transcribe/	VAD, chunking, ASR driver, confidence, checkpointing, coverage	[PATH NOT BUILT] — the directory does not exist. What shipped is FLAT under workshop/ pipeline/: VAD → audio_energy.py; ASR driver → run_faster_whisper.py; confidence emission → the same file; atomic write → the same file

Path	Responsibility	Where it actually is — measured 2026-09-02
workshop/pipeline/accuracy/	frozen normaliser + WER scorer + its mutation proof	:179–:184. Chunking and checkpoint/resume are unbuilt; the coverage identity landed in workshop/platform/backend/pkg/curriculum/curriculum.go [PATH NOT BUILT] — the directory does not exist. The normaliser is workshop/pipeline/compare_engines.py (norm_word :52, align :113); the scorer is workshop/scripts/verify-accuracy.sh over it plus workshop/pipeline/audit_windows.py; the mutation proof is that script's --selftest
workshop/platform/backend/testdata/benchmark/	the retrieval query set and the answering answerable/unanswerable set — both , in one directory, beside the gates that read them (see the benchmark-location correction below)	AS NAMED, but holds questions.tsv ONLY. retrieval.tsv (T064) is [PATH NOT BUILT] , so the “both” this row promises is one
workshop/platform/frontend/src/app/features/	chapters, transcript, search — one directory each	AS NAMED, and wider: areas ask chapters curriculum plans practice progress search transcript
workshop/scripts/	control-plane scripts over a Go adapter	AS NAMED for the control plane

Path	Responsibility	Where it actually is — measured 2026-09-02
	consuming submodules/ containers' API (not its cmd/boot — see T021), plus add-chapter.sh and redact.sh	(build.sh, start.sh, stop.sh, status.sh, restart.sh, _common.sh, _capabilities.sh, extract-videos.sh, ingest.sh, verify.sh, verify-accuracy.sh; the adapter is workshop/platform/orchestration/cmd/workshop-boot/main.go). [PATH NOT BUILT] for add-chapter.sh (T084), redact.sh (T038/T039), transcribe.sh (T111), index.sh (T115), crossref.sh (T116), bench-suggest.sh/bench-search.sh (T065), bench-answers.sh (T078 — it shipped elsewhere), bench-retrieval.sh (T118) AS NAMED for quickstart.md, user-guide.md, manual.md, faq.md, limits.md. Training shipped as the DIRECTORY
workshop/docs/	quickstart, user guide, manual, FAQ, training, and the honest-limits page	workshop/docs/training/, not training.md; the extension prompt as workshop/docs/prompts/add-a-chapter.md. search.md (T067) and answering.md (T083)

Path	Responsibility	Where it actually is — measured 2026-09-02
workshop/evidence/	machine evidence, retained with the commit that produced it	are [PATH NOT BUILT] EXISTS but holds 3 tracked files. The machine evidence FR-040 contracts landed at workshop/ platform/backend/ evidence/ — 21 tracked files. workshop/ platform/qa/evidence/ exists on disk with 0 tracked files. See T099

Deliberately NOT created: a bespoke Containerfile or compose stack. §11.4.76 makes that a violation, not a shortcut — orchestration is consumed from the submodule.

FILE PATHS RE-MEASURED 2026-09-02 — this feature was built BY HAND, and the paths moved

The 2026-09-01 block below corrected twelve paths. It was not the whole of it. This feature was implemented by hand rather than through speckit-implement, and a second, larger set of paths diverged from what the tasks name. **A path-existence check over this file therefore misjudges tasks in both directions:** it calls finished work unbuilt (the file moved) and it can read an existing sibling directory as evidence for a file that was never written.

Two markers now separate the two states, and they must never be blurred:

- **[PATH CORRECTED]** — the requirement is satisfied; only the location differs. The real path is written in the task line and the superseded one is named beside it so a reader can tell a corrected path from one that was never wrong. **Nothing about the requirement changed.**
- **[PATH NOT BUILT]** — the named path does not exist AND no equivalent work landed anywhere. The path is left exactly as written, because it is still the destination.

Ticks were not touched by this sweep, except for the five ids independently re-measured as complete on 2026-09-02 (T049, T063, T065, T101, T102). Correcting where a file is says nothing about whether the task is done; those are separate questions and this block answers only the first.

Re-derive the whole sweep rather than trusting it — every path below was checked with `ls/test` and every line number with `grep -n:`

```
grep -ohE '[a-zA-Z0-9_./-]+\.(go|py|sh|ts|md|json|tsv|
txt)' \
specs/001-workshop-curriculum-platform/tasks.md \
| tr -d '"' | sort -u | while read -r p; do
    [ -e "$p" ] && echo "EXISTS  $p" || echo "ABSENT  $p"
done
```

Note the residual limit (§11.4.6): that loop resolves paths relative to the repository root, so a bare filename or a path fragment quoted in prose reports ABSENT without that being a finding. It is a starting point for re-measurement, not a verdict.

FILE PATHS CORRECTED 2026-09-01 — **passage and verdict are root submodules, not backend packages**

`workshop/platform/backend/pkg/passage/` and `workshop/platform/backend/pkg/verdict/`: Both paths are **withdrawn**, struck through rather than deleted so a reader can tell a corrected path from one that was never wrong. **Twelve task file paths moved** — two rows in the table above and T008–T013, T015–T018. **No task was renumbered**. Other documents and agent reports cite these T-numbers, and renumbering would silently invalidate every one of those citations; the file is still T001–T120, contiguous, and that invariant is verified by parsing the file, not by counting.

- *Believed when written (2026-08-31)*: both units would live under the backend module's `pkg/`, which was already the *right* call in spirit — the paragraph below on `pkg/` versus `internal/` is the reasoning that produced it, and it was correct. It just did not go far enough.
- *Measured 2026-09-01*, re-derivable in one line each:

```
git config -f .gitmodules --get-regexp
    'submodule\.submodules/(passage|verdict)\.(path|
url)'
# submodule.submodules/verdict.path submodule/verdict
```

```
# submodule.submodules/verdict.url git@github.com:vasic-
digital/verdict.git
# submodule.submodules/passage.path submodules/passage
# submodule.submodules/passage.url git@github.com:vasic-
digital/passage.git
head -1 submodules/passage/go.mod # module github.com/
vasic-digital/passage
head -1 submodules/verdict/go.mod # module github.com/
vasic-digital/verdict
ls workshop/platform/backend/pkg/ # answer crossref
embed index search – no passage, no verdict
```

Both are now **separately published public repositories**, mounted as submodules at the project root and consumed by the backend through go.mod require + replace, exactly as digital.vasic.containers is. workshop/platform/backend/pkg/ no longer contains either.

- *When it changed:* during this feature's own implementation work, after the table was written.
- *What still holds:* everything the pkg/-not-internal/ paragraph below argues. Promotion to a separate module is that argument carried to its conclusion, not a departure from it — and the root placement is required, because §11.4.28 forbids nesting a submodule inside a submodule.
- *What is NOT claimed:* that the files those tasks name now exist. submodules/passage/pkg/passage/ currently holds pid.go, anchor.go, registry.go and their tests. resolve.go, redaction.go, symbol.go and ingest_match.go are **not there** — T011, T012, T015 and T016 remain unbuilt work. Only their destination changed.

BENCHMARK LOCATION DECIDED 2026-09-01 — one directory, one format, beside its gate

~~workshop/pipeline/benchmark/(queries.yaml,unanswerable.yaml) in this file; workshop/platform/qa/(retrieval-benchmark.jsonl,answerable-20.jsonl,unanswerable-10.jsonl) in [quickstart.md](#).~~ Both are **withdrawn**. The canonical location is **workshop/platform/backend/testdata/benchmark/**, TSV, and both documents now say so.

- *Measured 2026-09-01* — the decisive fact is that neither disputed path exists:

```
ls workshop/pipeline/benchmark/
    # No such file or directory
ls workshop/platform/qa/
    # No such file or directory
ls workshop/platform/backend/testdata/
    benchmark/          # questions.tsv
grep -n 'QUESTIONS=' workshop/platform/backend/gates/
    bench-answers.sh
#   QUESTIONS="${QUESTIONS:-$HERE/../../testdata/benchmark/
    questions.tsv}"
```

Two paths were argued over for a year of document-time and **neither was ever created**. A third location shipped, wired to a working gate, while the argument was in progress. That settles it: the only candidate with a producer, a consumer and bytes on disk wins.

- *Why this location:* it is where the gate that reads it already looks; testdata/ is Go's own convention for fixtures a test or gate consumes, and the toolchain excludes it from builds; and it puts the retrieval set beside the answering set instead of splitting two benchmarks of the same corpus across two trees. workshop/pipeline/ is the transcription pipeline — it neither produces nor consumes either benchmark, so the original placement filed them by topic rather than by who reads them.
- *Why TSV and not YAML or JSONL:* the shipped file is TSV with a class<TAB>question schema and a comment header, matching platform/gates/route-manifest.tsv. One format across the tree's fixtures, line-diffable in review, and parseable by awk in a gate with no dependency.
- *Why one file per benchmark rather than one file per class:* answerable-20.jsonl / unanswerable-10.jsonl encode their counts **in their filenames**, which are wrong the moment a twenty-first question is added, and two files can drift apart. A class column cannot.
- *What is NOT fixed by this decision, and must not be read as fixed:* ~~questions.tsv currently holds 8 A rows and 10 U rows. SC-010 needs ≥10 unanswerable — met. SC-009 needs ≥20 answerable — 8 is not 20, and the shortfall is real.~~ **WITHDRAWN, not restated — the count moved.** Re-measured 2026-09-02 with the command this bullet already carried:

```
awk 'NR>1 && !/^#/ && NF' workshop/platform/backend/  
testdata/benchmark/questions.tsv \  
| cut -f1 | sort | uniq -c  
# 24 A  
# 33 U
```

24 A and 33 U. SC-009's *count* floor (≥ 20 answerable) is **met**; SC-010's (≥ 10 unanswerable) was already met and is now met three times over. **What is short is the CERTIFICATION, not the count** — SC-009 requires 100% of citations to genuinely support their claim over those answers, and that is T079's open work, not T077's. Canonicalising the path says where the file lives; it says nothing about whether its contents satisfy the criteria. T064 remains open work.

- *Also unbuilt:* workshop/scripts/bench-retrieval.sh (T118) does not exist, and no retrieval benchmark file exists at any path. Only the answering half has shipped.

pkg/ not internal/, and why it is not cosmetic. Go's `internal/` is importable ONLY from within its own module — by language rule, not convention. Placing a component there forecloses reuse permanently, no matter how cleanly it is written. FR-043 to FR-045 therefore put every reusable unit in `pkg/`. The decoupling test is concrete and checkable: a component that must import a curriculum type to function is not decoupled, and SC-019 requires each to build and pass its tests with the curriculum absent.

Where reuse is genuine rather than theoretical, a unit should become its own module, following the in-repo precedent `digital.vasic.containers` (submodules/containers) — consumed via replace during development and a pinned SHA in production. ~~The counter-example is equally instructive: `LLMProvider` has 43 adapters and the right interface shape but an unresolvable module path and a relative-path replace on a sibling checkout, so nothing can consume it. Reusable in intent, unusable in fact.~~

CLAIM WITHDRAWN 2026-09-02 — `LLMProvider` is no longer the counter-example; it is consumed. The struck-through sentence is left visible rather than deleted so a reader can see that the counter-example was retired by work, not by editing. `LLMProvider` and `RAG` both joined the owned submodule fleet at the repository root and the backend now requires and replaces both. Re-derive:

```
grep -n 'llmprovider\|vasic.rag' workshop/platform/backend/
go.mod
# digital.vasic.llmprovider v0.0.0
# digital.vasic.rag v0.0.0
# replace digital.vasic.rag => ../../../../submodules/RAG
# replace digital.vasic.llmprovider => ../../../../
submodules/LLMProvider
```

The replace targets are now submodules declared in this repository's .gitmodules, not a sibling checkout absent from it — which is precisely the difference between “unusable in fact” and “consumed”. **What is NOT claimed:** that the 43-adapter figure was ever re-measured, or that every adapter works. Only the module path and the consumption are measured here.

Task Format

[ID] [markers] [Story] Description

Markers: [P] parallelisable · [TDD] RED-GREEN-REFACTOR · [REVIEW] review before proceeding · [SUBAGENT] delegable

Numbering is permanent and ids are never reused or renumbered.

Other documents and agent reports cite T-numbers, so renumbering would silently invalidate every one of them. Tasks added after the original T001–T105 therefore take new contiguous ids and are appended to the **phase whose subject they belong to**, which means ids ascend within each phase's appended block rather than monotonically down the file. Read the phase, not the number, to know when a task runs.

Path Conventions

The feature lives inside the workshop/ submodule, mirroring ai_interviewing/platform's backend/ + frontend/ + bin/ split (plan.md Structure Decision). Container orchestration is consumed from submodules/containers — **never reimplemented** (§11.4.76(4)).

Standing rules for every task in this file

These come from measurement, not preference. Violating any of them produces work that looks done and is not.

1. **No check may be added without a paired mutation proof that it FAILS when its guarded condition is broken** (FR-032, SC-012, §1.1). Two gates in this repository have already shipped with inoperative proofs — one whose control failed so zero mutations ever ran, one that exercised only sandboxed copies while the real entry point could not start. Every proof **MUST** include a case that runs the real entry point against the real tree.
2. **Three-valued exit everywhere**: 0 fine · 1 a real problem found · 2 could not determine. A missing dependency, unreachable backend or crashed helper is always 2. This conflation has been found and fixed four times here.
3. **An assertion that greps for a string is not a test**. It must execute the behaviour and check the observable result — this repo has shipped an assertion that stayed green while the thing it tested was deleted.
4. **No CI may be added, including inside workshop/** (FR-034, §11.4.156). Gate E now sweeps the submodule fleet, so an attempt will be caught.
5. **No frozen host assumptions**. Derive paths, tool locations and tuned values; provide env overrides. ffp_{probe} is NOT installed here (it is a Playwright symlink), so the pipeline must detect its media tooling rather than assume it.
6. **A task MUST NOT assume a capability nobody has verified exists**. Where a task depends on an entry in an unverified register, it names the entry and names the task that settles it. T014 and the P-U1 constraint above are the worked example.
7. **sc-### identifiers COLLIDE with feature 002 — always name the spec before citing one as acceptance**. Added 2026-09-03, after the ambiguity produced a wrong acceptance claim. The two specs number their success criteria independently and the same number means different things:

id	this spec (001)	spec 002
SC-009		

id	this spec (001)	spec 002
SC-010	100% of answer citations genuinely support the claim on ≥ 10 unanswerable questions, fabricates none — NOT met (see T078)	a media-backed citation lands inside its cited span identifier survival across correction and insertion — proven there under its T053

Re-derive before citing, never from memory:

```
grep -n '^-\ \*\*SC-010\*\*' specs/001-workshop-curriculum-
platform/spec.md \
specs/002-knowledge-areas-deep-
linking/spec.md
```

“SC-010 is proven” is TRUE of 002 and FALSE of this spec. A bare SC-010 in a note, a commit message or an agent report is therefore not a citation — it is an ambiguity, and it has already been resolved the wrong way once. The same applies to SC-009.

Phase 1: Setup (Shared Infrastructure)

Purpose: make the module buildable and its dependencies real.



T001 Create the module skeleton workshop/platform/{backend,frontend,bin} and workshop/{curriculum,pipeline,docs,evidence} per plan.md Structure Decision — note NO workshop/containers/ directory: orchestration is consumed from submodules/containers, and a local container stack would violate §11.4.76(4) (FR-013)



T002 Initialise the Go module in workshop/platform/backend/go.mod (Go 1.26.2, matching ai_interviewing/platform/backend/go.mod) (FR-013)



T003 Add replace digital.vasic.containers => ../../../../submodules/containers to workshop/platform/backend/go.mod per §11.4.76(2), and verify it builds (FR-046)



T004 [P] Scaffold the Angular client in workshop/platform/frontend mirroring ai_interviewing/platform/frontend conventions (package name, Karma/Jasmine, no new UI framework) (FR-013)



T005 [P] Create workshop/pipeline/requirements.txt pinning faster-whisper + ctranslate2, and workshop/pipeline/venv-setup.sh building a project-local venv — **[TICKED 2026-09-04. The [PATH NOT BUILT] note is WITHDRAWN BY NAME: workshop/pipeline/venv-setup.sh now EXISTS, and the venv build is no longer prose in a header.** requirements.txt was already correct and is unchanged — it pins faster-whisper==1.2.1 + ctranslate2==4.8.2 across 23 == pins. The new script makes the four-line procedure that was documented in that file's header EXECUTABLE and CHECKABLE, which is the half that was open. The load-bearing evidence is a REAL FRESH BUILD, not a dry run: bash workshop/pipeline/venv-setup.sh --venv /tmp/vsbuild built a venv from nothing and exited 0 in 31.86 s wall (time, 20.58 user / 1.62 sys), installing all 23 pinned packages from wheels only (--only-binary=:all:, which the script offers no flag to relax) and then IMPORTING faster_whisper and ctranslate2 — because a wheel that installs is not a wheel that runs, which is research item U1. torch absent, as required. The throwaway prefix was removed afterwards. Three-valued exit DEMONSTRATED, all four states driven 2026-09-04: --verify against the real hand-built venv -> 0 (23 of 23 installed at their pinned versions); --verify --venv /nonexistent/nv -> 2; --python /nonexistent/py -> 2; --bogus -> 2; --verify against an EMPTY venv -> 1 (these modules do not import: faster_whisper ctranslate2). A version-drift check compares every == pin against importlib.metadata.version so an import that succeeds against the WRONG version is a failure, not a pass. The script resolves the interpreter INSIDE the venv by absolute path and never consults PATH for an engine — /usr/bin/whisper on this host is a GTK notification client, so command -v whisper succeeds and proves nothing. R5 was exercised in BOTH directions, which is the evidence that the sweep discriminates rather than merely reddening: immediately after the file was created bash workshop/platform/gates/verify-check-registry-001.sh went to rc 1 with**

FAIL [R5] UNREGISTERED — pipeline/venv-setup.sh is under a declared scanroot but appears in no check, sibling proof, debt, or exempt row; after an exempt row was added it returns rc 0 at 4 0 PASS / 0 FAIL / 3 DEBT / 0 UNDET, sweeping 37 files (was 36). It is exempt, not check, on a stated reason: it PROVISIONS an

environment and asserts nothing about this module, and a host without wheels for its interpreter is not a finding about this curriculum. Honest boundary (§11.4.6): this does NOT close T051. T051 needs a timed fresh-clone-to-running run against SC-004's 15-minute budget; what is now measured is that ONE step of it — the venv — costs 31.86 s on this host with a warm pip cache, and is reproducible by a fresh clone at all, which it was not before] (population: in_process) (FR-012, SC-004)



T006 [P] Configure linting/formatting to match the reference module (gofmt, Angular ESLint config) — **[UNWRITTEN, AND THE TASK CONTRADICTS ITSELF. Measured 2026-09-03; NEW note.** Neither half is configured. Frontend: workshop/platform/frontend/ has no .eslintrc* and no eslint.config.*; package.json declares 11 scripts and none is lint; its devDependencies contain no package whose name matches lint or prettier. Go: nothing in workshop/ runs gofmt as a gate, and gofmt -l workshop/platform/backend reports 3 unformatted files today (cmd/knowledge-registry-backfill/main.go, internal/api/publication.go, internal/api/passages_publication_test.go), which is a live demonstration that no gate is enforcing it. The self-contradiction, and it needs an operator to resolve rather than an agent to guess: this task says “match the reference module”, and the reference is ai_interviewing/platform/frontend (plan.md Structure Decision) — measured, that module has no ESLint config either: 5 scripts, none lint, zero lint devDependencies. So “match the reference” and “Angular ESLint config” point in opposite directions and cannot both be satisfied. Adopt ESLint and DIVERGE from the reference, or match the reference and drop the ESLint half; either is defensible, neither is this note’s to choose. The gofmt half has no such ambiguity and is unblocked work] — BLOCKER: the task contradicts itself — it says “match the reference module”, and the reference (ai_interviewing/platform/frontend) has NO ESLint config either, so there is nothing to match · OWNER: operator — pick a lint baseline, or withdraw the “match the reference” clause (FR-013)**



T007 [REVIEW] Verify workshop/ still adds zero CI: git -C workshop ls-files | grep -E '^\.github/workflows/.*\.ya?ml\$' must be empty, and bash scripts/pre-push-gates.sh gate E must pass (FR-034, SC-014)

Execution notes: T003 is the §11.4.76 obligation — the build must consume the submodule, not a vendored copy. Verify the module builds before proceeding.

Phase 2: Foundational (Blocking Prerequisites)

Purpose: the passage contract, resolution, redaction identity and environment detection. **No user story can begin until this phase is complete** — every story depends on the passage identity contract, and everything that points at content resolves through the one function built here.



T008 [TDD] [REVIEW] Implement the ULID passage identifier minter in `submodules/passage/pkg/passage/pid.go` per [contracts/passage-contract.md](#) — minted at ingest, never positional, never content-derived (FR-037, SC-016, FR-044, FR-047)



T009 [TDD] Implement the `<!-- pid: ... -->` source anchor reader/writer in `submodules/passage/pkg/passage/anchor.go` (FR-037, SC-016)



T010 [TDD] Implement the passage registry (`passages.jsonl` → `pages.db`) in `submodules/passage/pkg/passage/registry.go`, with `content_hash` as a change-detection column explicitly documented as NOT identity (FR-037)



T011 [TDD] **[PATH CORRECTED 2026-09-02 — this task named `submodules/passage/pkg/passage/resolve.go`, which does not exist; the resolver landed in `registry.go` as `func (r *Registry) Resolve(p PID) Resolution` at :462, with the four Outcome constants at :299–:308]** Implement the four-outcome resolver in `submodules/passage/pkg/passage/registry.go` per [contracts/passage-contract.md](#) §7.1 — `found · redacted · not_in_registry · undetermined`, with NO fallback to fuzzy text matching, nearest-neighbour lookup, prefix matching or same-`content_hash` lookup. `undetermined` is NEVER collapsed into `not_in_registry`: collapsing them makes an unreadable database look like a curriculum that never contained the passage. Every citation, cross-reference and redaction resolves through this one function, and there is no second path (FR-037, FR-033, SC-016a)



T012 [TDD] [TICKED 2026-09-03 ON MEASUREMENT. PATH NOT BUILT—~~redaction.go does not exist ... there is no Redact/Unredact API anywhere. The WRITER is what remains~~ — that whole 2026-09-02 note is WITHDRAWN BY NAME, not deleted: every clause of it is now false. `submodules/passage/pkg/passage/redaction.go` EXISTS at exactly the path this task names, 472 lines, and it IS the writer. Measured: `grep -n '^func ' pkg/passage/redaction.go` returns the full API — `LoadRedactionLog :173, (*RedactionLog).Append :256, AppendFile :315, Materialise :349, Purge :401 ($7.3 R6), SyncDB :438, plus State :212, Suppressed :224, SuppressedPIDs :234`. The file's own header states the contract this task asks for — the bool is the MATERIALISATION of the log's latest entry, never the mechanism, and `unredact` is an explicit append rather than the deletion of the `redact` entry before it. The [TDD] half is proven, not asserted: `go test -count=1 ./... in submodules/passage` is ok, and `-run RED -v` shows 11 gate tests `G-RED-1..G-RED-6` all PASS, of which 5 are paired mutations — `TestGateRED1_PairedMutation_RewriteInsteadOfAppend` is the “append, not a rewrite” clause and `TestGateRED4_PairedMutation_ColumnFlipWithoutRebuild` is the “a bare column flip is a contract violation” clause, each named by this task. R2 is separately confirmed against the REAL Chapter 1 corpus at 10 redacted rows × 2 fields × 3 surfaces, R2 violations 0 (see T040)]

Implement the append-only redactions log and the passages. `redacted` materialisation in `submodules/passage/pkg/passage/redaction.go` per [contracts/passage-contract.md](#) §4.3 (redactions DDL) and §7.3 R2/R6 (FR-039) — a bare column flip is a contract violation, `unredact` is a new append rather than a rewrite of history, and once a pid is redacted its `text` and `machine_text` are absent from every serialisation at every layer. This is the registry half of FR-039; propagation is T059, T061 and T075, and the gate is T076 (population: `in_process`) (FR-039)



T013 [TDD] Prove the two survival guarantees in `submodules/passage/pkg/passage/pid_test.go` as gates **G-PID-1** and **G-PID-2** ([contracts/passage-contract.md](#) §5.1, §5.2). **G-PID-1**: create a passage, cite it, correct its text, re-run ingest and re-index, then assert the pid is unchanged, the citation still resolves, `machine_text` is byte-identical to before, and `content_hash` **did** change. **Paired mutation**: make ingest re-mint on `content_hash` change — i.e. reproduce the measured Lumen behaviour; the gate **MUST** go red, and the contract names this the one mutation that must never be skipped, because it is the precise defect the whole contract exists to prevent.

G-PID-2: reproduce the measured D-SEARCH-1 run 3 by prepending a section to a transcript so every later section shifts by 4 lines, then assert every pid is unchanged **and** every affected source_ref was updated to the new lines — source_ref is a cache, never a key. **Paired mutation:** key ingest matching on (path, line_start); the gate MUST go red. (This is SC-016; both alternatives were measured to fail — see research.md D-SEARCH-1) (FR-037, SC-016, FR-032, SC-012)



T014 [REVIEW] [TICKED 2026-09-03 ON MEASUREMENT — P-U1 IS SETTLED AND THE PRODUCER IS NAMED. ~~PATH NOT BUILT — workshop/pipeline/detect_symbols.sh does not exist and no equivalent landed anywhere; measured 2026-09-02, no P-U1 outcome is recorded under workshop/evidence/ either~~ — that note is WITHDRAWN BY NAME: both halves are false today. workshop/pipeline/detect_symbols.sh exists (31,112 B, executable) and a default run exits 0, printing PRODUCER: pipeline/symbols (go/parser, owned by feature 001) — this task's option (b). Candidate (a) (Lumen's SQLite store) is REJECTED on evidence, not on preference: A3 988 duplicate (file_path, symbol) key(s) across 15108 Go row(s) — the key does not identify a passage, A4 0 of 3104 Go method row(s) carry a §6.2-shaped receiver. Candidate (b) passes every check: B1 built · B2 emits §6.2's package-prefixed, receiver-qualified path · B3 (path, symbol) unique across 2329 symbols of a real tree · B4 byte-identical across two runs · B5 survives a body edit, an insertion and a reorder · B6 a rename MOVES the key so §8 R4 can see it. Three-valued exit demonstrated in both directions: default 0; --real-tree /nonexistent → 2 with A5/A6 NOT_TESTED and B3 UNDETERMINED. §1.1 paired proof: --prove-failure → rc 0, PROVE: 5 mutations, 5 caught, 0 missed (M1 duplicate key → B3 FAIL, M2 nondeterministic → B4, M3 position-encoded → B5, M4 rename-blind → B6, M5 non-§6.2 shape → B2). Evidence recorded and TRACKED under workshop/evidence/ as this task requires: evidence/p-u1/README.md (line 7: "*Outcome: exit 0. The producer is pipeline/symbols*") and evidence/p-u1/result.json, both in git ls-files. The producer itself is workshop/pipeline/symbols/ (main.go, go.mod). Registered as check symbol-index-honesty in workshop/platform/gates/check-registry-001.tsv. ****Honest boundary (§11.4.6):** the TRACKED evidence/p-u1/result.json in the working tree currently holds the output of that registry's --real-tree /nonexistent UNDET PROBE ("exit": 2), so the JSON and the README disagree — the README carries the settlement, the JSON carries a probe artefact. A5/A6 remain NOT_TESTED because

--reindex-probe writes to a shared store and was not run. Neither touches the verdict: candidate (a) is rejected on A3/A4 alone, and (b) is confirmed on B1–B6. T015, T016 and T017 are therefore UNBLOCKED — they are unwritten work, not blocked work] Settle P-U1 before any code-identity work begins ([contracts/passage-contract.md](#) §10): §6.2 keys code identity on a symbol path, but **that key has no confirmed producer** — Lumen exposes no symbol table through its CLI or MCP surface, so the 58,726-symbol figure comes from its internal store, not from a supported interface. Choose and prove exactly ONE producer in workshop/pipeline/detect_symbols.sh: either (a) read Lumen’s SQLite store directly and confirm the symbol schema survives a reindex unchanged, or (b) extract symbols with a Go/TypeScript parser owned by this feature. Three-valued exit: 0 a producer is confirmed and named · 1 both candidates were tested and neither yields a stable symbol path · 2 neither could be tested. Record the outcome and its evidence under workshop/evidence/. **T015, T016 and T017 are blocked until this exits 0** — see Global Constraints (population: in_process) (SC-016a, FR-033, FR-040, FR-032, SC-012)



T015 [TDD] [PATH NOT BUILT — `symbol.go` does not exist. Measured 2026-09-02, only the SCHEMA landed, in `registry.go`: `Symbol *string at :73, CREATE UNIQUE INDEX ... passages_symbol_key ON passages(path, symbol) at :934, CREATE TABLE ... symbol_aliases at :966. No code reads or writes symbol_aliases — registry.go:1091 names it among the tables this package never writes — so the behaviour this task contracts is still owed at the path named here] Implement the code-passage matching key (path, symbol) and the symbol_aliases table in submodules/passage/pkg/passage/symbol.go per contracts/passage-contract.md §6.2, §6.4 and the §4.3 symbol_aliases DDL (SC-016a) — origin separates authored (written by the maintainer performing the rename; authoritative) from detected (ingest’s heuristic, which can mis-attach two similar helpers), so a reviewer can see exactly which links rest on a guess. Conditional on T014 — UNBLOCKED 2026-09-03. T014 now exits 0 and names the producer (pipeline/symbols, a go/parser extractor owned by this feature, at workshop/pipeline/symbols/); see T014’s own note for the A3/A4 rejection of Lumen’s store and the B1–B6 confirmation of this one. This task is therefore UNWRITTEN WORK, not blocked work, and the block that used to excuse it is gone — BLOCKER: none but the work — only the SCHEMA landed in registry.go; symbol.go is unwritten · OWNER: implementer — unblocked today (SC-016a, FR-037)`



T016 [TDD] **[PATH NOT BUILT — ingest_match.go does not exist and no equivalent landed anywhere]** Implement the §8 ingest matching branches for code passages in submodules/passage/pkg/passage/ingest_match.go — R2 attach on (path, symbol) · R3 attach through symbol_aliases, recording WHICH alias was used so detected links stay reviewable · R4 an unaliased rename ⇒ **exit 1** for workshop/-owned code (§6.6 makes an authored alias a mandatory step of any rename inside the tree) and, for code outside workshop/, mint a new pid and record the orphaned old one so the broken citation is LOUD. Minting stays the last branch (M4). **Conditional on T014 — UNBLOCKED 2026-09-03. T014 now exits 0 and names the producer (pipeline/symbols, a go/parser extractor owned by this feature, at workshop/pipeline/symbols/); see T014's own note for the A3/A4 rejection of Lumen's store and the B1–B6 confirmation of this one. This task is therefore UNWRITTEN WORK, not blocked work, and the block that used to excuse it is gone — BLOCKER: none but the work — ingest_match.go is unwritten and no equivalent landed anywhere · OWNER: implementer — unblocked today (SC-016a, FR-037)**



T017 [TDD] **[PATH NOT BUILT — symbol_test.go does not exist; measured 2026-09-02 the identifier G-PID-4 appears nowhere in submodules/passage]** Prove SC-016a via gate G-PID-4 in submodules/passage/pkg/passage/symbol_test.go (contracts/passage-contract.md §6.5, §6.6): rename a workshop/-owned symbol WITH an authored alias and assert the pid is preserved and its citations still resolve; then rename one WITHOUT an alias and assert the old pid resolves not_in_registry — a loud dead link — and that ingest reports the unaliased rename as exit 1. **Paired mutation:** make unaliased renames mint a new pid silently and re-point the old citation by nearest-text match; the gate MUST go red. 100% of stale code references failing loudly is the entire criterion — a confident link to the wrong code is the failure it exists to prevent. **Conditional on T014 — UNBLOCKED 2026-09-03. T014 now exits 0 and names the producer (pipeline/symbols, a go/parser extractor owned by this feature, at workshop/pipeline/symbols/); see T014's own note for the A3/A4 rejection of Lumen's store and the B1–B6 confirmation of this one. This task is therefore UNWRITTEN WORK, not blocked work, and the block that used to excuse it is gone — BLOCKER: predecessor T015 + T016 — G-PID-4 tests code that does not exist yet · OWNER: implementer, after T015/T016 (SC-016a, FR-032, SC-012)**



T018 [P] [TDD] Implement three-valued exit helpers in submodules/
verdict/pkg/verdict/verdict.go (0 / 1 / 2) used by every command
and endpoint (FR-033, SC-013, FR-044, FR-047)



T019 [P] [TDD] **[PARTIAL — BUILT AND PROVEN, BUT ITS GATE
READS THE WRONG EXIT CODE AND THE CONTRACTED
OVERRIDE DOES NOT EXIST. Measured 2026-09-03; this is a NEW
note, the task previously carried none.**** workshop/pipeline/
detect_media.sh EXISTS (26,095 B, executable) and does exactly
what the first half of this task asks: it probes capability, never
presence. :27 — “*Never --version. Never command -v. Never [-x
]*” — with ffprobe exercised by -show_format -show_streams on a
s fixture (:192, :209) and ffmpeg by an actual decode. §1.1 paired
proof RUN: --prove-failure → rc 0,
PROVE: 4 mutations, 4 caught, 0 missed, and it opens with the
control this task’s own text demands —
PROVE: control – BROKEN fake PASSES a naive '--version'
presence check (rc 0). It is registered as check media-presence in
workshop/platform/gates/check-registry-001.tsv with a real rc-2
probe (--scratch-dir /nonexistent/... → rc 2, measured). The trap
this task exists to catch is live on this host and the tool catches
it: a plain run resolves ffprobe to /home/.../.cache/ms-playwright/
ffmpeg-1011/ffmpeg-linux, which answers --version and prints Un
recognized option 'show_format', and the probe reports ffprobe
UNUSABLE with that evidence. TWO MEASURED DIVERGENCES
FROM G-CLI-3, and they are the open half. (1) The exit code is
wrong against the contract. An unusable tool exits 1 (RESULT 1
UNUSABLE – a required tool cannot do its job), and mutations
M2/M3 assert rc=1. But contracts/pipeline-cli.md §3.2 P4 and
gate G-CLI-3 at :317 both require exit 2 with reason.code:
"ffprobe_unusable" — “*an unusable tool determines nothing*”,
this task’s own words. grep -n 'ffprobe_unusable\|reason.code'
workshop/pipeline/detect_media.sh returns zero. (2) WORKSHOP_FFP
ROBE / WORKSHOP_FFMPEG do not exist — grep -oE '\\${?}[A-
Z_]*FFPROBE[A-Z_]*' over that script returns nothing — so G-
CLI-3 as written (“*point WORKSHOP_FFPROBE at Playwright’s ffmpeg
binary*”) cannot be run at all, even though the situation it
describes is the host’s actual state. Which of 1 and 2 is right is a
real question and NOT a clerical one — standing rule 2 says a
missing or crashed dependency is always 2, while a tool that is
present and answers wrongly is arguably a determined finding
— but the contract, the gate and this task all say 2, and the code
says 1, so something must change and it is not this note’s call to
decide which. Until then G-CLI-3 is unsatisfied] Implement

media-tooling detection in workshop/pipeline/detect_media.sh — probe ffmpeg/ffprobe for actual capability, not --version; ffprobe here is a Playwright symlink that answers --version and rejects -show_format. Gate **G-CLI-3** ([contracts/pipeline-cli.md](#) §3.2, §5): point WORKSHOP_FFPROBE at Playwright's ffmpeg binary and assert exit 2 with a tooling reason, never 1 — an unusable tool determines nothing. **Paired mutation**: replace the capability probe with ffprobe --version; the gate MUST go red, because that binary answers --version happily and the probe would pass over a tool that cannot do the job — **BLOCKER**: none but the work — the script exists and runs; its gate reads the wrong exit code and the contracted override is unwritten · **OWNER: implementer** — unblocked today (FR-033, SC-013, FR-032, SC-012)



T020 [P] [TDD] Implement backend/model detection in workshop/pipeline/detect_backend.sh following scripts/lumen-reindex.sh's env → config → live probe → documented fallback ladder — **[PATH NOT BUILT — workshop/pipeline/detect_backend.sh does not exist. Measured 2026-09-02, PART of the substance landed at a different path, for the ANSWERING backend only: workshop/scripts/_capabilities.sh carries the env-override rung, a live daemon probe and a documented read-only fallback. The config-file rung and ASR-model-backend detection are absent, so this task is not satisfied by that file — read it before rebuilding, then build the missing rungs]** — **BLOCKER**: none but the work — detect_backend.sh is unwritten · **OWNER: implementer** — unblocked today (FR-023)



T021 [P] **[PARTIAL — ONE OF ITS TWO GATES IS BUILT AND PROVEN, THE OTHER EXISTS NOWHERE. Measured 2026-09-03; NEW note.** The scripts and the adapter are real: workshop/scripts/start.sh, stop.sh, status.sh and the Go adapter workshop/platform/orchestration/cmd/workshop-boot/main.go, which reports RUNNING/STOPPED/DEGRADED “or exit 2 when it cannot tell” (: 126). G-CLI-17 is DONE: bash workshop/platform/gates/verify-status-exit-contract.sh → rc 0, PASS=3 FAIL=0 UNDET=0 — A1 an absent project prints STOPPED and exits 1, A2 status.sh passes the control plane's code through unchanged, A3 RUNNING → 0 while STOPPED → 1 are distinguishable by exit code alone; and its §1.1 paired mutation prove-status-exit-contract.sh → rc 0, CAUGHT=2 MISSED=0, seeding MS1 (STOPPED exits 0 — the exact return nil regression this task names) and MS2 (STOPPED filed as 2), each proven to COMPILE and LINK before being judged, then the tree restored. G-CLI-12 IS UNBUILT, and so is the behaviour it**

would gate. `grep -rn 'G-CLI-12'` over `workshop/` returns zero files — no gate, no fixture, no registry row — and the branch itself does not exist: `grep -rn 'unidentif\|identity cannot\|port holder'` over `cmd/workshop-boot/main.go`, `status.sh` and `_common.sh` also returns zero. There is no path on which `status.sh` meets a published port held by a process whose identity cannot be established and answers 2; today that case is simply not distinguished from the DEGRADED/STOPPED negatives, which is the exact conflation standing rule 2 forbids. What remains: the unidentifiable-holder branch, its 2, and the paired mutation that makes it return 1] Create `workshop/scripts/{start,stop,status}.sh` over a Go consuming adapter that calls submodules/containers' Go API — consumption, not reimplementation (§11.4.76(4)). **Do NOT wrap cmd/boot:** measured 2026-09-01, that CLI hardcodes a helixagent endpoint, accepts no `ComposeFile`, and constructs `BootManager` with neither an orchestrator nor a health checker, so pointed at this project it **exits 0 having started nothing** — a silent false pass, which standing rule 2 forbids. The adapter constructs the manager with this project's compose file and health checker; anything the adapter needs that the submodule cannot express must be contributed upstream, not forked here. Gate **G-CLI-12** ([contracts/pipeline-cli.md](#) §4.8, §5): bind the published port with a process whose identity cannot be established and assert `status.sh` exits 2, not 1 — an unidentifiable holder is could-not-determine, while DEGRADED (containers exist but not all are running) is a determined negative and stays 1. **Paired mutation:** make the unidentifiable-holder branch return 1; the gate MUST go red. Gate **G-CLI-17** ([contracts/pipeline-cli.md](#) §4.8, §5) — **assigned to this task 2026-09-02 by operator decision**, because the contract itself calls it the sibling of G-CLI-12 on the same script: `status.sh` against a project with **no containers** prints STOPPED and exits 1, a determined negative distinguishable by exit code alone from RUNNING's 0. Reproduce with `WORKSHOP_PROJECT_NAME=<unused-name> bash workshop/scripts/status.sh`. **Paired mutation:** restore the return nil in `cmd/Status's len(statuses) == 0` branch — i.e. reproduce the behaviour measured on 2026-09-01; the gate MUST go red — **BLOCKER:** none but the work — one of the two contracted gates exists nowhere · **OWNER: implementer** — unblocked today (FR-012, FR-033, SC-013, FR-032, SC-012)



T022 [REVIEW] Review the passage contract implementation against [contracts/passage-contract.md](#) before anything consumes it — every other component depends on its shape — **BLOCKER: predecessor T015-T017** — the contract cannot be reviewed as a whole

while its symbol half is unwritten; the review itself is a human judgement, not a gate · **OWNER: human reviewer**, after T015-T017 (FR-037)



T106 [TDD] **[PATH CORRECT, FILE PARTIAL — measured 2026-09-02, workshop/scripts/_common.sh EXISTS at 273 lines but is the CONTAINER CONTROL-PLANE helper, not the §2.1 contract. Present: the three-valued exit constants, undetermined()/problem(), derived-never-hardcoded paths, set -euo pipefail. Absent: the §2.2 four-rung resolution order over every WORKSHOP_* var, the §2.4 (size, mtime, inode) fingerprint helpers, the §2.9 self-hash re-check, and the §1.4 ERR/EXIT traps — grep -c '^s*trap ' returns 0. Extend that file; do not write a second one]** Implement the shared wrapper library workshop/scripts/_common.sh contracted in contracts/pipeline-cli.md §2.1 — sourced, never executed. It owns the §2.2 resolution order **environment → config file → live probe → documented fallback**, identical for every WORKSHOP_* variable and copied from scripts/lumen-reindex.sh so an operator learns one pattern rather than two, printing at -v the host facts it measured and the arithmetic it performed; the §2.4 source-fingerprint helpers that record (size, mtime, inode) for every source before a run and re-check them after; the §2.9 self-hash re-check that aborts with 2 and reason.code: "script_modified_while_running" when a long-running script is edited mid-flight; and the §1.4 ERR and EXIT traps. Gate **G-CLI-10** (§1.4, §5): assert every *unclassified* non-zero status maps to 2 — 126, 127, 128+N and any other unclassified non-zero — that only statuses set by a command's own classification logic may be 0 or 1, and that 3–125 stay reserved and unused. **Paired mutation:** remove the ERR trap; the gate MUST go red, because a missing binary then surfaces as 127 and a tool that was never there reads as a real problem found — **BLOCKER:** none but the work. **RE-MEASURED 2026-09-04: the LIBRARY half landed this session and the GATE half did not.** workshop/scripts/_common.sh gained 983 lines / 39 ws_* functions, purely additive (0 deletions): the §2.2 four-rung resolver ws_resolve with -v host-fact reporting, the §2.4 (size, mtime, inode) fingerprint helpers (inode via POSIX ls -i, not stat, per _portable.sh's recorded GNU-vs-BSD trap), the §2.9 self-hash re-check aborting 2 with reason.code: script_modified_while_running, and the §1.4 traps — grep -c '^s*trap ' moved **0 -> 4**. The §1.4 mapping was **DRIVEN:** a fixture calling a missing binary produced **UNDETERMINED:** unclassified failure – exit status 127 and rc 2. Regression-checked in both directions: bash -n passes on all nine control-plane scripts and redact.sh/start.sh/stop.sh/ingest.sh/verify.sh --help

output is BYTE-IDENTICAL before and after; `status.sh` still returns RUNNING, rc 0. **What is NOT done: gate G-CLI-10 and its paired mutation were never written** — design settled, no file exists · **OWNER: implementer** — unblocked today; only the gate remains (FR-033, SC-013, FR-032, SC-012)



T107 [TDD] [PATH CORRECT, CONTENT NOT BUILT — measured 2026-09-02, no §2.5 evidence writer exists at that path or any other: `grep -rln 'findings.jsonl' workshop --include='*.py' --include='*.sh' --include='*.go'` returns zero files, and `result.js` on is written by exactly one script, `workshop/scripts/verify-accuracy.sh`, into its own tree with no `manifest.json`, no E2 pre-flight abort and no E4 rule. Every pipeline wrapper task below depends on this. HALF OF THAT NOTE IS NOW WITHDRAWN BY NAME — re-measured 2026-09-03. The clause “no §2.5 evidence writer exists at that path OR ANY OTHER” and the clause “no `manifest.json`, no E2 pre-flight abort” are FALSE. `workshop/scripts/redact.sh` implements the §2.5 writer and I DROVE it: every run creates `$WORKSHOP_EVIDENCE_DIR/redact/<UTC-timestamp>-<run_id>/` holding `manifest.json`, `result-exit.json`, `stdout.log`, `stderr.log`. E1 is real — `redact.sh:161–:169` writes evidence on all three outcomes, with the comment this task’s own text asks for (“a run that could not determine anything is the run whose evidence matters most”), confirmed live at rc 0, 1 and 2. E2 is real and was DRIVEN: `WORKSHOP_EVIDENCE_DIR=/nonexistent/ev` `bash workshop/scripts/redact.sh --check-review --chapter 01 → rc 2`, “the evidence directory ... could not be created; set `WORKSHOP_EVIDENCE_DIR` to a writable location”, printed BEFORE any work, exactly as `:109–:119` contracts. E4 holds by construction: the manifest records “`env_names_resolved`”: [`“WORKSHOP_ROOT”`, `“WORKSHOP_CURRICULUM_DIR”`, `“WORKSHOP_CHAPTERS_DIR”`, `“WORKSHOP_EVIDENCE_DIR”`] — NAMES only, no values, no paths-as-secrets. WHAT IS STILL GENUINELY OPEN, and it is why this stays unticked — four things, each measured: (1) It is in the wrong file. This task says “Implement the §2.5 evidence writer in `workshop/scripts/_common.sh`”; it lives in `redact.sh` and is one command’s private implementation, so `ingest.sh`, `verify-accuracy.sh` and every wrapper still to be written each get their own or none. `grep -nE 'findings\.jsonl|manifest\.json|result\.json|stdout\.log|stderr\.log|api_key_env' workshop/scripts/_common.sh` returns zero lines. (2) `findings.jsonl` is written by nothing — `grep -rln 'findings.jsonl' over workshop/ *.sh/*.go/*.py` still returns zero files, so the one artifact that carries the findings themselves

does not exist. (3) The file is named `result-exit.json`, not the contracted `result.json`, and carries `{run_id, exit, at}` only — no state, which is what T119's G-CLI-1 must compare against the process exit. (4) Gate G-CLI-9 exists nowhere — `grep -rn 'G-CLI-9' over workshop/` returns zero — so “evidence exists for every run outcome, especially 2” is true today by measurement but is guarded by no check, and its paired mutation (skip evidence on the failure path) has never been run] Implement the §2.5 evidence writer in `workshop/scripts/_common.sh` per [contracts/pipeline-cli.md](#) — every run writes `manifest.json`, `result.json`, `stdout.log`, `stderr.log` and `findings.jsonl` under `$WORKSHOP_EVIDENCE_DIR/<command>/<UTC-timestamp>-<run_id>/`, with E2: if the evidence directory cannot be created or written the command exits 2 **immediately, before doing any work**, printing the reason to `stderr` rather than proceeding unrecorded or attempting to write evidence about failing to write evidence; and E4: `api_key_env` records the NAME of an environment variable, never its value and never a file path. Gate **G-CLI-9** (§2.5 E1, §5): assert evidence exists for every run outcome — 0, 1 and **especially 2**, the run whose evidence matters most, because a run that determined nothing is the one a reader most needs the record of. **Paired mutation**: skip evidence writing on the failure path; the gate MUST go red (FR-040, SC-018) — **BLOCKER**: none but the work. **RE-MEASURED 2026-09-04: three of the four measured gaps CLOSED this session; gap (1) and the gate did not.** The shared §2.5 writer now exists in `_common.sh` (`ws_evidence_begin/ws_write_manifest/ws_evidence_finding/ws_evidence_finish/ws_run_with_evidence`): the T107 `grep` that returned **zero** lines now returns **30**. Gap (2) CLOSED — `findings.jsonl` is written. Gap (3) CLOSED — the file is `result.json` and carries state, derived from a single `ws_state_for_exit` mapping so agreement with the process exit is STRUCTURAL rather than asserted, which is exactly what T119's G-CLI-1 needs; nothing in the tree reads the old `result-exit.json`. DRIVEN at outcome 2, the outcome that matters most (E1): all five files written, `result.json` `n = {"state": "could_not_determine", "exit": 2, ...}`. **STILL OPEN: gap (1)** — `redact.sh` was NOT switched over and still uses its own private writer, so there are two writers today, not one; and **gate G-CLI-9 was never written**. Honest boundary (§11.4.6): **E2 and E4 were NOT re-driven after the move** — the code paths are written and parse, and E4 is now enforced rather than documented (`ws_evidence_api_key_env` refuses anything containing `/`), but neither was re-proven, so do not record them as re-verified · **OWNER: implementer** — unblocked today (FR-040, SC-018, FR-033, SC-013, FR-032, SC-012)



T108 Implement `workshop/scripts/build.sh` per [contracts/pipeline-cli.md §4.8](#), building in this order: the workshop-boot adapter into `$WORKSHOP_BOOT_BIN`, then the Go backend and the Angular frontend into `workshop/platform/bin/`, then the container image(s) the compose file names — **through the containers submodule, never a direct podman build** (§11.4.76(4)). Three-valued exit: 0 everything built · 1 compile, test or image-build failure, a real problem in the code or the compose definition · 2 toolchain missing (Go, Node, **no container runtime detected**), network needed and unreachable, or disk full. “No container runtime detected” is emphatically 2 and never 1: the host could not be assessed, which is not a finding about the code (FR-012, FR-033, SC-013)



T109 Implement `workshop/scripts/restart.sh` per [contracts/pipeline-cli.md §4.8](#) — `stop.sh` then `start.sh`, with the same arguments forwarded to `start.sh`. It is **not** a compose restart: a full down/up is what makes the restart honest about picking up a changed compose file or a rebuilt image. Its exit is `start.sh`’s exit, **except** that a `stop.sh` exit of 1 or 2 short-circuits and propagates unchanged — restarting on top of a stack that could not be stopped would report success over an unknown, which is standing rule 2’s failure shape at the lifecycle layer (FR-012, FR-033)



T110 [TDD] Prove gate **G-CLI-16** ([contracts/pipeline-cli.md §4.8, §5](#)) — **consumption, not reimplementation** (§11.4.76(4)): assert that no lifecycle path under `workshop/` invokes `podman`, `docker`, `podman-compose` or `docker compose` to bring the stack up or down, and that every such transition goes through `workshop-boot`, which reaches the runtime only through the submodule’s `pkg/compose`. Read-only diagnostics on a failure path — tailing a dead container’s logs — are exempt, and the gate **MUST show** each exemption to be one rather than assert it. **Paired mutation**: add a `podman-compose up -d` fallback to `start.sh` for when `workshop-boot` is missing; the gate **MUST go red**, because a well-meaning fallback for a missing binary is exactly the parallel implementation the clause forbids and is the shape drift takes (FR-046, FR-032, SC-012)

Checkpoint: passage identity is proven to survive correction and movement; code-passage identity either has a confirmed producer and fails loudly when stale, or P-U1 is recorded as unsettled and SC-016a is explicitly open. Stop here for human approval.

Phase 3: User Story 1 — Read what was actually said (Priority: P1) MVP

Goal: a faithful, timestamped, verifiable transcript of Chapter 1.

Independent test: open the markdown transcript, read Chapter 1 end to end, sample five passages at random and confirm each is accurate and its timestamp lands on the corresponding moment.



T023 [US1] Wire `workshop/pipeline/transcribe/reassemble.sh` to invoke the EXISTING `workshop/scripts/extract-videos.sh` — FR-007 is already implemented there with per-part, archive and extracted-video hash verification; do not reimplement it. **[PATH NOT BUILT — `workshop/pipeline/transcribe/reassemble.sh` does not exist, nor does the `transcribe/` directory. `workshop/scripts/extract-videos.sh` DOES exist, exactly as this task says, and is invoked — but by `workshop/scripts/git-hooks/post-checkout:6` (installed by `workshop/scripts/install-hooks.sh`) and by a documented command in `workshop/docs/quickstart.md:80`, NOT by any pipeline stage. The one pipeline file that names it, `workshop/pipeline/calibrate.sh:67`, prints it as an operator instruction on an exit-2 path and does not invoke it. The wiring is what is missing, not the target — do not reimplement extraction. RE-MEASURED 2026-09-03: unchanged and still accurate. `workshop/pipeline/transcribe/` now EXISTS (T025's `chunker.py` and T032's `pdf_notes.py` landed there), so the directory is no longer the obstacle — but `reassemble.sh` is not in it, and `grep -rn 'extract-videos' over workshop/ *.sh/*.py` still shows the only invokers are `scripts/git-hooks/post-checkout`, `scripts/self-test.sh` and `scripts/extract-videos.test.sh`. No pipeline stage calls it.**

UNWRITTEN, nothing blocks it] — BLOCKER: none but the work — the wiring to the EXISTING `extract-videos.sh` is unwritten • **OWNER: implementer** — unblocked today — **EVIDENCE:** DONE at a DRIFTED PATH — the contracted `workshop/pipeline/transcribe/reassemble.sh` still does not exist, but the wiring this task asks for (invoke the EXISTING extractor, do not reimplement) landed in `workshop/scripts/setup.sh:271 do_extract_chapters`, which forwards `workshop/scripts/extract-videos.sh` and passes its three-valued status through unchanged (:288-:295). Exercised 2026-09-08: `bash workshop/scripts/extract-videos.sh workshop -> rc 0`, skip (intact) + sidecar ... (verified) for all 3 chapter recordings. (population: `in_process`) (FR-007, FR-006)



T024 [US1] [TDD] [PATH CORRECTED 2026-09-02 — this task named `workshop/pipeline/transcribe/vad.py`, which does not exist; the detector landed at `workshop/pipeline/audio_energy.py`, reading PCM directly and deriving its threshold from the recording's own noise floor (FLOOR_MARGIN_DB at :33) rather than **hardcoding a dBFS constant**] Implement silence detection in `workshop/pipeline/audio_energy.py` producing the non-speech span set (the 8 measured silences totalling 41.33 s are a free test fixture) (population: in_process) (FR-001, SC-001)



T025 [US1] [TDD] Implement ≤ 300 s chunking that cuts INSIDE measured silence, in `workshop/pipeline/transcribe/chunker.py` — **[TICKED 2026-09-03 ON MEASUREMENT. PATH NOT BUILT — neither `chunker.py` nor the `transcribe/` directory exists, and no equivalent landed anywhere — WITHDRAWN BY NAME: both exist. `workshop/pipeline/transcribe/chunker.py` is 16,320 B and its own line 3 names this task — “*cut a recording into ≤ 300 s chunks, INSIDE measured silence (T025)*”. The ceiling is `DEFAULT_MAX_SECONDS = 300.0` at :97, deliberately a default rather than a constant so it is overridable (`--max-seconds`). It consumes `pipeline/audio_energy.py` (T024) rather than growing a second silence detector. Three-valued exit at :61: 1 for a real finding (no qualifying silence in an over-long span, a forced cut), 2 for could-not-determine; an over-long span with no qualifying pause is refused with code: “`no_silence_in_span`” (:170) and a remedy, not silently force-cut. §1.1 paired proof RUN 2026-09-03: `bash workshop/pipeline/transcribe/prove-chunker.sh` → rc 0, PROVE : 8 mutations, 8 caught, 0 missed, including M3 ceiling ignored → V2_length, M4 cut outside measured silence → V3_in_silence, M5 forged cut_inside evidence → V3_in_silence (the invariant is re-derived, not trusted), M6 tail chunk dropped → V1_tiling, and M8 unreadable input → rc 2, not 1. Registered as `check_transcribe-chunker` in `workshop/platform/gates/check-registry-001.tsv` with `undet-probe /nonexistent.wav`] (population: in_process) (FR-001, SC-001)**



T026 [US1] [TDD] [PATH CORRECTED 2026-09-02 — this task named `workshop/pipeline/transcribe/asr.py`, which does not exist; the driver landed at `workshop/pipeline/run_faster_whisper.py`, which passes `condition_on_previous_text=False` at :107 and records it at :153] Implement the faster-whisper

driver in workshop/pipeline/run_faster_whisper.py with condition_on_previous_text=False — the same setting that makes it resumable also suppresses repetition-loop hallucination (FR-001, FR-003, FR-029)



T027 [US1] [TDD] **[PATH CORRECTED 2026-09-02 — this task named workshop/pipeline/transcribe/confidence.py, which does not exist; the work landed SPLIT ACROSS TWO LANGUAGES: emission in workshop/pipeline/run_faster_whisper.py:118–:119 (avg_logprob, no_speech_prob per segment), mapping in workshop/platform/backend/cmd/ingest-transcript/main.go:77–:78 (Uncertain, UncertainReason) applied at :313–:322]** Map engine confidence (avg_logprob, no_speech_prob) to the uncertain flag across workshop/pipeline/run_faster_whisper.py and workshop/platform/backend/cmd/ingest-transcript/main.go — FR-003 requires marking, never guessing (FR-003)



T028 [US1] [TDD] Implement atomic checkpoint/resume (write-temp → fsync → rename) in workshop/pipeline/transcribe/checkpoint.py per FR-029 — **[PATH NOT BUILT for the CHECKPOINT half; the ATOMIC-WRITE half moved. Measured 2026-09-02: checkpoint.py and the transcribe/ directory do not exist, but workshop/pipeline/run_faster_whisper.py:179–:184 already implements exactly the contracted shape — tmp = out.with_suffix(out.suffix + ".tmp") → json.dump → fh.flush() → os.fsync(fh.fileno()) → tmp.replace(out). Do NOT rebuild that. What is genuinely absent is checkpoint/resume across chunks and any --resume flag, and it depends on T025's chunker, which is also unbuilt — UNBLOCKED 2026-09-03: T025's chunker is now BUILT, PROVEN (8/8 mutations caught) and TICKED at workshop/pipeline/transcribe/chunker.py, so the dependency this note named is discharged and the remaining work is unwritten, not blocked. Re-measured the same day: workshop/pipeline/transcribe/checkpoint.py still does not exist, and the atomic-write half at run_faster_whisper.py:179–:184 is unchanged — do NOT rebuild it] — BLOCKER: none but the work — the checkpoint half is unwritten · OWNER: implementer — unblocked today (FR-029)**



T029 [US1] [TDD] **[PATH CORRECTED 2026-09-02 for the IDENTITY, [PATH NOT BUILT] for the GATE — this task named workshop/pipeline/transcribe/coverage.py, which does not exist. The**

identity landed in Go, in the backend: workshop/platform/backend/pkg/curriculum/curriculum.go ComputeCoverage at :428, UnexplainedGapS at :503, Complete = UnexplainedGapS == 0 at :504, the identity string in the Identity field at :376; it is served at internal/api/chapters.go:177/:362 and made a contract violation for a published chapter at :216. Do not rebuild the arithmetic.** Gate G-CLI-4 and its tolerate-gaps mutation are what remain, and they must gate whatever actually computes coverage — which is this Go path, not a Python module. — **BLOCKER:** none but the work — the identity landed in Go; the GATE is unwritten · **OWNER: implementer** — unblocked today (FR-001, SC-001)

RE-MEASURED 2026-09-03 — the note above still holds in both directions, and the gap is now quantified. The arithmetic is present and unchanged: ComputeCoverage curriculum.go:428, UnexplainedGapS :503, Complete = UnexplainedGapS == 0 :504. **grep -rn 'G-CLI-4' over workshop/ returns ZERO files** — the gate does not exist — and **grep -rn 'ComputeCoverage' --include='*_test.go'** over platform/backend also returns **zero**, so the identity is not covered by a unit test either; let alone by the delete-a-chunk gate or the tolerate-gaps mutation this task names. SC-001 is arithmetic in the code and ungated in the tree]** Implement the coverage identity check in workshop/platform/backend/pkg/curriculum/curriculum.go: passage spans $\cup$ VAD silence MUST equal [0, duration_s) exactly — this makes SC-001 arithmetic rather than judgement. Gate **G-CLI-4** ([contracts/pipeline-cli.md](#) §4.1, §5): delete one chunk's output and assert the coverage check reports exit 1. **Paired mutation:** make coverage tolerate gaps under a threshold; the gate MUST go red — this is the mutation proof D-TRANS-4 names



T030 [US1] Emit the markdown transcript with pid anchors and timestamps to workshop/curriculum/chapter-01/transcript.md — **[PATH CORRECTED 2026-09-02 — this task named workshop/chapters/01/transcript/transcript.md; that directory does not exist (workshop/chapters/01/ holds the recording and its archive parts only) and the timestamped transcript landed at workshop/curriculum/chapter-01/transcript.md. The TIMESTAMP half is built; the pid-anchor half is not —**
grep -rl '<!-- pid:' workshop --include='*.md' returns zero files across the whole submodule, and passage identity currently lives only in workshop/curriculum/passages.jsonl. RE-MEASURED 2026-09-03: unchanged. workshop/curriculum/chapter-01/transcript.md EXISTS (and T040's R2 assertion was

re-run against it, 0 violations across 3 surfaces), and `grep -rl '<!-- pid:' workshop --include='*.md'` STILL returns ZERO files. The consequence reaches further than this task: T009 built the anchor reader/writer at `submodules/passage/pkg/passage/anchor.go` and it is ticked, but nothing in the tree has ever written an anchor into a document, so T113's `--write-anchors` has nothing to switch on and the §8 R1b/R1c duplicate- and foreign-anchor branches have never been exercised against a real source. PARTIAL: timestamps done, anchors unwritten, nothing blocking] — BLOCKER: none but the work · OWNER: implementer — unblocked today (FR-001, FR-002, FR-037)



T031 [US1] [TDD] Implement the immutable machine layer + append-only correction overlay in `workshop/platform/backend/pkg/transcript/layers.go` per FR-038 — the machine output is evidence and must survive correction. [PATH NOT BUILT — neither `layers.go` nor the `pkg/transcript/` directory exists, and no equivalent landed anywhere. The nearest thing on disk is the corrections DDL at `submodules/passage/pkg/passage/registry.go:936–:944`, which `registry.go:1091` says that package never writes; a table declaration is not the layered type this task contracts] — BLOCKER: none but the work — `layers.go` is unwritten · OWNER: implementer — unblocked today (FR-038)



T032 [US1] [P] [TICKED 2026-09-03 ON MEASUREMENT. PATH NOT BUILT—`pdf_notes.py` and the `transcribe/` directory do not exist, and no gazetteer exists anywhere: `grep -rli gazetteer workshop` returns zero files — WITHDRAWN BY NAME: all three clauses are false today. `workshop/pipeline/transcribe/pdf_notes.py` is 23,036 B and its own lines 3–4 name this task — “*read a chapter’s notes PDF for SECTION STRUCTURE and a PROPER-NOUN GAZETTEER, and for nothing else (T032)*”. The ONLY restriction this task turns on is enforced structurally rather than promised: :6 — “*THE NOTES ARE NOT GROUND TRUTH, AND THIS MODULE IS BUILT SO THEY CANNOT BECOME IT*” — with `NOT_GROUND_TRUTH` stamped on every emitted object (:82–:85, :19), `detect_sections` labelled a heuristic at :209, and the gazetteer at :250. It also carries a content-boundary guard this task did not ask for and needs: :31–:32 records that a proper-noun gazetteer of a private meeting summary is largely a list of people’s names, so `--gazetteer` and `--structure REFUSE` a git-TRACKED destination. §1.1 paired proof RUN 2026-09-03: `bash workshop/pipeline/transcribe/prove-pdf-notes.sh → rc 0, PROVE`

: 9 mutations, 9 caught, 0 missed, including M4 scanned PDF with no text layer → rc 1 naming OCR as a separate capability, M5 tracked destination refused with no file created, M7 the summary carries no term (counts and categories only), and M9 guard removed ⇒ harm occurs — the term list landed in a TRACKED path, so the guard is load-bearing rather than decorative. Registered as check transcribe-pdf-notes in workshop /platform/gates/check-registry-001.tsv with undet-probe --pdftotext /nonexistent. Honest boundary (§11.4.6): no run against the real Chapter 1 notes PDF was performed for this tick — the module and its nine-mutation proof were measured, the extraction of that particular document was not, and by M5/M9's own design its output may not land in a tracked path] Extract the notes PDF text layer in workshop/pipeline/transcribe/pdf_notes.py for section structure and a proper-noun gazetteer ONLY — never ground truth; it is a summary and it renders "Spatkit" for SpecKit (population: in_process) (FR-011, FR-006)



T033 [US1] [TDD] [REVIEW] [PATH CORRECTED 2026-09-02 — this task named workshop/pipeline/accuracy/score.py; neither it nor the accuracy/ directory exists. The scorer landed as workshop/scripts/verify-accuracy.sh, driving workshop/pipeline/compare_engines.py (align :113, norm_word :52) and workshop/pipeline/audit_windows.py (a_words_in), imported at verify-accuracy.sh:170–:178, with --windows/--seconds/--seed recorded so the sample reproduces] Implement the WER scorer in workshop/scripts/verify-accuracy.sh over workshop/pipeline/compare_engines.py and workshop/pipeline/audit_windows.py, sampling the AUDIO TIMELINE (≥30 stratified 30 s windows), not passages — sampling passages makes whole-region deletions structurally invisible and biases accuracy upward exactly where the transcript is worst (FR-004, SC-002)



T034 [US1] Freeze and hash the normaliser in workshop/pipeline/compare_engines.py BEFORE the first measurement — [PATH CORRECTED 2026-09-02 — this task named workshop/pipeline/accuracy/normaliser.py; neither it nor the accuracy/ directory exists, and the normaliser landed inside workshop/pipeline/compare_engines.py (norm_word :52, align :113), which workshop/scripts/verify-accuracy.sh:112 resolves as \$NORMALISER. The FREEZING is what is missing, not the normaliser: verify-accuracy.sh measures its sha256 on every run and enforces it only when --normaliser-sha256 is passed (default empty at :113),

and no frozen hash is committed anywhere — no *.sha256 under workshop/pipeline/, no normaliser record under workshop/evidence/. RE-MEASURED 2026-09-03: unchanged and still accurate. NORMALISER_SHA="" at verify-accuracy.sh:113, set only by --normaliser-sha256 at :132, and enforced at :588 — so the enforcement path is BUILT and simply never armed. The only *.sha256 file anywhere under workshop/ outside venv//engines/ is the chapter-01 recording's own checksum, which is a different artifact. The whole remaining task is: measure compare_engines.py's sha256 once, commit it as the frozen value, and make verify-accuracy.sh default to it instead of to the empty string — after which an edit to the normaliser makes every prior accuracy figure loudly unreproducible instead of silently incomparable. This must happen BEFORE T037(b)'s reference is scored, not after. UNWRITTEN, nothing blocking] — BLOCKER: none but the work — the normaliser must be frozen and hashed BEFORE the first measurement, so this gates T035 and T037 · OWNER: implementer — unblocked today, and it is the head of the accuracy chain (FR-004, SC-002)



T035 [US1] [TDD] Paired mutation proof for the scorer in workshop/scripts/verify-accuracy.sh --selftest — a transcript with a known injected WER must produce that WER. [PATH CORRECTED 2026-09-02 — this task named workshop/pipeline/accuracy/score_mutation_test.sh; neither it nor the accuracy/ directory exists, and the mutation harness landed inside the scorer wrapper as workshop/scripts/verify-accuracy.sh --selftest (:88, :140, :393, :405). Do not build a second harness** — extend that one. What is genuinely missing is THIS task's assertion: the existing selftest degrades a reference and checks that a threshold is crossed, which is not the same as asserting that a KNOWN INJECTED WER produces THAT WER. — BLOCKER: predecessor T034 — a scorer mutation proof is meaningless until the normaliser is frozen · OWNER: implementer, after T034 (FR-004, FR-032, SC-012)

RE-MEASURED BY EXECUTION 2026-09-03 — the note above is CONFIRMED, and the remaining work is now known to be ONE assertion, not a harness. bash workshop/scripts/verify-accuracy.sh --selftest → rc 0: absent reference → 2 (twice), perfect reference → 0, degraded reference → 1, and the bluffing mutant CAUGHT. The fixture already injects a known WER: verify-accuracy.sh:476–:478 degrades exactly every second word — “Degrade HALF the words: a real, measurable ~50% WER, not a

syntax break” — and the scorer **already measures it exactly**, printing accuracy 0.5000. What the selftest then asserts is 0.5000 < --min-accuracy 0.95 $\Rightarrow$ rc 1. **So the injected value and the measured value are both present and equal, and nothing compares them.** T035 is satisfied by adding that comparison — assert the measured accuracy equals the injected 0.50 within a stated tolerance — plus its own paired mutation (perturb the scorer’s arithmetic and require the new assertion to go red). Note for whoever does it: everything currently in --selftest proves **G-CLI-5**, which belongs to T112 (ticked); this task’s assertion would be the first thing in that block that is about the SCORER rather than about the absent-reference guard]**



T036 [US1] Run the 5-minute calibration on the extracted 300 s sample; this settles research open items U1, U2 and U3 in one run (FR-?)



T037 [US1] Produce and publish the Chapter 1 accuracy report: the METHOD statement (FR-004), the SAMPLE CENSUS, and the MEASURED accuracy figure (SC-002). — **BLOCKER: predecessor T034 + T035** — no accuracy figure may be published before the scorer that produced it is frozen and mutation-proved · **OWNER: implementer**, after T034/T035 (FR-004, SC-002)

[OPEN. REWRITTEN 2026-09-03, because the old text demanded something its own success criterion never asked for, and the demand made the task unsatisfiable by construction. Counts and offsets only below — no window text, no transcript line, no name, per docs/content-boundary.md.

WITHDRAWN, WITH THE REASON, NOT DELETED. This task read “...with the measured figure **and its confidence interval**”. The interval half is withdrawn **as a requirement of this task.**

Measured: neither SC-002 nor FR-004 asks for an interval.

spec.md:270 SC-002 — “Transcript accuracy is measured on a random sample of at least 30 passages, and the measured figure is published alongside the transcript”. spec.md:168 FR-004 — “...a verification report stating measured transcript accuracy and the method used to measure it”. Neither contains the word “interval”.

The demand traced **only** to research/transcription.md:809 and :856–:857, which is a research **decision record**, not a success criterion. The previous note’s framing — “closing it needs *EITHER* a spec amendment dropping the interval *OR* a T112 change” — is

therefore **withdrawn as overstated**: no spec amendment is needed, because the spec never asked. The interval survives as a **recorded, unbuilt research decision**, carried as item (c) below on its own feet, instead of as a hidden precondition that silently made this task impossible.

(a) METHOD STATEMENT AND SAMPLE CENSUS — DONE
2026-09-03, then RE-DONE the same day against a RE-EMITTED plan. Read (d) first: the window set changed, so every figure in the first census is SUPERSEDED. `research/transcription.md` §5.2 point 5 explicitly requires this enumeration — *“the passages overlapping the sampled windows are enumerated and counted in the report”* — and it had only ever been an argument, never a count.

SUPERSEDED BY NAME, NOT DELETED — the confidence-stratified census of 2026-09-03. It read: **174** distinct machine segments (**5.8×** the SC-002 floor), **2,057** machine words (`N_hyp`), segments per window **4 / 5 / 10**, words per window **50 / 70 / 83**, strata **10 / 10 / 10** high/mid/low confidence. Every one of those numbers described the confidence-stratified plan that (d) replaced. The plan that produced them is preserved byte-identical at `workshop/evidence/verify-accuracy/plan-superseded-2026-09-03-confidence-strata.json` (`sha256 9f017603...`) and is re-derivable on demand with `--strata confidence`, so the old census can be reproduced rather than merely believed.

CURRENT CENSUS — measured 2026-09-03 against the re-emitted `workshop/chapters/01/transcript/accuracy-plan.json` and `workshop/pipeline/transcripts/full_ch01.faster-whisper.json` (1,055 segments, 15,610 word timings). The 30 seeded windows are **900.0 s = 12.9894 %** of 6,928.713 s of audio; they overlap **163 distinct machine segments**; they contain **2,012 machine words** — this is `N_hyp`, NOT `N_ref`, and must never be published as a reference word count; segments per window min/med/max **2 / 5 / 8**; words per window **28 / 69 / 83**; **0 empty windows, 0 overlapping windows; 0 windows with zero machine segments**; strata **3 / 3 / 3 / 3 / 3 / 3 / 3 / 3 / 3 / 3** across ten equal temporal strata of 692.871 s. **SC-002’s “≥ 30 passages” floor is MET BY THE WINDOW DESIGN at 5.4×** — down from 5.8× and still clearing the floor by a factor of five, which is a computed fact rather than §5.2’s reconciliation argument. Re-derive (the command below reads the CURRENT plan, so it re-derives the current census; point it at the superseded file to reproduce the old one):

```
# ONE physical shell line on purpose: this block is indented
# inside a list item, and an
# indented heredoc terminator does not close a heredoc – copy-
# paste would hang, not run.
workshop/pipeline/venv/bin/python -c "import
    json;P=json.load(open('workshop/chapters/01/transcript/
    accuracy-plan.json'));S=json.load(open('workshop/
    pipeline/transcripts/full_ch01.faster-whisper.json'))
    ['segments'];o=lambda s,w: s['end']>w['t0'] and
    s['start']<w['t1'];c=lambda w: sum(1 for s in S if
    o(s,w) for x in (s.get('words') or []) if
    w['t0']<=(x['s']+x['e'])/2<w['t1']);seg={i for w in
    P['windows'] for i,s in enumerate(S) if
    o(s,w)};per=[c(w) for w in
    P['windows']];print('sampled_s',sum(w['seconds'] for w
    in
    P['windows']), 'of', P['audio_seconds']);print('distinct
    segments',len(seg), '>=30
    ->',len(seg)>=30);print('N_hyp',sum(per), 'per-window
    min/med/max',min(per),sorted(per)[len(per)//
    2],max(per))"
# sampled_s 900.0 of 6928.713
# distinct segments 163 >=30 -> True
# N_hyp 2012 per-window min/med/max 28 69 83
```

(b) THE MEASURED FIGURE — BLOCKED ON A NAMED OPERATOR INPUT. This is not an impossibility, and it must not be recorded as one. bash workshop/scripts/verify-accuracy.sh 01 exits 2 — “*UNDETERMINED: --reference is required*” — re-run 2026-09-03. That is the correct answer, not a failure: **an engine cannot be its own ground truth**, and research/transcription.md §5.1 refuses both available shortcuts by name (the notes PDF is a summary with no verbatim speech; a second Whisper-family engine shares training data and agrees on the same hallucinations). No accuracy.json exists — find workshop -name 'accuracy*.json' returns exactly one file, the plan. Publish precondition **B2 is NOT met** and must not be recorded as met; GET /api/chapters/01/accuracy correctly keeps reporting measured: false, wer: null.

THE HANDOVER, stated exactly so it can be acted on without reading anything else. It is written against the RE-EMITTED plan of (d); a reference against the superseded window set would match nothing.

What

A blind human verbatim transcript of the **30 windows** listed in workshop/chapters/01/transcript/accuracy-plan.json — 30.0 s each, **900.0 s = 15 minutes** of audio total, spread 3 per stratum across ten equal 692.871 s strata.

Where the windows are	In that file, field <code>windows[]</code>, each with <code>t0/t1</code>. Do not re-derive them; do not renumber them. JSON matching that file's own <code>reference_schema</code>: <code>{"windows": [{"t0":<float>,"t1":<float>,"text":"..."}]}</code>
Form	— one object per window, <code>t0/t1</code> copied exactly from the plan (the scorer keys on them to a centisecond and will not match a nearest neighbour; a mismatched pair is <code>windows_short</code> → exit 2, not a score). <code>onset_s</code> — the true onset of the first word, read off the waveform — and speaker per utterance. §5.4 point 2 asks for both and calls them “<i>nearly free while they are already listening</i>”. Nothing computes them today and the scorer ignores unknown keys, so recording them costs this run nothing; omitting them costs a second full listening pass later, because they are the only inputs the timestamp-error (SC-003) and speaker-attribution (FR-005) companion metrics can ever have.
Two optional fields worth filling while listening	Transcribed from the audio alone, without reading the machine transcript. A reference produced by editing machine output measures nothing: Whisper's characteristic failure is fluent, plausible, wrong text, which is exactly what an anchored reviewer accepts. Mark genuinely unintelligible audio [unintelligible]; never guess it.
Hard constraint	
Cost	900 s at §5.4's stated 4–8× realtime = 1–2 hours of human work.
On whom	The operator. No agent can produce it — §5.1 refuses both available shortcuts by name.
Command afterwards	<pre>bash workshop/scripts/verify-accuracy.sh 01 --reference <path> (no --strata flag needed; temporal/10 are the defaults the plan was emitted with).</pre>

What it unblocks — TWO criteria, not one	(1) spec 001 SC-002 / FR-004 : writes workshop/chapters/01/transcript/accuracy.json → publish precondition B2 met → the published measured figure exists. (2) spec 002 FR-064a / T135 — see (e) below.
---	---

(c) THE CONFIDENCE INTERVAL — BLOCKED TWICE, and only one of the two blocks is (b). It is no longer a requirement of this task (see the withdrawal above); it is carried here so the research decision is not quietly dropped. Block 1: an interval needs the figure from (b). Block 2, **independent of (b) and actionable today — the estimator does not exist**: `grep -rEin 'confidence_interval|ci_low|ci_high|ci_95|bootstrap|margin_of_error|wilson|binom'` over workshop/scripts/ and workshop/pipeline/.sh/.py returned **0 matches**, and the payload built by `verify-accuracy.sh` carries measured, wer, accuracy, chapter, measured_at, tool, method, counts, sample, windows, normaliser, transcript, reference, min_accuracy, companion_metrics, verdict — no interval field. **That grep now returns 3 matches, all in workshop/scripts/verify-accuracy.sh, and NONE of them is an estimator — they are the prose of the estimator_unit warning added 2026-09-03, naming bootstrap and binomial in order to forbid the wrong one.** The finding is unchanged: **nothing in this project computes a confidence interval**, and the payload still carries no interval field. The “0 matches” figure is superseded as a *count*, not as a *conclusion*; re-read the matched lines rather than the number.

If it is written, its UNIT is the decision, and getting it wrong is worse than omitting it. A word-level binomial/Wilson interval over $N_{\text{hyp}} = 2057$ would be **wrong twice**: words inside one 30 s window are not independent draws, and WER is not a proportion at all — insertions let $(S+D+I)/N_{\text{ref}}$ exceed 1. The honest unit is the **window: n = 30 clusters**, a bootstrap resampling window-level (errors, N_{ref}) pairs under a ratio estimator. The difference is not cosmetic — at the limit of perfect intra-window correlation a word-level interval is narrower by up to $\sqrt{(2057/30)} \approx 8\times$, which would **overstate** what was measured, the precise failure research:856 says the interval exists to prevent. Whoever builds it states the unit in the report. **Owner: a T112 change, not this task.**

(d) THE SAMPLING-FRAME DECISION — TAKEN BY THE OPERATOR AND EXECUTED 2026-09-03. This section previously ended “the decision is an operator decision”; that is now **DONE**, and the figures it carried are **SUPERSEDED**, not deleted.

What was found. research/transcription.md §5.2 specifies the procedure as **10 equal temporal strata** of 692.875 s with **3 windows each**. The plan emitted before this change stratified by **confidence** instead (high/mid/low, 10 windows each). Its temporal occupancy under §5.2's own scheme, re-derived and reproduced exactly before anything was overwritten:

[5, 1, 3, 3, 3, 3, 2, 2, 5, 3] — **0 strata empty, 3 strata below 3**, largest unsampled contiguous gap **702.2 s (11.7 min)** at [3972.2, 4674.4). **Read the split honestly, because it is the part most easily overstated: §5.2's stated worst case — “a simple random draw can leave a 20-minute region unsampled” — did NOT occur, so that plan was never disqualified and is not being described as broken.** What was false was applying §5.2 point 4's guarantee to it: “Stratification guarantees the whole session is represented” is a claim about **temporal** strata, and those were **confidence** strata, so the guarantee on offer was not the one being supplied. **The operator chose conformance to the method regardless**, and the re-emit had to precede (b) because it changes the seeded window set and would invalidate any reference already transcribed.

Precondition checked before overwriting anything: no reference existed to invalidate. find workshop -iname '*reference*' outside venv//.git//engines/ returned **nothing**, and find workshop -name 'accuracy*.json' returned exactly one file — the plan itself. **The old plan is preserved byte-identical** at workshop/evidence/verify-accuracy/plan-superseded-2026-09-03-confidence-strata.json; its name deliberately does not match accuracy*.json, so that glob still returns exactly one file.

What was changed — the EMITTER, never the artifact. A hand-written plan is unrepeatable and the next re-emit would silently diverge from it, so workshop/scripts/verify-accuracy.sh gained --strata temporal|confidence and --strata-count (default **temporal, 10**), and the mode is now recorded IN the plan (stratification, strata_count, strata_seconds, method) and in accuracy.json's sample block, so a reader can tell which procedure produced a window set without re-running anything. Placement inside a stratum draws per uniforms on $[0, \text{width} - \text{per} \cdot (\text{span} + \text{GAP})]$, sorts them and offsets window i by $i \cdot (\text{span} + \text{GAP})$ — a uniformly distributed non-overlapping set in one pass, with $\text{GAP} = 0.02 \text{ s}$ guaranteeing the **rounded** windows stay disjoint. Disjointness is nevertheless ASSERTED in code and a violation is a named 2, never a silent fix.

The material improvement is NOT the gap figure, and must not be sold as one. Measured on the re-emitted plan: largest unsampled gap **686.5 s (11.44 min)** at [5917.6, 6604.1) versus the superseded **702.2 s** — a **15.7 s (2.2 %)** reduction, which is noise, and the temporal design's geometric worst case is in fact *looser* than what the old draw happened to achieve. Two real, measured differences carry the decision instead:

1. **Equal allocation by construction, not by luck.** Occupancy is [3, 3, 3, 3, 3, 3, 3, 3, 3, 3] — §5.2 point 4's guarantee is now the guarantee actually being supplied. Re-derived two independent ways (from the recorded `stratum_index`, and from `t0` alone) and they agree.
2. **Recogniser-dropped regions are now reachable.** The confidence planner can only ever centre a window on a segment the recogniser PRODUCED, so a region it dropped entirely is structurally unsamplable by it — the exact upward bias §5.2 opens by naming. Uniform offsets can land there. Measured: audio inside the sample covered by **no machine segment at all** rose **50.2 s → 60.2 s**, across **14 → 22** windows. That is the deletion-detection surface the human transcription will actually see.

Measured properties of the re-emitted plan (`bash workshop/scripts/verify-accuracy.sh 01 --emit-plan workshop/chapters/01/transcript/accuracy-plan.json, exit 0`): 10 strata of equal duration **692.871 s**, contiguous and covering [0, 6928.713); **3 windows each**; **30 of 30** placed, shortfall **0**, `strata_short` empty; **0 empty strata**; **0 overlapping pairs**, minimum inter-window separation **22.91 s**; every window contained in its own stratum; all durations exactly **30.0 s**; **900.0 s = 12.9894 %** of the recording.

Two proofs that the emitter, not the artifact, is the source of truth. (i) A second independent emission produces an **identical** window set — reproducible, so the plan can be regenerated rather than guarded. (ii) `--strata confidence` reproduces the **superseded window set exactly**, tuple for tuple including stratum labels — so the refactor changed the default, not the old planner's behaviour, and the previous plan is recoverable by flag and not only from the backup. `bash workshop/scripts/verify-accuracy.sh --selftest` exits **0** (G-CLI-5 both halves → 2, perfect → 0, degraded → 1, bluffing mutant CAUGHT), exercising the temporal planner end-to-end including its `want < strata_count` reduction path. `bash workshop/platform/gates/verify-check-registry-001.sh` exits **0** at 20 PASS / 0

FAIL / 1 DEBT (pre-existing cross-registry-attribution, untouched by this work); R5 PASS at 19 swept files — **no new script was created**, an existing one was extended.

Re-derive the SUPERSEDED occupancy (the command below reads whichever plan you point it at; against the current plan it prints the temporal result, against the preserved backup it prints [5, 1, 3, 3, 3, 3, 2, 2, 5, 3]):

```
workshop/pipeline/venv/bin/python -c "import
json;P=json.load(open('workshop/chapters/01/
transcript/accuracy-
plan.json'));A=P['audio_seconds'];K=10;w=A/
K;occ=[sum(1 for x in P['windows'] if
min(K-1,int(x['t0']/w))==k) for k in
range(K)];V=sorted((x['t0'],x['t1']) for x in
P['windows']);b=[0.0]+[t1 for t0,t1 in V];e=[t0 for
t0,t1 in V]+[A];g=max(zip([y-x for x,y in
zip(b,e)],b,e));print('temporal occupancy',occ,'|
research 5.2 demands 3 per stratum');print('largest
unsampled gap %.1f s at [%.1f, %.1f)')%g)"
# against the CURRENT plan:
#   temporal occupancy [3, 3, 3, 3, 3, 3, 3, 3, 3, 3] |
#       research 5.2 demands 3 per stratum
#   largest unsampled gap 686.5 s at [5917.6, 6604.1)
# against workshop/evidence/verify-accuracy/plan-
#   superseded-2026-09-03-confidence-strata.json:
#   temporal occupancy [5, 1, 3, 3, 3, 3, 2, 2, 5, 3] <-
#       SUPERSEDED
#   largest unsampled gap 702.2 s at [3972.2, 4674.4) <-
#       SUPERSEDED
```

(e) THE SAME TRANSCRIPTION IS THE MISSING SPEECH-WER BASELINE FOR SPEC 002's OCR PHASE. Verified from source 2026-09-03, not inherited. This is recorded here AND at specs/002-knowledge-areas-deep-linking/tasks.md T135, because a dependency written down in one file only is one rediscovery away from being missed again.

The claim, checked rather than assumed. specs/002-.../tasks.md T135 (gate **G-OCR-9, FR-064a**) requires the OCR accuracy budget to be “*derived from the recorded speech-recognition calibration for the same corpus and read from it at run time, never written as a literal*”, three-valued, with “*calibration record or ground-truth sample unreadable* ⇒ 2”. That record does **not** exist: workshop/pipeline/CALIBRATION.md's own **U4** row reads “*achievable WER | open — needs §5's blind human reference*”, and the same line states the engine-to-engine divergence **3.19 %** “*is **not** a WER and must never be quoted as one*”. Quoting it would be precisely what T135's own

paired mutation (a) forbids. **Both halves reproduce exactly as described.** T135 gates spec 002's closure (SC-031→SC-034), so this sat on the critical path unseen.

Does a 30-window SAMPLE satisfy it, or does T135 need a whole-chapter WER? A sample satisfies it, on the spec's own terms — three pieces of evidence, none of them inference. (i) FR-064a says “*read from the recorded speech-recognition calibration*” and never says exhaustive. (ii) FR-064 requires the OCR side itself to be “*a hand-truthed ground-truth **sample** drawn by a recorded, seeded, reproducible rule with the population size published*” — the OCR figure is a sampled estimate, so demanding an exhaustive speech figure to compare it against would be an asymmetry nothing asks for. (iii) T135's own three-valued clause names “*calibration record or ground-truth **sample** unreadable ⇒ 2*” — the task anticipates a sample. `accuracy.json` as `verify-accuracy.sh` writes it is that **record**: seeded, reproducible, population published. **If a whole-chapter WER had been required, this decision would not close T135 and that would be said here instead.**

Does the OCR comparison need anything the SC-002 measurement does not? Measured per axis, and the answer differs by axis — which is why it is stated rather than assumed.

- **Textual axis (T133 / SC-031): nothing extra, and the comparability is structural rather than lucky.** SC-031 requires the OCR figure be “*scored by the same edit-distance method the speech-recognition calibration already uses, so the two figures are comparable rather than merely adjacent*”. `verify-accuracy.sh` **imports** `align/norm_word` from `workshop/pipeline/compare_engines.py` rather than reimplementing them, and T133's own note records that the same file is what the OCR scorer must share. Same normaliser, same alignment, one definition of WER. **The reference text needed for SC-002 is bit-for-bit the reference text needed here.**
- **Temporal axis (T134 / SC-032): this is where the two purposes diverge, and it is the reason the two optional fields in (b) matter.** SC-032 measures whether a declared visibility interval *contains* the true moment; SC-033 requires both OCR figures be published “*beside the speech-recognition figures ... in the same document, in the same run*”. The speech-side analogue of a temporal figure is this project's own **timestamp error** companion metric (SC-003, `|machine time of first reference word – true onset|`, median and p95), whose only possible input is the `onset_s` mark §5.4 point 2 asks the reviewer to

make. **Capture it in the same pass or it costs a second 1–2 hours.** Honest boundary: SC-033's wording does not *demand* a speech temporal figure, so omitting onset_s does not by itself fail T135 — it forecloses the symmetric publication cheaply available today.

- **A second asymmetry, naming a T112 change rather than more human work.** SC-031 requires the OCR figure as **WER and CER**. `verify-accuracy.sh` computes **WER only**; §5.3 already calls CER “*nearly free once alignment exists*”. Producing a speech CER needs **no additional transcription** — the same reference text yields it. **Owner: a T112 change. Cost to the operator: zero.**
- **Scope, stated so it is not over-read.** FR-064b and SC-034 are **per chapter** — “*an accuracy figure measured on one chapter is not evidence about another*”. This reference covers **chapter 01 only**, so it supplies T135's floor for chapter 01 and for **no other chapter**.

The estimator unit is the same finding on both sides, and getting it wrong is worse than omitting the interval. Whoever computes an interval — over the speech WER of (c) or over an OCR figure under T133 — uses the **window/sample unit (n = 30 clusters)**, never the word. Words inside one 30 s window are not independent draws, and WER is not a proportion: insertions let $(S+D+I)/N_{\text{ref}}$ exceed 1, which no binomial admits. At the limit of perfect intra-window correlation a word-level interval is narrower by up to $\sqrt{(N_{\text{hyp}}/30)} \approx 8\times$ and would **overstate** precision. This is now carried in three machine-readable places as well as here: the plan's own `estimator_unit` field, `accuracy.json`'s `sample.estimator_unit`, and T135's note.

(f) A MACHINE CROSS-CHECK WAS RUN OVER THE SAME 30 WINDOWS ON 2026-09-04, AND IT DOES NOT MEASURE SC-002. IT IS RECORDED HERE SO THAT NOBODY RE-DISCOVERS IT LATER AND MISTAKES IT FOR (b). The operator asked for the second transcript to be produced *by a locally-run model, or by bridging to the session's own models* — explicitly not by sending the media anywhere. That was done with the strongest local path available; what it produced is **inter-engine agreement**, not accuracy, and every artifact says so in its own words.

What was run. Engine **B** — `whisper.cpp / ggml large-v3-turbo-q8_0`, CPU-only, `-nfa` for word timestamps, settings matched to engine A (`-bs 5 -tp 0.0 -mc 0`) — over **exactly** the 30 planned windows, via the existing `workshop/pipeline/run_audit.sh` against a

copy of the plan (run_audit.sh writes engine_b.json back into the plan it is given; the canonical accuracy-plan.json is byte-unchanged, sha256 dda4aa4b... before and after). 30 of 30 windows produced output, **0 failures**; **5,853 s** of wall clock for **900 s** of audio (per window min/median/max **96 / 178 / 318 s**), under load average 10–23 from other work on the box.

The figure, under its honest name — INTER-ENGINE WORD DISAGREEMENT, n = 30 window clusters. (sub + b_only + a_only) / N_engine_B = **0.0858** (171 / 1,992), agreement 0.9142; percentile **bootstrap 95 % over window-level pairs under a ratio estimator: [0.0671, 0.1086]**, 10,000 reps, seed 0. The unit is the **window**, never the word, for the reason (c) already states. Composition: **53** substitutions, **65** engine-B-only, **53** engine-A-only. Engine A contributed **1,980** words on the same windows. **That is not the same count as (a)'s N_hyp = 2,012, and the difference is a containment rule, not a discrepancy:** (a) assigns a word to a window by its **midpoint**, while the scorer's a_words_in requires the word to be **fully contained** — 32 words straddle an edge. verify-accuracy.sh uses the containment rule, so **1,980** is the number a WER over these windows would divide by, and both figures are correct about different questions. **0 of 30** windows had zero disagreement; the worst four windows sit at **0.353 / 0.321 / 0.197 / 0.139** (offsets 4116.83, 4761.1, 5320.07, 5887.62) — those are review targets, which is the one thing this number is genuinely good for.

The boundary artifact was measured rather than argued away, and it is NOT what produces the figure. Engine B saw each window as an isolated 30 s cut, so a word straddling an edge was halved for B while engine A placed it outside the window. Recomputing with **1.0 s shaved off both ends of both sides** gives **0.0932** (171 / 1,834), CI **[0.0727, 0.1189]** — the rate goes **up**, not down, and 9 of 30 windows gain disagreements when trimmed. The disagreement lives in the window interiors.

Read against pipeline/CALIBRATION.md's 3.19 %, this is the more useful half of the result. That figure came from one 300 s window the calibration itself records as uniformly clean (no_speech_prob 0.0000 on all 44 segments). Sampled across the whole session under (d)'s temporal design the same quantity is **2.7×** larger. **Both figures are inter-engine divergence and neither is a WER** — CALIBRATION.md's own U4 row says so of its own number — but a quantity that nearly triples when the sample stops being one clean window is a warning about extrapolating from that window, not a reason to start quoting the new one instead.

What was built, and the marker that keeps it from becoming a bluff. workshop/pipeline/engine_b_crosscheck.py (registered exempt in workshop/platform/gates/check-registry-001.tsv, same class as compare_engines.py: a measurement, not a gate) emits the artifact in the plan's own reference_schema shape and computes the comparison. The artifact carries PROVENANCE: MACHINE, provenance, is_human_reference: false, not_a_reference, machine_generated, sc002_eligible: false and a prose WARNING at top level, **repeats provenance: machine on every window** — because a scorer reads windows[] and nothing else — and its filename ends .NOT-A-REFERENCE.json. **The marker is ENFORCED, not trusted:** workshop/scripts/verify-accuracy.sh gained a PROVENANCE_GUARD that fails a machine-marked reference as **reference_not_human, exit 2, writing no accuracy.json**. Measured against the real chapter, not only in a fixture:

```
bash workshop/scripts/verify-accuracy.sh 01 --reference
<artifact> → 2, and find workshop -name 'accuracy*.json' still
returns exactly one file, the plan. The §1.1 paired mutation is in --
selftest and it is the load-bearing half — delete the guard and the
same artifact scores WER 0.0000, accuracy 1.0000, which is
precisely the bluff. bash workshop/scripts/verify-accuracy.sh --
selftest exits 0 covering: absent reference → 2 (twice), perfect → 0,
degraded → 1, machine top-level marker → 2, machine per-
window-only marker → 2, no accuracy.json after either, and both
mutants caught.
```

The bridge option was assessed and it is NOT available here — measured, not assumed. (i) This session's harness cannot take audio at all: Read on a synthetic 2 s tone WAV returns *"This tool cannot read binary files"*, and a tool search over every deferred/MCP tool for audio, speech or transcription returns none. The probe was deliberately run on a generated tone, never on the recording. (ii) No local generative model can either: ollama list holds 4 models — one 3B text instruct (ollama show reports capabilities completion, tools, no vision, no audio) and three embedders. So the operator's second option is foreclosed by capability, and the first option is what was executed.

WHAT THIS CHANGES ABOUT (b): NOTHING. research/transcription.md §5.1 rejects a second ASR engine as the reference **by name** — *"the most tempting shortcut available and it is a bluff"* — because two Whisper-family models share training data and agree on the same hallucinations. Agreement bounds the **union** of the two engines' error from **below**, localised to windows; it cannot bound engine A's error from **above**, which is the only thing an

accuracy figure means. **SC-002 and FR-004 remain UNMET, publish precondition B2 remains unmet, GET /api/chapters/01/accuracy correctly still reports measured: false, wer: null, and spec 002's FR-064a/T135 floor is still absent.** Artifact (git-ignored, private speech, reproducible from `run_audit.sh + emit`): `workshop/pipeline/audit/sc002/engine-b-machine-cross-check.NOT-A-REFERENCE.json`. Counts-only report, no transcript text — enforced in code by an assertion that refuses any string outside a fixed allow-set, so a leaked word aborts the write rather than shipping: `workshop/evidence/verify-accuracy/engine-b-cross-check-2026-09-04.json`.

THE BOX STAYS UNTICKED, AND NEITHER (a) NOR (d) NOR (f) BEING DONE IS A REASON TO TICK IT. SC-002 demands a *published measured figure*; `accuracy.json` **does not exist**, B2 is **not met**, and `bash workshop/scripts/verify-accuracy.sh 01` still exits 2 — “*UNDETERMINED: --reference is required*” — which is the correct answer, not a failure. The census in (a) is the method half of FR-004, and re-emitting the plan in (d) improved the sampling frame without measuring anything. **An unticked box with an honest note beats a ticked one that lies.** What remains: **(b) on the operator** — 1–2 hours of blind transcription, blocked by nothing else, now unblocking **two** criteria; **(c) on a T112 change**, optional against SC-002/FR-004; **(d) DONE**; **(e) recorded in both directions, waiting on (b)**; **(f) DONE and it moves nothing** — a machine cross-check exists, is named honestly, and is refused by the scorer as a reference]**



T038 [US1] [TDD] [THE PATH NOT BUILT NOTE IS WITHDRAWN — re-measured 2026-09-03 while working T040. It read “*workshop/scripts/redact.sh does not exist and no equivalent landed anywhere; it also depends on T012's writer, which is itself unbuilt*” and both halves are now false: `workshop/scripts/redact.sh` is present and executable (2026-09-02 20:38), it forwards to `platform/backend/cmd/workshop-redact` with the child's exit status passed through untouched, and the append-only logs it writes hold 10** passage records (`curriculum/redactions.jsonl`) and 1 span record (`curriculum/redactions-spans.jsonl`). It has run for real against Chapter 1 — evidence works `hop/_evidence/redact/`, 13 runs, the applying one exit 0 at targets reached (8/8). ~~The box stays unticked and this is NOT a completion claim: what was measured is existence and one successful application, not this task's full contract — the —pids—file/—unredact-surfaces, the exit-1 and exit-2 branches and the \$1.1~~

paired mutation were not exercised here. — **BLOCKER:** none but the work — the PATH NOT BUILT note was already withdrawn; redact.sh exists · **OWNER: implementer** — unblocked today — EVIDENCE: workshop/scripts/redact.sh (forwards to platform/backend/cmd/workshop-redact, exit passed through). All three exit values DRIVEN 2026-09-08: --help-> rc 0; --dry-run --pid <absent>-> rc 1 PROBLEM: passage: identifier is not in the registry; WORKSHOP_EVIDENCE_DIR=/nonexistent/ev --check-review --chapter 01-> rc 2 (E2 pre-flight, printed before any work). Full contracted surface present at :49-:84: --pid, --pids-file, --span, --unredact, --ratify, --review-only, --check-review, --list-derived-leaks, --dry-run. It has applied for real: workshop/curriculum/redactions.jsonl holds 210 records, redactions-spans.jsonl 1. Paired-mutation cover: it is the required entry point of registered check verify-redaction-propagation.sh, run 2026-09-08 -> rc 0. (population: in_process) (FR-039)

THAT CAVEAT IS WITHDRAWN AS THE REASON — every branch it names was exercised on 2026-09-03 and every one holds. THE BOX STILL STAYS UNTICKED, FOR ONE DIFFERENT AND MUCH NARROWER REASON, NAMED AT THE END. Driven live against the real registry with --dry-run and a scratch WORKSHOP_EVIDENCE_DIR; no chapter content, no pid and no reason text is reproduced here:

probe	rc	contracted
--pid <absent-from-registry> --reason ... --by ...	1	✓ “exit 1 when a supplied pid is not in the registry”
already-suppressed pid under a DIFFERENT reason code	1	✓ “or a pid is re-redacted under a different reason” — refuses rather than picking one
same pid under the SAME reason code	0	✓ idempotent; the log is append-only and appends nothing
--reason/--by omitted	1	“an unexplained or unattributed suppression cannot be reviewed”

probe	rc	contracted
WORKSHOP_CURRICULUM_DIR=/nonexistent/...	2	✓ “exit 2 when the registry is unwritable” — unreadable half
registry dir chmod -w, real (non-dry-run) write	2	✓ unwritable half: permission denied, not a finding about the corpus
WORKSHOP_EVIDENCE_DIR=/nonexistent/ev	2	✓ §2.5 E2 — refuses BEFORE doing any work
--pids-file F (with a # comment line)	0	✓ the surface exists and parses
--unredact	0	✓ the surface exists and runs

The full contracted flag surface is present in workshop/scripts/redact.sh — --pid (repeatable), --pids-file, --reason, --by, --unredact, plus --span, --purge, --dry-run, --json. **§1.1 paired mutation EXISTS AND WAS RUN:** bash workshop/platform/gates/prove-redaction-propagation.sh → rc 0, all 6 mutations were caught — M3 “a refused decision is reported as success”, M4 “an unreadable registry is reported as a corpus problem instead of undetermined”, M6 “one of the eight propagation targets is never registered”; and verify-redaction-propagation.sh → rc 0 across A1–A10b. The registry-half writer this task appends to is T012’s submodules/passage/pkg/passage/redaction.go, now ticked.

THE ONE REMAINING CLAUSE, AND IT IS THE WHOLE REASON THE BOX IS UNTICKED: “and marks the live generation as requiring a rebuild” IS NOT IMPLEMENTED. Measured 2026-09-03: grep -n 'rebuild\|generation' workshop/platform/backend/cmd/workshop-redact/main.go returns **zero** lines, and no propagation target is the index generation — the eight are exports, terms, areas, questions, lesson_sections, mentions and their siblings in internal/redaction/plan.go. The burden is left with the caller **in the code’s own words:** workshop/platform/backend/pkg/index/generation.go:58–:60 — “until the rebuild that removes a redacted passage has been [built] ... Callers that redact must therefore treat the window between Build’s start ...” — **and no caller does it.** That is the gap between a redaction being RECORDED and a redaction

being EFFECTIVE against a live generation, which is exactly what T059’s R5 half contracts. Do not tick this until a redaction marks the live generation stale]** Implement workshop/scripts/redact.sh per contracts/pipeline-cli.md §4.6 (FR-039) — --pid P (repeatable), --pids-file F, mandatory --reason CODE and --by NAME, --unredact; it appends to the redactions log built in T012, materialises passages.redacted, and marks the live generation as requiring a rebuild. Exit 1 when a supplied pid is not in the registry or a pid is re-redacted under a different reason; exit 2 when the registry is unwritable or the rebuild that makes the redaction effective could not be started. It lands in THIS phase, not a later one, because Chapter 1’s transcript exists from T030 onward and FR-039 blocks export before review



T039 [US1] [TDD] [TICKED 2026-09-03. THE PATH NOT BUILT NOTE IS WITHDRAWN — re-measured 2026-09-03 while working T040. It read “redact.sh does not exist, and find workshop -name 'redaction-review.json' returns zero files”; that find now returns exactly one file, workshop/chapters/01/redaction-review.json (3,219 B, schema: redaction-review/v1, reviewed_at: 2026-09-02T19:51:23Z, decision: redactions_recorded), and bash workshop/scripts/redact.sh --check-review --chapter 01 exits 0** with “is fresh; B5 is met for chapter 01”. Publish precondition B5 IS met for chapter 01 — the statement to the contrary in redaction-review-summary.md was withdrawn there on the same date. (population: in_process) (FR-039)

THE “BOX STAYS UNTICKED” CAVEAT IS ITSELF NOW WITHDRAWN BY NAME, AND THE TICK IS EARNED BY EXECUTION. It read “the stale-review exit-1 branch, the”none required” decision path and the §1.1 paired mutation were not exercised here; only the fresh-review path was”. All three were exercised on 2026-09-03, against the REAL review artifact copied into a scratch WORKSHOP_CHAPTERS_DIR so the module tree was never written to. The freshness rule is three-valued and every branch was DRIVEN, not read:

probe	rc	what it printed
--check-review --chapter 01 (unmodified)	0	is fresh; B5 is met for chapter 01
review absent (--chapter 99)	1	no review artifact at ...; B5 is NOT

probe	rc	what it printed
		met. Recording "none required" is a valid decision; skipping the review is not.
reviewed_at back- dated to 2020-01-01T00:00:00Z	1	the review ... is STALE; B5 is NOT met.
one artifact's sha256 corrupted, mtimes untouched	1	... is STALE — content drift, not only mtime
one artifact renamed to a path that cannot be measured	2	... could not be checked

platform/backend/internal/redaction/review.go is where that lives: FreshnessState :200, the three constants :205/:208/:210, Freshness.Exit() :220 mapping fresh/stale/undetermined onto 0/1/2, CheckFreshness :239. **The “none required” path is implemented AND guarded in both directions:** DecisionNoneRequired :50, derived automatically at cmd/workshop-redact/main.go:474, refused at review.go:123 when the registry names suppressed rows (driven live: rc 1, review says "none_required" while naming 10 passage(s) and 1 span(s) as suppressed; the two statements contradict each other) and refused in the mirror direction at :131; TestReview_RefusesAContradictoryDecision PASSES. **The §1.1 paired mutation EXISTS AND WAS RUN:** bash workshop/platform/gates/prove-redaction-propagation.sh → rc 0, all 6 mutations were caught, of which **M5 is this task's own** — “the review is recorded in the report but no artifact is written” (rc 1) — and its gate bash workshop/platform/gates/verify-redaction-propagation.sh → rc 0 with **A7** (“every refusal returns its own code, and 2 is never collapsed into 1”) and **A8** (“-review-only writes the artifact, changes nothing else, and goes stale when an artifact changes”) both green. **Honest boundary (§11.4.6): a SUCCESSFUL none_required write could not be driven on this tree** — chapter 01 genuinely has redactions, and no second chapter exists in the registry (-chapter 02 → rc 2, no registry row carries scope "02", which is the honest answer). The path is proven by its guard, its derivation site and A8's fixture chapter, not by a live none_required artifact on this corpus. Separately, this task's parenthetical “(§4.7, enforced by

T087)” is a POINTER, not a deliverable of this task — **T087 is unbuilt, so B5 is checkable on demand but is enforced by no publish path.** The warning below still stands — do not mistake workshop/curriculum/publication-reviews.jsonl for this artifact: it is feature 002’s per-AREA publication review, keyed on area_id and read by pkg/knowledge/reviews.go — a different artifact for a different obligation]** Implement redact.sh --review-only and the review artifact workshop/chapters/NN/redaction-review.json per contracts/pipeline-cli.md §4.6 and contracts/passage-contract.md §7.3 R7 — recording “none required” is an explicit, valid decision; **skipping the review is not**, and a review older than the transcript it reviews is stale and exits 1. This artifact is publish precondition B5 (§4.7, enforced by T087); without it FR-039 is decorative



T040 [US1] [REVIEW] [TICKED 2026-09-03 ON MEASUREMENT — ALL THREE PARTS CLOSED. PART 2, THE DEFERRED SEVEN, LANDED; see the note below for the counts and for the two things that did NOT close with it.] Record the Chapter 1 redaction review and apply whatever it identifies, BEFORE the transcript is exported, published, served or committed (FR-039). This is not a fixture and not a hypothetical: the Chapter 1 recording features an identifiable third party, which is the reason FR-039 exists. Re-emit workshop/curriculum/chapter-01/transcript.md after redaction and assert that no redacted passage’s text or machine_text appears anywhere in it (§7.3 R2). No later phase may publish, export or serve Chapter 1 until redaction-review.json exists and is newer than the transcript. (FR-039)

[PART 2 CLOSED 2026-09-03 (later the same day). THE HEADING BELOW — “THE BOX STAYS UNTICKED, AND THE SECOND PART IS WHY” — IS WITHDRAWN BY NAME, not deleted, and everything under it that describes the REFUSAL is kept because the refusal was correct and is the whole lesson. Counts only, per docs/content-boundary.md.

What was refusing, and what changed. Plan.Apply is all-eight-targets-or-nothing, so the two surfaces that reported PROBLEM cancelled every write. Neither was worked around; both were fixed.

surface	before	after	what was actually wrong
taxonomy residual	34	0	three distinct causes, measured separately:

surface	before	after	what was actually wrong
			25 of the 34 were a withdrawn term matching INSIDE a longer, legitimately visible token (bytes.Contains over the raw line); 8 were an area.external_key, which is DERIVED from the member terms and so still carried a term the unlink pass had just removed; 1 was a live-evidence term row whose own name carries a withdrawn term the detector reported ONE ROUND. Suppressing a row moves its own text onto the withheld side, so each application produced a new, smaller finding — it never converged because nobody was ever shown the size of the decision contradiction and discarded-duplicate rows carry their evidence under names that are not evidence_pids; 85 suppressed identifiers survived there (84 in outside_passages, 1 in overlap_passages). proposal_member_terms
registry-derived undecided	48	0	
proposal_member_terms / proposal passage lists	uncovered	swept	

surface	before	after	what was actually wrong
			itself held 0 withdrawn terms on this tree — the gap was real, its yield today is zero, and both halves are stated

The fixes, one line each. The withdrawn-term sweep now walks the DECODED row and tests every string value with a word-boundary rule, reporting the JSON member path — **not** a length threshold, which would have stopped protecting exactly the short strings most likely to be initials. An area's derived `external_key` is re-keyed on its own id when a member term is dropped. A term row whose own NAME carries a withdrawn term is withdrawn too, iterated to a fixed point (**1** row on this tree, and **it still has 3 live evidence passages** — that is a real cost to index completeness, counted separately as `TermsWithdrawnByName` and reported separately). `CloseDerivedLeaks` iterates the minted-row detection to a FIXED POINT and reports the transitive closure; it terminates because the suppressed set only grows and is bounded by the registry.

What was applied. `curriculum/redactions.jsonl` went **10 -> 142** entries: **84** passages under six per-finding reason codes (F8 2, F9 6, F12 3, F13 2, F14 69, F15 2) plus **48** minted registry rows (36 `kg_term`, 12 `kg_area`) under `derived_from_withheld_passage` — the transitive closure, cross-checked against an independent computation and identical. **F16 needed no corpus change:** the listing boundary it names is already in force. Final propagate exit **0**, targets reached (8/8), 0 PROBLEM · 0 UNDETERMINED, and a re-run is idempotent.

Part 3 re-measured after the apply. Residual of every suppressed passage's text, shape-aware, across `transcript.md`, both timing sidecars, `exercise-01.md`, its sections sidecar, `taxonomy.jsonl` and `area-materials.jsonl`: **0 / 0 / 0 / 0 / 0 / 0 / 0**. §7.3 R2 holds. --check-review --chapter 01 exits **0**.

TWO THINGS DID NOT CLOSE, and neither is a tooling defect.

1. **A regeneration from clean re-derives 2 of the 27 withdrawn index terms**, and it does so from passages that are still VISIBLE (3 visible occurrences each), not by reading withheld text — 25 of 27 do not come back. Whether those passages should also be

withheld is a REVIEW decision about the corpus, not something a redaction tool can settle. (2) The regeneration's own reconcile_and_taxonomy stage exits 2 on a 180-second go run ./cmd/knowledge-mint timeout, so the row-level rebuild is UNDETERMINED; the extract stage completed (rc 0) and is what the sweep above measured.

~~[THIRD PART CLOSED 2026-09-03; THE BOX STAYS UNTICKED, AND THE SECOND PART IS WHY.~~ Read the split before reading the exit codes. Counts only below — no chapter content, no personal name, no pid, no transcript line number, per docs/content-boundary.md.

The path question is SETTLED and the old note is WITHDRAWN as a live blocker. The 2026-09-02 note said this task named workshop/chapters/01/transcript/transcript.md, “which does not exist”. Re-measured 2026-09-03: that *directory* now exists and holds exactly one file, accuracy-plan.json (see T037); transcript/transcript.md inside it still does not, and never did. The transcript landed at workshop/curriculum/chapter-01/transcript.md (107,847 B) — the file this task now names. **This was a RENAME, not an imagined artifact and not an ungenerated one:** git -C workshop log -- curriculum/chapter-01/transcript.md shows it tracked since 5b611f97d (2026-09-01). The path text is correct as it stands and the third part was doable.

Part 1 — record the review: DONE. workshop/chapters/01/redaction-review.json exists (3,219 B, schema: redaction-review/v1, reviewed_at: 2026-09-02T19:51:23Z, decision: redactions_recorded, 9 artifact digests, 10 pids, 1 span). bash workshop/scripts/redact.sh --check-review --chapter 01 exits 0 — “*is fresh; B5 is met for chapter 01*” — re-run 2026-09-03. Publish precondition **B5 is met**. T039's note claiming find workshop -name 'redaction-review.json' returns zero files is stale, and so is T038's claiming workshop/scripts/redact.sh does not exist; both were written before 2026-09-02 20:38.

Part 3 — re-emit and assert R2: DONE, and it was already done before this session measured it. The re-emit is not a separate command: the applying run's exports surface rewrites the document and both sidecars in place. Evidence workshop/_evidence/redact/20260902T194901Z-942251/ — exit 0, “*APPLIED exports:transcript.md ... 10 passage(s) suppressed across the document and its two sidecars (30 artifact-level change(s))*”, plus APPLIED spans:... 1 non-passage span(s) suppressed, targets

reached (8/8), 6 applied · 2 clean · 3 absent · 0 PROBLEM · 0
UNDETERMINED. The artifacts carry 11 · 10 · 10 [REDACTED] markers
respectively (11 in the document = 10 passages + the title span), so
the assertion below is not vacuously true of an empty file. **The R2
assertion, run 2026-09-03: 10 redacted rows × 2 fields × 3
surfaces, R2 violations 0, exit 0.** Re-derive:

```
python3 - <<'PY'
import json,re,unicodedata
C='workshop/curriculum/'
red=[d for d in
    map(json.loads,filter(str.strip,open(C+'passages.jsonl'))))
    if d.get('redacted')]
n=lambda s:re.sub(r'\s+',',',
    ',unicodedata.normalize('NFKC',s)).strip().lower()
arts=[C+'chapter-01/transcript.md',C+'chapter-01/
    transcript.segments.json',C+'chapter-01/
    transcript.words.json']
hay={a:n(open(a,encoding='utf-8').read()) for a in arts}
bad=[(d['pid'],f,a) for d in red for f
    in('text','machine_text')
    for a in arts if isinstance(d.get(f),str) and d[f].strip()
    and n(d[f]) in hay[a]]
print('redacted rows',len(red),'| surfaces',len(arts),'| R2
    violations',len(bad))
raise SystemExit(1 if bad else 0)
PY
```

Whole-string absence is the literal contract, and it is a weak instrument on its own — so two stronger ones were run and both are recorded, including the one that produced a finding. (i) **Longest surviving common substring** between any redacted text/machine_text and any of the three artifacts, over all 60 (row × field × surface) pairs: **17 characters**, and at most **5 words**. (ii) A **5-gram survival sweep** over 756 shingles reported **4 hits — one distinct 5-gram**, in the document and its segment sidecar, matched from both fields (all 10 redacted rows have text == machine_text, so a field-level hit necessarily doubles). **That one is a measured FALSE POSITIVE, not a leak:** the same 5-gram occurs in **2 non-redacted registry rows**, its two occurrences in the document sit at lines with no [REDACTED] marker within ±3 lines, and it is ordinary recurring speech that happened also to fall inside a redacted passage. It is written down rather than dropped because a sweep that only ever reports zero is indistinguishable from a broken one.

Independent corroboration at the level that actually matters — the identifier strings themselves. Each Category-1 token was recovered *by index* from the pre-redaction sidecar blob at 5b611f97d (never typed, never printed) and counted, word-boundary and

case-insensitive over deduplicated match spans, across all four in-scope artifacts, before and after. Every third-party identifier goes to **zero**: PARTICIPANT-A 13→0, THIRD-PARTY-B 15→0, THIRD-PARTY-C 3→0, THIRD-PARTY-D 3→0, ORG-1 3→0. INSTRUCTOR reads 10→6 and **that is the recorded decision F3 = KEEP**, not a miss — the repository owner’s own name, which he is the one entitled to publish. Totals 47 → 6, and all 6 remaining are F3. The pre-redaction column reproduces the private review’s own occurrence table exactly — 47 total on the same per-artifact split — which is what makes this a corroboration of that review rather than a second opinion about it. **A first pass of this count read 75 → 6 and is WITHDRAWN by name: it matched two case variants of the same string with a case-insensitive regex and counted each occurrence twice for two of the six labels (13→26, 15→30). The instrument was wrong, not the tree.**

Part 2 — apply whatever it identifies: NOT DONE, and this is the only reason the box is unticked. The private review records 17 decisions: 13 REDACT, 3 KEEP, 1 NOT REMEDIABLE. Operator decision 26 applied the **Category 1 direct-personal-identifier subset only** — F1 (the title span) and F2, F4, F5, F6, F7 — **6 of the 13 REDACT decisions. 7 REDACT decisions are DEFERRED and unapplied: F8, F9, F12, F13, F14, F15, F16** — Category 2 indirect identifiers and Category 3 third-party-confidential material, which are *descriptive* findings that no pid-level token redaction reaches. F3, F10 and F11 are KEEP; F17 is NOT REMEDIABLE and already public (docs/content-boundary-incident-2026-09-01.md is its class). The deferral is recorded in the review artifact’s own note field, so it is visible rather than silent — but a deferral is not an application, and “apply whatever it identifies” is not satisfied while 7 REDACT decisions stand. **Resuming those 7 is an operator decision and is not taken here.** One measured mitigation, recorded without being counted as closure: F8’s *string* (“infrastructure named after PARTICIPANT-A”) reads **0** occurrences post-redaction across all four artifacts, because it is the same token as F2 — but F9 and F12–F16 are untouched.

APPLICATION OF THE DEFERRED SEVEN WAS ATTEMPTED 2026-09-03 UNDER OPERATOR AUTHORIZATION AND DID NOT LAND. The box therefore stays unticked, and the reason is a TOOL REFUSAL, not a judgement about the findings. Counts only, per this note’s own rule — no pid, no line number, no content. The seven were resolved to the registry from the review’s own cited locations (§4b I2/I3/I6, §4c C1–C6, including C4, which the §5 F14 row folds in): **84 distinct passages** across two source artifacts — F8

2, F9 6, F12 3, F13 2, F14 69, F15 2, after resolving 4 passages cited by two findings to the narrower third-party ground. All 84 measured **unredacted** and text-present on disk before the attempt. Six reason codes were prepared, one per finding, so each decision would land in the append-only log with its own - - reason and - - by rather than as an untracked edit. **redact.sh refused, three times, on two independent surfaces, and Plan.Apply returns before writeAll when any surface reports PROBLEM — so the run is all-eight-targets-or-nothing and NOTHING WAS WRITTEN.** (1) **registry-derived** — a value-level pass over MINTED rows, added to plan.go on 2026-09-03 and uncommitted at the time of measurement — reported **29 of 9144 minted rows** carrying a withheld-only string and undecided, from a run-time roster of **36** strings; the surface states in its own words that suppressing them is an operator decision. This is a genuine cascade: the 84 raise the withheld-token count from 108 to **909**, so deciding one wave mints the next. A first wave of **19** was decided independently the same day under `derived_from_withheld_passage` (registry redacted moved 10 → **29**), and the cascade did not close. (2) **taxonomy** — this one is in COMMITTED code and is the binding blocker, because it fires on the 84 alone and does not cascade: withdrawing a term row whose last evidence is suppressed leaves **34** residual lines still carrying a withdrawn index term. Two measured causes: `proposal_member_terms` on contradiction (13) and discarded-duplicate (355) rows is not one of the eight propagation targets — a limitation `taxonomy.go` documents in its own words as a *reported* violation rather than a silent survival — and **26** term rows would be withdrawn of which **5 carry a 3- or 4-character string**, so a `bytes.Contains` residual sweep collides with longer, legitimately-visible terms that merely contain them. **Neither was worked around. No gate was weakened, no residual sweep was loosened, no corpus file was hand-edited, and the tooling was not built from HEAD to dodge the newer surface** — `Apply` at HEAD differs from the working tree by exactly one added line, and choosing the weaker instrument to obtain a write would be the same defect as disabling the gate. **F16 is the one of the seven that needs no corpus edit, and it is ALREADY SATISFIED WITH A GATE:** its remedy is a listing boundary, and `GET /api/chapters/{c}` discloses no source filename — `curriculum.MaterialID` is a one-way sha256-derived m-... fingerprint, `Material.file` is unexported so it never reaches the wire, the title is a generic kind-plus-ordinal, and `pkg/curriculum.TestMaterialIDNeverContainsSourceWords` is the durable assertion. Verified alongside: `verify-redaction-propagation.sh 0` (A1–A10b, on the INVENTED fixture corpus —

which is exactly why it stays green while the real corpus refuses),
verify-limits-completeness.sh **0** (15/15), the pipeline/extract suite
264 tests OK in pipeline/venv (system python3 lacks wordfreq and
errors 32 of them — an environment fault, a 2, never a failure), and
the gate-attachment closure check **31 ids, unattached: 0**. A
hardlink-safe backup of all 49 corpus files was taken first, and the
corpus was re-verified byte-identical to it afterwards. **What is
owed before the seven can land: close the taxonomy propagation
gap for proposal_member_terms, decide how a withdrawn short
term is swept without either colliding or loosening the sweep,
and carry the registry-derived cascade to a fixed point. All
three are operator or tooling decisions, not corpus decisions.**

**FR-039's ordering clause was violated once, historically, and
redaction did not undo it.** This task says “BEFORE the transcript is
exported, published, served or **committed**”. Measured over the
file's whole history: 5b611f97d (2026-09-01) committed the
transcript with **6 of the 10** currently-redacted passages' exact text
present and **0** redaction markers; 692a27aa8 (2026-09-02) is clean at
0/10 present and 11 markers, and equals the working tree. The
other 4 are **COULD NOT DETERMINE** — the registry's text is
today's rendering and the 2026-09-01 blob is 788 B larger, so their
absence there is not evidence they were absent from that
rendering. The commit reached the workshop remote, which is
PRIVATE, so this is not a public disclosure and is a different class
from the umbrella's own incident — but a push is publication and
the review came second. **This is containment, not remedy, and it
is not closeable by anything this task can do.**

Not done here, stated plainly: no durable gate asserts R2 against
the REAL chapter. G-PID-5 (workshop/platform/gates/verify-
redaction-propagation.sh) asserts the whole of R1–R7 against the
invented fixture-corpus, which is deliberate — that generator
exists so no gate has to read private material. The assertion above
is therefore a recorded, reproducible **[REVIEW]** measurement with
its recipe published, not a registered check, and no G- identifier was
minted; the gate-attachment closure check is unaffected (re-run
2026-09-03: **31 ids, unattached: 0**])**



T041 [US1] [REVIEW] Human review checkpoint: speaker
attribution is HUMAN (D-TRANS-2 — the recording is dual-mono at
–90.3 dB and AGC-flattened, so both diarization cues are
measurably absent) — **[BLOCKED ON THE OPERATOR. Measured
2026-09-03; NEW note.** This is human labour by construction:**

D-TRANS-2 measured both diarization cues absent, so no tool on this host can attribute a line to a speaker, and no agent can substitute for the person who was in the room. Nothing else blocks it. A TRAP, AND IT IS THIS FILE'S OWN COLLISION WARNING BITING: workshop/docs/session-evidence/t041-decision-packets.md (untracked, written 2026-09-03) IS NOT THIS TASK. Its subject is R3 contradiction typology over taxonomy terms — that is 002:T041. Its own opening line says “*T041 remains []*”, and a reader who resolves that bare id against this file will conclude something false about speaker attribution. Cite it as 002:T041, per standing rule at the head of this document. No artifact recording a human speaker-attribution review exists for 001:T041] — BLOCKER: operator decision, and it is human labour — speaker attribution is HUMAN by D-TRANS-2: the recording is dual-mono at -90.3 dB and AGC-flattened, so both diarization cues are measurably absent. No amount of engineering substitutes for someone listening · OWNER: operator — nothing an agent can do closes this (FR-005)



T111 [US1] [TDD] [PATH NOT BUILT — workshop/scripts/transcribe.sh does not exist and no --resume/--from-parts/--chunk-seconds/--sample-seconds wrapper landed anywhere. Note it is contracted over T024–T029, whose modules moved (see each) and ~~two of which are unbuilt~~ — re-measured 2026-09-03, that count is now ONE: T025's chunker is built, proven and ticked, leaving T028's checkpoint/resume as the only unbuilt module beneath this wrapper. T023's reassemble.sh wiring (which --from-parts needs) is also still absent. RE-CONFIRMED the same day: workshop/scripts/transcribe.sh does not exist, and gate G-CLI-2** appears nowhere in workshop/**] Implement the workshop/scripts/transcribe.sh <chapter-slug> wrapper per contracts/pipeline-cli.md §4.1 over the T024–T029 modules — --resume, --from-parts, --chunk-seconds, --model, --threads, --sample-seconds (the calibration run T036 uses) and --dry-run. Its progress figure is projected **from the measured rate so far, never from the D-TRANS-1 estimate**, which is itself UNVERIFIED. Exit 1 for an unexplained coverage gap, a VAD-declared silence contradicting a measured long silence, a source (size, mtime, inode) change during the run, or an extract-videos.sh checksum mismatch; exit 2 for ffmpeg/ffprobe unusable, venv or model missing, a crashed helper, disk full, an unsatisfiable RAM cap, a held lock, interruption, or a part extract-videos.sh could not read. Gate G-CLI-2 (§2.4 S1–S4, §5): run the full pipeline and assert every source file's

(size, mtime, inode) is unchanged, since sources are opened 0_RDONLY and derived artifacts land only under chapters/NN/transcript/, curriculum/, _evidence/ and .locks/. **Paired mutation:** have transcribe rewrite the notes PDF in place; the gate MUST go red (FR-006) — **BLOCKER:** none but the work — transcribe.sh is unwritten. Note it is contracted over T024-T029, several of which moved · **OWNER: implementer** — unblocked today (FR-001, FR-029, FR-006, FR-033, SC-013, FR-032, SC-012)



T112 [US1] [TDD] Implement the workshop/scripts/verify-accuracy.sh <chapter-slug> wrapper per contracts/pipeline-cli.md §4.2 over T033's scorer — --reference (required), --windows 30, --seed (recorded, so the sample is reproducible), --normaliser (whose SHA-256 must match T034's frozen hash) and --min-accuracy — writing chapters/NN/transcript/accuracy.json, which is publish precondition B2. Gate **G-CLI-5** (§4.2, §5): run with --reference pointing at a nonexistent file and assert exit 2. **Paired mutation:** return 0 with wer: 0.0 when the reference is absent; the gate MUST go red. This is the most important line in §4.2 — **the absence of a human reference is 2, never 0** — because a command that cannot measure accuracy must not report that accuracy is fine, and until this has run GET /api/chapters/{c}/accuracy reports measured: false, wer: null (SC-002, SC-013). **RUN 2026-09-03, and the “most important line” was exercised for real rather than only in a gate: bash workshop/scripts/verify-accuracy.sh 01 exits 2 with UNDETERMINED: --reference is required, writes no accuracy.json, and says why in its own words. The tick was already earned by the build; what is new is that the behaviour is now confirmed against the real chapter and not only against the G-CLI-5 fixture.** Its --emit-plan path produced workshop/chapters/01/transcript/accuracy-plan.json — 30 of 30 seeded, stratified windows. **Honest boundary (§11.4.6), and it belongs to T037, not here: this wrapper emits no confidence interval and computes none, which T037 requires — see T037's own note** (population: in_process) (FR-004, SC-002, FR-033, SC-013, FR-032, SC-012)



T113 [US1] [TDD] **[PARTIAL — THE SCRIPT EXISTS AND INGESTS FOR REAL; ITS CONTRACTED FLAG SURFACE AND ITS GATE DO NOT. Measured 2026-09-03; NEW note.**** workshop/scripts/ingest.sh EXISTS, 499 lines, and is a genuine four-stage driver (build the chapter document + payload → split prose into doc_section/code passages → mint identities through the passage library → publish passages.jsonl where the server reads it),

with the three-valued exit contract stated in its own header at : 27. Six flags are contracted and ONE exists. `--write-anchors`, `--no-write-anchors`, `--kinds`, `--dry-run` and `--check-idempotent` have no case arm at all (measured by counting)-terminated arms); the only two arms are `--require-suspend` and `--corpus`, and `--corpus` sets `CORPUS_MODE=1` rather than selecting the D2 corpus this task names. Anchor writing is not implemented in any form — `grep -rl '<!-- pid:' workshop --include='*.md'` returns 0 files across the whole submodule (the same measurement that keeps T030 open), so `--write-anchors` has nothing to switch on. Gate G-PID-6 exists nowhere: `grep -rn 'G-PID-6' over worksh` `op/` and `submodules/passage/` returns zero, so I1–I6 — the run-twice, zero-mints, byte-identical-passages.jsonl idempotency assertion — is unproven, and so is its R5-unconditional paired mutation. It is also worth recording that this task is contracted “*over T016’s matching branches*”, and T016 is unwritten]

Implement the `workshop/scripts/ingest.sh` [`<chapter-slug>`] wrapper per [contracts/pipeline-cli.md](#) §4.3 over T010’s registry and T016’s matching branches — `--write-anchors` (default on for `worksh` `op/-owned` text), `--no-write-anchors`, `--kinds`, `--corpus` (D2), `--dry-run`, `--check-idempotent` — writing a sorted, byte-stable curriculum/`passages.jsonl` plus `passages.db`, anchors back into `workshop/-owned` text sources, and sidecars for anchorless formats. It implements [contracts/passage-contract.md](#) §8 exactly and restates none of it, so the two cannot drift. Exit 1 on a duplicate anchor (R1b), a foreign anchor (R1c), an unaliased rename of `workshop/-owned` code (R4), a non-idempotent second run, a pid collision, or a passage violating a field invariant F1–F7; exit 2 when the registry is unreadable or locked, the symbol source is unavailable (P-U1, T014), disk is full, or the run was interrupted. Gate **G-PID-6** ([contracts/passage-contract.md](#) §8.1): run `ingest` twice over unchanged inputs and assert **I1–I6** by diffing the tree and counting mints — zero pids minted, zero anchors written, a byte-identical `passages.jsonl`, every source byte-identical, no new index generation, exit 0. **Paired mutation**: make R5 unconditional so it mints whenever no anchor was *read this run*; the gate MUST go red on I1, I3 and I5 simultaneously (FR-027) — **BLOCKER**: none but the work — the script exists and ingests for real; its contracted flag surface and its gate do not exist · **OWNER: implementer** — unblocked today (FR-027, FR-037, SC-016, FR-033, FR-032, SC-012)

Checkpoint: transcript exists with a measured accuracy figure AND a recorded redaction review. Stop for human approval — everything downstream inherits this artifact's quality, and nothing downstream may publish Chapter 1 without that review.

Phase 4: User Story 2 — Browse and watch (Priority: P2)

Goal: the curriculum is browsable and the recording plays.

Independent test: start with the documented command, confirm Chapter 1 is listed, its materials readable, its recording plays.



T042 [US2] [TDD] **[PATH CORRECTED 2026-09-02 — this task named workshop/platform/backend/internal/store/chapter.go; neither it nor the internal/store/ directory exists. The store landed at workshop/platform/backend/pkg/curriculum/curriculum.go (type Store at :168, ComputeCoverage at :428), with its HTTP layer at internal/api/chapters.go. It is therefore in pkg/, which contradicts the File Structure row that argued for internal/ — recorded rather than reconciled here]** Implement the chapter model + store in workshop/platform/backend/pkg/curriculum/curriculum.go per [data-model.md](#) (FR-008)



T043 [US2] [TDD] Implement GET /api/chapters and GET /api/chapters/{slug} per [contracts/http-api.md](#) (FR-008)



T044 [US2] [TDD] Implement GET /api/chapters/{slug}/transcript returning passages with pid, timestamps, provenance and uncertainty (FR-009, FR-002, FR-003, FR-038)



T045 [US2] [TDD] **[PATH CORRECTED 2026-09-02 — this task named workshop/platform/backend/pkg/media/serve.go; neither it nor the pkg/media/ directory exists. Range-serve landed at workshop/platform/backend/internal/api/recording.go, delegating to http.ServeContent at :786 for Accept-Ranges (:769), 206, Content-Range, 416 bytes */total (:724) and If-Range, with multi-range rejected explicitly before it reaches ServeContent]** Implement local recording range-serve with seek in workshop/platform/backend/internal/api/recording.go — a local file with HTTP range support (decision D3), NOT a streaming service (FR-009, SC-003)



T046 [US2] [P] [SUBAGENT] Build the chapter list and detail views in workshop/platform/frontend/src/app/features/chapters/ (FR-008)



T047 [US2] [P] [SUBAGENT] Build the transcript reader with timestamp→recording seek in workshop/platform/frontend/src/app/features/transcript/ (FR-009, FR-002, SC-003)



T048 [US2] [P] Reuse design-system/learning-kit/ curriculum CSS (verified present, framework-free, on the --od-* token contract) rather than authoring new styling (FR-046)



T049 [US2] [TDD] Persist reader position/progress per FR-010 — **[TI CKED 2026-09-02 after independent re-measurement. Backend: workshop/platform/backend/internal/api/progress.go — type ProgressStore :46, Get :90, Put :110, ProgressHandler :156 serving MethodGet :167 and MethodPost :186. The route is live and gated: cmd/workshop-server/web_test.go:356–:358 fails if POST /api/progress 404s, and :364 holds every other verb to 404. Frontend: workshop/platform/frontend/src/app/core/progress.ts:4 cites FR-010 (T049) over a per-chapter local store. Honest boundary (§11.4.6): no Go test suite was executed for this tick — the assertions were read, not run]** (population: source) (FR-010)



T050 [US2] [TDD] Prove SC-003: transcript passage → recording seek lands within 5 s, in workshop/platform/frontend/src/app/features/transcript/seek.spec.ts (SC-003, FR-002)



T051 [US2] Write workshop/docs/quickstart.md and time a fresh-clone-to-running run against SC-004's 15-minute budget — *path is correct: workshop/docs/quickstart.md EXISTS. The MEASUREMENT is what is open, not the document: no fresh-clone-to-running wall clock appears in it, so SC-004 is unmeasured — [RE-MEASURED 2026-09-03 AND SHARPENED. The document is 20,937 B and grep -c 'SC-004' over it returns ZERO** — it does not cite the criterion it is supposed to satisfy. Its only timing figures are the budget claim restated at :3 and runtime latencies unrelated to setup (elapsed_s at :415/:435/:444, container uptime :495, index generation :553). The single SC-004 string anywhere under workshop/ is docs/session-evidence/phase9-brief.md:22, and that is about idempotency, not a timed run. workshop/evidence/ holds 5 directories and none records a clone run. PARTIAL, and the missing half is a stopwatch, not prose — but note it is not*

purely clerical: T005's `venv-setup.sh` does not exist, so a fresh clone currently contains a hand-built step that cannot be timed because it cannot be run. Fix T005 first or the figure measures a host, not a procedure] — **BLOCKER: needs a real fresh-clone-to-running run to be TIMED against SC-004's 15-minute budget** — the document exists; the measurement does not. T005 removed the last unscripted step (the `venv`, now 31.86 s), so this is now performable · **OWNER: implementer/operator** — unblocked as of T005, but needs a clean clone (FR-030, FR-012, SC-004)



T114 [US2] [TDD] [UNWRITTEN — NOTHING BLOCKS IT. Measured 2026-09-03; NEW note.** G-CLI-15 occurs 4× in contract `s/pipeline-cli.md (:701, :987, :991, :1012)` and in this file, and `grep -rl 'G-CLI-15' workshop` returns ZERO files — no gate, no proof, no registry row. The nearest instrument is `workshop/scripts/verify.sh:754–:762 (R2 stack-liveness)`, and it is exactly the bluff this gate exists to catch: it classifies from the verdict WORD that `status.sh` prints, never asking the container runtime whether the compose project's containers are running and never proving `/api/health` was answered through the PUBLISHED port. A green control plane therefore does not yet imply the infra came up. The mutation target exists and is buildable (`workshop/platform/orchestration/cmd/workshop-boot`, built into `platform/bin/workshop-boot` by `scripts/build.sh:82`), so the stub-that-starts-nothing mutation is available today] Prove gate G-CLI-15 ([contracts/pipeline-cli.md §4.8, §5](#)) — the **containers-actually-booted anti-bluff** (§11.4.76(5)): after `start.sh` exits 0, assert the container runtime reports the compose project's containers **running** and that `/api/health` was answered by one of them **through the published port**, never against a container's internal address, which would pass while the port mapping is broken. A green control plane MUST imply the infra was up. It lands in this phase rather than with T021 because this is the first phase in which the stack actually serves an HTTP endpoint to probe. **Paired mutation:** replace `workshop-boot` with a stub that exits 0 and starts nothing — reproducing `cmd/boot`'s measured behaviour exactly; the gate MUST go red. The mutation is not hypothetical: it is what the upstream CLI does today, which is why this is the gate that would have caught the withdrawn `cmd/boot` specification (T021) — **BLOCKER:** none but the work. Re-measured 2026-09-04: NOT STARTED — `grep -rl 'G-CLI-15' workshop` still returns ZERO files. The premise it needs was measured and holds: the stack is live, compose labels carry both `com.docker.compose.project` and `io.podman.compose.project`, and `/api/health` returns 200.

One design note for whoever takes it: `compose.yml` uses `network_mode: host`, so `NetworkSettings.Ports` is `{}` — there is no port map to inspect, and “through the published port” must mean probed from the host at the address the container wrote into the bind-mounted `platform/run/server.json` · **OWNER: implementer** — unblocked today (FR-012, FR-033, FR-032, SC-012)

Checkpoint: a reader can consume Chapter 1 without a terminal. Stop for approval.

Phase 5: User Story 3 — Find anything by meaning (Priority: P3)

Goal: semantic search with fast type-ahead and cross-references.

Independent test: issue meaning-based queries whose expected passages are known, including queries sharing no literal words with the target.



T052 [US3] [TDD] Implement the lexical FTS5 prefix index in `workshop/platform/backend/pkg/search/lexical.go` — measured p95 9.58 ms; FTS5 works in `modernc.org/sqlite`, already a dependency, so no new library (population: `in_process`) (FR-015, SC-005)



T053 [US3] [TDD] Implement `GET /api/suggest` backed by the LEXICAL path only — a query embedding measured 18–21 s under load, so semantics cannot meet SC-005’s 200 ms budget. Gate **G-HTTP-8** ([contracts/http-api.md](#) §3.6, §5): issue 200 `/api/suggest` calls and assert at the socket level that **zero** requests reach the embedding endpoint, and that `legs` is always exactly `{"lexical": ...}`. **Paired mutation:** add an embedding call to the suggest path; the gate **MUST** go red (population: `unstated`) (FR-015, SC-005, FR-032, SC-012)



T054 [US3] [TDD] Implement the semantic leg in `workshop/platform/backend/pkg/search/semantic.go` over the passage registry (FR-014, SC-007, SC-008)



T055 [US3] [TDD] [REVIEW] **[PATH CORRECTED 2026-09-02 — this task named `workshop/platform/backend/pkg/search/verdict.go`, which does not exist; the three-state contract landed at `workshop`**

`/platform/backend/pkg/search/envelope.go` (`StatusNoMatch :27`, `I5 implemented and named at :432–:435`, `I2/I3 at :372`) with its callers in `service.go` and the reindexing promotion in `degraded.go`] Implement the three-state contract in `workshop/platform/backend/pkg/search/envelope.go` per [contracts/http-api.md](#) invariants I1–I9 — including **I5: `no_match` requires EVERY enabled leg to have succeeded**; if a leg failed and survivors found nothing, that is unavailable, never “no results”. Three gates land here ([contracts/http-api.md](#) §5). **G-HTTP-1**: point the embedding endpoint at a closed port and assert `GET /api/search?q=x` returns **503** with status `:"unavailable"` and a `reason.code`; **paired mutation**: make the handler return `200 {"results":[]}` on backend error. **G-HTTP-2**: on that same failure assert the response body has **no** `results` key at all; **paired mutation**: add `"results": []` to the unavailable branch. **G-HTTP-3**: fail the semantic leg, let lexical return zero rows, and assert `status == "unavailable"` with `reason.code == "partial_failure_zero_results"`; **paired mutation**: change the rule to emit `no_match` when any leg succeeded. Every one of the three mutations **MUST** turn its gate red — the contract flags I5 as the invariant most likely to be “simplified” later (FR-019, FR-020, FR-033, FR-032, SC-012)



T056 [US3] [TDD] Parse and promote Lumen’s glued “No results found. | Warning: Index is being updated...” string to degraded, never forward it as a result. Gate **G-HTTP-4** ([contracts/http-api.md](#) §5): feed the search service that literal string and assert `status=="ok"` with `degraded.semantic=="reindexing"` when lexical has rows, and that the raw upstream string appears **only** under `degraded.evidence`. **Paired mutation**: forward the upstream string into `results[0].snippet`; the gate **MUST** go red (FR-020, FR-032, SC-012)



T057 [US3] [TDD] Prove gate **G-HTTP-7** ([contracts/http-api.md](#) §5) — the frontend half of the degradation contract, which the API cannot enforce and which is therefore stated in the contract and tested here: render each of the three envelopes and assert the no-results copy does **not** appear for the unavailable envelope, and the unavailable copy does **not** appear for `no_match`. **Paired mutation**: restore the `ai_interviewing` error handler shape, literally `error: () => loading.set(false)`; the gate **MUST** go red (FR-019, FR-020, FR-032, SC-012)



T058 [US3] [TDD] Implement index generations with a verification gate and atomic swap in `workshop/platform/backend/pkg/index/generation.go` — measured necessity: during a live rebuild chunks

moved while `last_indexed_at` still advertised the previous generation (FR-020)



T059 [US3] [TDD] [PARTIAL — R1 IS DONE ON BOTH LEGS AND PROVEN; R5 IS A COMMENT. Measured 2026-09-03; NEW note.**
R1, lexical leg: `pkg/index/generation.go:80–:101` drops `rec.Redacted` from the member set AND from the root hash at BUILD time — not at render time — with `passages_fts` exclusion by construction in `submodules/passage's BuildDB` (documented at `generation.go:8–:14`). Proven live: `verify-redaction-propagation.sh` → rc 0 with A10b R1 holds: a suppressed row is absent from the lexical index, and its §1.1 pair → rc 0, 6/6 mutations caught. R1, embedding leg: `pkg/search/semantic.go:381–:385` — `SELECT p.pid ... WHERE p.redacted = 0` inside `IndexVectors`, i.e. also at build time, counter-checked at `cmd/index-embed/main.go:99`; the read-time filter at `semantic.go:136–:137` is additive belt-and-braces, not the primary, which is exactly what this task demands. R5 IS NOT IMPLEMENTED. `generation.go:57–:62` states the requirement in a comment — “*Callers that redact must therefore treat the window ... as degraded*” — and the state vocabulary two lines away at `:46–:50` is `building / verified / live / superseded`, with no degraded member. Live GET `/api/index/status` returns 6 keys and none is degraded. Do not mistake `pkg/search/degraded.go:35–:37` for this: those `Degraded*` constants are upstream-Lumen SEARCH markers (T056's reindexing string), a different concept at a different layer. So between a recorded redaction and the completed rebuild, the index serves a generation it knows contains the passage and says nothing — which is the same unclosed window T038's missing “*mark the live generation as requiring a rebuild*” clause leaves open, seen from the other end. These two are one gap and should be closed together] Enforce redaction rules R1 and R5 in `workshop/platform/backend/pkg/index/generation.go` per [contracts/passage-contract.md](#) §7.3 (FR-039): redacted pids are excluded from `passages_fts` AND from the embedding set at **generation-build time**, so neither leg can return a redacted passage; and until the post-redaction rebuild completes the index reports degraded rather than serving a generation known to contain the passage. Filtering at render time is not compliance — the passage would still be retrievable — **BLOCKER**: none but the work — R1 is done and proven on both legs; R5 is a comment · **OWNER**: **implementer** — unblocked today (FR-039, FR-020)



T060 [US3] [TDD] Implement cross-reference derivation and storage in `workshop/platform/backend/pkg/crossref/`, cycle-safe per the spec edge case (FR-018)



T061 [US3] [TDD] Enforce redaction rule **R3** in `workshop/platform/backend/pkg/crossref/` and in `GET /api/passages/{pid}/crossrefs` (`contracts/passage-contract.md` §7.3, `contracts/http-api.md` §3.9): a cross-reference whose endpoint is redacted is suppressed from traversal and counted in `redacted_omitted` — the count only, never the pids and never the content. Reporting the count is honest that something was suppressed; returning a silently shorter list is not (FR-039, FR-018)



T062 [US3] [P] [SUBAGENT] Build the search UI with type-ahead in `workshop/platform/frontend/src/app/features/search/` — it **MUST** render the three states distinguishably; the reference anti-pattern is `ai_interviewing/.../search.component.ts` (FR-015, FR-019, FR-020)



T063 [US3] [P] [TDD] WCAG 2.1 AA + full keyboard operability for search (FR-041, FR-042, SC-017); `@axe-core/playwright` is already a dependency of `_tests/` with `_tests/evidence/ally-audit/run-audit.js` as precedent — **[TICKED 2026-09-02 after independent re-measurement. The suite landed at `workshop/platform/frontend/e2e/ally-responsive.spec.ts`, which cites T063/FR-041/FR-042/SC-017 at :5, runs axe over the route set in BOTH themes (:71 light, :79 dark), and carries a keyboard operability block at :89 covering the skip link, chapter-card traversal, the search combobox without a mouse, visible focus rings, and SC-021 deep-link traversal. Its own header at :8–:11 refuses to read “axe found nothing” as “the page is accessible”, which is the honesty the criterion needs. Honest boundary (§11.4.6): no Playwright run was executed for this tick — the assertions were read, not run, and stale failure artifacts exist under `workshop/platform/frontend/e2e/artifacts/test-results/` whose current status was NOT established]** (population: source) (FR-041, FR-042, SC-017)



T064 [US3] **[PATH NOT BUILT — `retrieval.tsv` does not exist; that directory holds `questions.tsv` alone. Do not mistake `workshop/pipeline/benchmark/retrieval_benchmark.json` + `run_retrieval_benchmark.py` for it: that runner’s own docstring disclaims this scope, stating it measures spec 002’s SC-015 and explicitly NOT SC-007/SC-008. No zero-literal-overlap subset column exists**

anywhere] Build the ≥ 20 -query retrieval benchmark in `workshop/platform/backend/testdata/benchmark/retrieval.tsv` (TSV, subset<TAB>query<TAB>expected_pid under a comment header, matching the shipped `questions.tsv` schema — see the benchmark-location correction in File Structure), including ≥ 8 queries sharing no literal words with their target, flagged in the subset column so the zero-overlap set is machine-selectable rather than judged by eye (SC-007, SC-008) — **[TICKED 2026-09-04. The [PATH NOT BUILT] note is WITHDRAWN BY NAME.**** `workshop/platform/backend/testdata/benchmark/retrieval.tsv` holds 27 data rows in the contracted subset<TAB>query<TAB>expected_pid schema under a comment header matching the shipped `questions.tsv` — split lexical 16 / zero_overlap 11, clearing both floors (≥ 20 total, ≥ 8 zero-overlap). Every claim below I re-derived myself rather than accepting on report. (1) All 27 expected_pid values exist in `curriculum/passages.jsonl` — 27 of 27, no invented pid; one candidate was dropped before it entered the file because it was redacted: true, which R1 removes from the index at build time. (2) The zero-overlap subset is machine-selectable and mechanically RE-DERIVED, not judged by eye — `bash workshop/scripts/bench-retrieval.sh --verify-overlap-only -> rc 0`, “27 row(s), 11 zero-overlap flag(s) RE-DERIVED and all true”, contacting no service so no retrieval figure is implied. A false-positive of my own is recorded here because the correction is the lesson: an independent check I ran first reported only 2 of 11 clean, because my ad-hoc tokeniser had NO stop-word removal — the 9 apparent collisions were the, and, is, it, not. The harness subtracts a closed, listed STOPWORDS set (`bench-retrieval.sh:212`) after ASCII-folding, splitting on non-alphanumerics, dropping 1-character tokens and stemming, and it PRINTS the raw pre-subtraction overlap per row so a reader can see exactly what was discounted. Under that normalisation all 11 rows share 0 content tokens. Scoring “literal word overlap” without declaring a stop-word policy is not a measurement, and this file declares one] (population: source, in_process) (SC-007, SC-008)



T065 [US3] [TDD] [PATH CORRECTED + TICKED 2026-09-02 — this task named two scripts, `workshop/scripts/bench-suggest.sh` and `workshop/scripts/bench-search.sh`; NEITHER exists. Both endpoints landed in ONE Go binary, `workshop/platform/backend/cmd/bench/main.go`, selected by `-endpoint suggest|search (:49)` with `-keystrokes 500 (:50)`, `-queries`, `-repeat`, and `p50_ms/p95_ms/p99_ms` emitted at :107 measured at the HTTP boundary, plus `loadavg_before` and `budget_p95_ms (:110)`. The figures are recorded in w

orkshop/docs/limits.md:216–:218: /api/suggest p95 14.2 ms, /api/search p95 2 094.8 ms, /api/health p95 2.2 ms, n=40 each. Read for the record, not a defect in this task: 2 094.8 ms breaches SC-006's 2 s budget, and limits.md:222 records a 20× same-day spread — the measurement is done, the criterion is not met, and those are different statements] Measure and record p95 latencies for suggest and search (SC-005, SC-006) with the harness quickstart.md US3 steps 1–2 contract: cmd/bench -endpoint suggest -keystrokes 500 and cmd/bench -endpoint search -queries 20 -repeat 5, both reporting p50, **p95** and p99 measured **at the HTTP boundary**, not inside the FTS5 call — the measured FTS5 p95 of 9.58 ms leaves roughly 190 ms of SC-005's budget for HTTP, serialisation and paint, and it is that remainder where the budget is actually spent or lost. bench-search.sh's report MUST also record host load average and whether an index build was in flight: three identical two-word embed calls, model resident, minutes apart, measured 20.16 s / 11.05 s / 0.10 s at load 8.25, a 200× spread driven purely by queue contention, so a p95 without its load conditions is not interpretable (population: served) (SC-005, SC-006)



T066 [US3] [TDD] Prove SC-016 and gate **G-PID-3** (contracts/passage-contract.md §5.3) behaviourally — *“an assertion that greps a file for a string is not a test”*: correct a transcript passage, re-index, and assert content_hash changed, the pid did **not**, and **every** stored citation and cross-reference to that passage still resolves to the same row. **Paired mutation**: replace the citation lookup key with content_hash; the gate MUST go red. content_hash is change detection and never identity, in every serialisation (SC-016, FR-037, FR-032, SC-012)



T067 [US3] [**PATH NOT BUILT** — workshop/docs/search.md does not exist, and the media boundary is not stated at any other path either; the docs directory holds faq.md, knowledge-model-contract.md, limits.md, manual.md, quickstart.md, README.md, user-guide.md, work-register.md plus the prompts/, research/, training/ and session-evidence/ trees] Document the media boundary in workshop/docs/search.md — audio and video are reachable THROUGH transcripts, not indexed directly; Lumen's extension allowlist is a compile-time var with no override (FR-031 requires stating what the system cannot do) — **BLOCKER**: none but the work — docs/search.md is unwritten. Two clauses of the task text are FALSE and must be corrected rather than transcribed; see the note · **OWNER: implementer** — unblocked today (FR-031, FR-016)



T115 [US3] [TDD] **[PATH NOT BUILT — workshop/scripts/index.sh does not exist and no equivalent landed anywhere]** Implement the workshop/scripts/index.sh wrapper per contracts/pipeline-cli.md §4.4 over T058's generations — --chapter, --lexical-only (the honest escape hatch when the embedding backend is saturated), --semantic-only, --verify-only, --swap/--no-swap, and --timeout-ms 5000 per call, because an unbounded query is how a 10-minute stall happens. It MUST reserve embedding capacity for interactive queries, which is a hard requirement rather than a nicety: scripts/ollama-tune.sh records that with OLLAMA_NUM_PARALLEL resolving to 1 a single embed went from 0.74 s to a >90 s client timeout, stalling indexing entirely — *"queue depth is not the defect — serialisation is"* — and without reservation SC-006 fails on every chapter ingest, precisely when people are using the system. Gate **G-CLI-6** (§4.4, §5): point the embedding endpoint at a closed port and assert exit 2 with reason.code: "embedding_backend_exhausted" **and** that the previously live generation is still live and serving. **Paired mutation:** map backend failure to exit 1; the gate MUST go red. The line that must not be blurred: all embedding servers exhausted is 2, never 1 — reporting 1 there makes a broken backend accuse a healthy curriculum — **BLOCKER:** none but the work — index.sh is unwritten · **OWNER: implementer** — unblocked today (SC-006, FR-020, FR-033, SC-013, FR-032, SC-012)



T116 [US3] **[PATH NOT BUILT — workshop/scripts/crossref.sh does not exist. The derivation it wraps DOES exist at workshop/platform/backend/pkg/crossref/ (T060, ticked); the CLI over it does not, and no --rebuild-derived/--check-cycles surface exists anywhere]** Implement the workshop/scripts/crossref.sh wrapper per contracts/pipeline-cli.md §4.5 over T060's derivation — --chapter, --min-score, --max-per-passage 20, --rebuild-derived, --check-cycles. It rebuilds origin: "derived" edges for the target generation and **never touches authored edges**, which are content rather than derivation (contracts/passage-contract.md §7.4 X4); self-references are rejected and traversal is cycle-checked. Exit 1 on a cycle among authored edges (a content defect a human introduced), an edge whose endpoint is not in the registry, or a self-reference in authored content; exit 2 when the embedding backend is unavailable for similarity scoring, the registry is unreadable, or the run was interrupted — an unscorable backend is not a finding about the content — **BLOCKER:** none but the work — the derivation it wraps EXISTS (T060); the CLI over it does not · **OWNER: implementer** — unblocked today (FR-018, FR-033, SC-013)



T117 [US3] [TDD] Prove gate **G-HTTP-6** ([contracts/http-api.md](#) §5): over 100 randomised search outcomes spanning all three states, assert the X-Workshop-Search-Status response header equals body . status on every single one — a header and a body that can disagree hand a cache or a proxy a different answer than the reader gets. **Paired mutation:** hardcode the header to ok; the gate MUST go red (FR-020, FR-032, SC-012)



T118 [US3] [TDD] **[PATH NOT BUILT — workshop/scripts/bench-retrieval.sh does not exist, and neither does its input: T064's retrieval.tsv is absent, so there is no query set to measure SC-007/SC-008 against]** Build workshop/scripts/bench-retrieval.sh per [quickstart.md](#) US3 step 3 and measure **SC-007** and **SC-008** with it against T064's query set: per-query top-5 hit/miss against the expected pid, $\geq 90\%$ of the ≥ 20 queries returning the expected passage in the top five, and $\geq 80\%$ of the **zero-literal-overlap subset** succeeding. That subset MUST be identified in the benchmark file itself and verified by a mechanical check that query and target share no token after normalisation — “obviously different wording” judged by eye is not a measurement. **Paired mutation:** run the benchmark against an **empty index generation**; both figures MUST collapse to 0% and the run MUST exit non-zero, because a benchmark that still scores well without an index is scoring the fixture rather than the system. ~~One divergence is recorded here rather than resolved: quickstart.md names workshop/platform/qa/retrieval-benchmark.jsonl as this harness's input while T064 builds workshop/pipeline/benchmark/queries.yaml, and quickstart.md is owned by another agent this session.~~ **RESOLVED 2026-09-01:** neither path existed on disk, a third location shipped with a working gate, and the canonical input is now workshop/platform/backend/testdata/benchmark/retrieval.tsv in **both** documents — see the benchmark-location correction in File Structure — **[TICKED 2026-09-04. The [PATH NOT BUILT] note is WITHDRAWN BY NAME — and read the criteria line carefully, because the harness is DONE and the criteria are NOT MET, which are different statements (same precedent as T065).**** workshop/scripts/bench-retrieval.sh exists, is executable and parses. It ran to completion against generation 68 / pid_count 12979 / state: live: SC-007 = 51.9% (14/27) against a $\geq 90\%$ bar — NOT MET. SC-008 = 0.0% (0/11) against a $\geq 80\%$ bar — NOT MET. rc 1, with per-query outcomes printed and written to workshop/platform/backend/evidence/retrieval/bench-2026-09-04.tsv (28 rows). The figure is trustworthy, and the reason is the three-valued contract doing its job: two

earlier attempts returned rc 2 — one died on Remote end closed connection without response, one on a /api/health timeout — and the gate reported COULD NOT DETERMINE rather than booking a 0%. The completed run had all 27 queries return status: ok or no_match, none unavailable, and a direct re-probe showed legs = {lexical: ok, lumen: skipped, semantic: ok} — so 0.0% is a measurement of a healthy two-leg system, not of a degraded one. PAIRED MUTATION VERIFIED BY ME: --prove-failure -> rc 0, PROVE: 11 mutations, 11 caught, 0 missed. It is not a stub — it starts a second workshop-server (the same binary the stack runs) over an EMPTY registry on its own port, asserts the resulting generation 1 / 0 passages / live shape, and only then credits the collapse of both figures to 0.0%; the production container is never touched. Other mutations caught: a falsely-flagged zero_overlap row, an invented pid, a sub-20-row benchmark, an unknown subset token, and — the important ones — absent benchmark / unreadable corpus / unreachable service each -> 2, never a 0% score. A REAL DEFECT WAS FOUND AND FIXED DURING THIS VERIFICATION: the battery's cleanup() EXIT trap referenced a local from an already-returned function, so under set -u it died work: unbound variable and rm -rf NEVER RAN — the temp tree leaked on every battery run. Fixed by hoisting the path to a global; re-run gives 11/11 caught, rc 0, no message, and 0 leaked directories. THE FINDING TO CARRY FORWARD, and it is a system finding rather than a task defect: SC-008 = 0/11 means meaning-based retrieval of a SPECIFIC passage is not working on this deployment. Every zero-overlap query returned a full set of plausible neighbours tightly clustered around ~0.68 with the expected passage outside the top 20. Queries were written against targets chosen from the corpus FIRST and measured once — not tuned against results. Scope is narrowed and the file says so: targets are docs/ sections and chapter-01/exercise-01.md sections; transcript segments are deliberately not targets] (population: served, in_process) (SC-007, SC-008, FR-014, FR-033, FR-032, SC-012)

Checkpoint: search works and degrades honestly. Stop for approval.

Phase 6: User Story 4 — Ask a question, get a grounded answer (Priority: P4)

Goal: cited answers that refuse rather than fabricate.

Independent test: questions with known answers return correct cited answers; genuinely unanswerable questions are all declined.

~~**BLOCKED ON AN OPERATOR ACTION:** no generative model exists on this host — ollama serves two models and both are the same embedding model; the whole store is 309 MB, too small for a decoder.~~

NOT BLOCKED — CLAIM WITHDRAWN 2026-09-02, and withdrawn on measurement, not on optimism

The struck-through paragraph is left visible so a reader can see that this phase was unblocked by an operator action that actually happened, not by an edit. **A generative model exists on this host and is WIRED IN.** Re-derive:

```
ollama list
#   qwen2.5:3b-instruct-q4_K_M    1.9 GB    (generative)
#   nomic-embed-text:latest       274 MB    (embedding)
#   jina-embeddings-code-cpu / ordis/jina-embeddings-v2-
      base-code    323 MB each (embedding)
podman inspect workshop-curriculum_platform_1 --format
'{{.Config.Cmd}}'
#   ... -answer-provider ollama -answer-model qwen2.5:3b-
      instruct-q4_K_M ...
```

Every clause of the old paragraph is false as measured 2026-09-02: ollama serves **four** models, not two; they are **not** all the same embedding model; and the running container's own argv names qwen2.5:3b-instruct-q4_K_M as -answer-model, so the answering path is wired to it today rather than merely capable of being wired to it. The 57-question benchmark under T078/T079 was driven through that model. Separately, an **entailment** judge also loads here and produced a decided verdict — workshop/docs/limits.md:1642 records entail=0.9924 — over pkg/entail's ONNX and ollama judges.

What is NOT claimed: that a 3B q4 instruct model is FIT for this work. That is a separate question this block does not answer, and T078's measured SC-010 NOT met is the evidence that it is a live question. **The parenthetical “(3 fabrications)” this line carried is WITHDRAWN, not restated** — re-measured 2026-09-02 the

count is 1, and T078 records why that is not the improvement the two numbers suggest. T069's extractive adapter remains the zero-generative route and is unaffected.



T068 [US4] [TDD] [REVIEW] Implement the Provider interface with adapters none (default), extractive, ollama, openai_compatible in workshop/platform/backend/pkg/answer/provider.go — copy the interface SHAPE from LLMPProvider, not the dependency (~~its module path is unresolvable and it carries a relative path replace on a sibling checkout absent from .gitmodules~~). **[CLAIM WITHDRAWN 2026-09-02 — the parenthesis is struck rather than deleted so the reason it died stays visible. LLMPProvider is no longer unresolvable: submodules/LLMPProvider and submodules/RAG are declared gitlinks of this repository, and workshop/platform/backend/go.mod now carries require digital.vasic.llmprovider v0.0.0 and require digital.vasic.rag v0.0.0 with replace targets ../../../../submodules/LLMPProvider and ../../../../submodules/RAG. The backend CONSUMES it — per that go.mod's own comment block, for the HTTP transport — so “copy the shape, not the dependency” is now a design choice about the Provider seam, not a workaround for a broken module path. The tick is unaffected: the seam was built and it still stands]** (population: source) (FR-023, FR-021)



T069 [US4] [TDD] Implement the extractive adapter — ~0.3 s, genuinely grounded, structurally unable to fabricate, and works today with zero generative capability (population: unstated) (FR-021, FR-023)



T070 [US4] [TDD] Implement L1: the calibrated retrieval gate with BOTH min_score and min_margin — the margin test is what catches the near-miss that scores high while being unanswerable (FR-022, SC-010)



T071 [US4] [TDD] Implement L2: JSON-schema-constrained generation where "minItems": 1 on citations makes an uncited claim structurally undecodable (FR-021, SC-009)



T072 [US4] [TDD] Implement L3: deterministic citation pid set-membership against the LIVE generation — **SC-009 is unreachable without this**; attaching a citation is easy, proving it points at a real passage is a microsecond set check (SC-009, FR-021)



T073 [US4] [TDD] [PARTIAL — L4 EXISTS AND IS WIRED, BUT BOTH HALVES OF THIS TASK'S OWN PARENTHESIS DIVERGE, AND IT CANNOT BE EXERCISED ON THIS HOST. Measured 2026-09-03; NEW note.** Built: pkg/answer/verify.go:363–:470 EntailmentSupport, wired into VerifyClaims at :158; go test ./... across all 25 backend packages is rc 0, pkg/entail and pkg/answer included. Divergence 1 — the floor is LEXICAL, not an embedding floor.

NewEntailmentSupport(lexical *LexicalSupport, judge entail.Judge) at :392–:403. The file states this against itself at :226–:227 and :237, which is honest, but this task says “*embedding floor*” and no embedding is consulted. Divergence 2 — it is not BATCHED. pkg/entail/entail.go:57–:69 takes one (premise, hypothesis) per call and grep -n 'Batch' over non-test pkg/entail/*.go returns 0; verify.go loops claim × citation. On CPU-only generation (~21 s to ~95 s per call, per T083's own figures) that is the difference between a usable layer and an unusable one. Divergence 3 — it CANNOT RUN HERE, and the tree says so rather than pretending. verify-entailment-loads.sh → rc 2 UNDETERMINED (“*U4 was not asked*”) because no model.onnx exists anywhere under workshop/ (find = 0 hits), even though pipeline/venv carries onnxruntime 1.29.0. Its pair prove-entailment-loads.sh → rc 0 (5 caught, 2 controls at rc 2), and gates/prove-mutations.sh → rc 2 at PROVED 25 / COULD-NOT-RUN 3 of 28, M26 and M28 skipped for that same missing model. That rc 2 is the correct answer and is NOT a pass — it is also why the whole answering chain above (T078, T079) is being measured with L4 effectively absent. Obtaining or building the ONNX judge model is plausibly an operator decision; the lexical-vs-embedding floor and the batching are unwritten work] Implement L4: support verification (embedding floor, then batched entailment) — **BLOCKER: host capability** — L4 exists and is wired, but it cannot be exercised on this host · **OWNER: operator** — needs a host that can run it, or an explicit decision to record it unmeasurable (SC-009, FR-021, FR-022)



T074 [US4] [TDD] Any layer failing = refuse the WHOLE answer; never strip claims silently (FR-022, SC-010)



T075 [US4] [TDD] [PARTIAL — THE A4 HALF IS DONE AND TESTED; THE STORED-ANSWER HALF IS UNBUILT AND THERE IS A LIVE CACHE IT DOES NOT COVER. Measured 2026-09-03; NEW note.** DONE: pkg/answer/verify.go:126–:134 maps passage. OutcomeRedacted to Decline: ReasonRedactedEvidence and withdraws the WHOLE answer — claims are never stripped to keep it presentable (§7.2 C4); the constant is pkg/answer/outcome.go:133, enumerated at :160, and it is covered by pkg/answer/pipeline_test.go:270 and citations_verified_test.go:173, both passing under the rc-0 go test ./.... NOT DONE, and the reasoning that excuses it is measurably out of date. internal/redaction/plan.go:596–:606 declares propagation target stored_answers as StatusAbsent, count 0, on the ground that “*no answer cache is persisted*” — but pkg/answer/jobs.go:25–:45 holds a 30-minute in-memory job cache carrying outcome *Outcome (:45, :164, :200), and grep 'redact\|withdraw' jobs.go returns 0 hits. So an answer computed before a redaction and polled after it is served unchecked for up to thirty minutes. Not persisted is not the same as not served, and this task’s own words are “*Redaction propagates to stored answers, not merely to the rendered transcript*”. Do not mistake the withdrawn vocabulary that already exists for this one: internal/redaction/taxonomy.go:83–:136 and internal/api/api.go:73 ErrTermWithdrawn withdraw TAXONOMY TERMS (feature 002), not answers] Enforce redaction rule R4 and contracts/http-api.md §3.10 A4 (FR-039): a citation resolving redacted invalidates the WHOLE answer as declined{redacted_evidence} (contracts/passage-contract.md §7.2 C4 — claims are never stripped to keep an answer presentable), and any **stored** answer whose citations intersect the redaction set is marked withdrawn and MUST NOT be served, with re-asking required. Redaction propagates to stored answers, not merely to the rendered transcript — **BLOCKER:** none but the work — the A4 half is done and tested; the stored-answer half is unbuilt and there is a live cache it does not cover · **OWNER: implementer** — unblocked today (FR-039, FR-022)



T076 [US4] [TDD] [PARTIAL, AND THE MISREADING THIS NOTE PREVENTS MATTERS MORE THAN THE CLASSIFICATION. Measured 2026-09-03; NEW note.** tasks.md’s own T040 note calls workshop/platform/gates/verify-redaction-propagation.sh “G-PID-5”. The identifier G-PID-5 occurs NOWHERE in workshop/ ** — not in a *.sh, *.go, *.tsv or *.py — only in contracts/

passage-contract.md:557, analysis.md:59 and this file. That gate is real and green (rc 0, A1–A10b; pair rc 0, M1–M6) but it makes one of this task’s four assertions. Measured by grep

'redacted_omitted\|410\|stored_answer\|crossref' over that gate: 0 hits for all four strings. It asserts R1 (A10b, search cannot return it) and eight-target propagation (A5). It does not assert resolve → redacted with the API answering 410 Gone; it does not assert crossrefs omitted with a non-zero redacted_omitted (R3); it does not assert a stored answer withdrawn (R4 — which T075 shows is unbuilt anyway). Its six mutations are M1 derived-term-index, M2 lookup key, M3 refusal-as-success, M4 unreadable registry, M5 report-without-artifact, M6 one-of-eight-targets: none is this task’s “*propagate only to the rendered transcript*”, and none demonstrates R1, R3 and R4 individually rather than as one aggregate, which is the clause this task adds in bold. Pieces exist elsewhere and are worth reusing: 410 Gone is implemented at internal/api/

crossrefs.go:138, passages.go:30, graph_traverse.go:165, knowledge_reverse.go:117, and redacted_omitted is asserted by Go tests at internal/api/crossrefs_test.go:193/:241/:395. Do not mistake verify-r3-rules.sh (rc 0) for R3 — that is spec-002’s extraction-reconciliation R3, a different rule with the same name.

BLOCKED-ON-PREDECESSOR on T075 for the R4 leg; the R3 and 410 legs are writable today] Prove G-PID-5 end to end (contracts/passage-contract.md §7.3) — this is the earliest point at which all three propagation paths exist. Redact a cited passage, then assert: search cannot return it (R1, T059), resolve returns redacted and the API answers 410 Gone (contracts/http-api.md §3.8), cross-references to it are omitted with a non-zero redacted_omitted (R3, T061), and the stored answer citing it is withdrawn (R4, T075). **Paired mutation:** propagate the redaction only to the rendered transcript; the gate **MUST** go red, and **MUST** demonstrate R1, R3 and R4 individually rather than as a single aggregate failure — **BLOCKER: predecessor** — see the note; the identifier confusion it prevents matters more than the classification · **OWNER: implementer**, after its named predecessor (FR-039, FR-032, SC-012)



T077 [US4] Build the ≥ 10 -question adversarial unanswerable set as the U rows of workshop/platform/backend/testdata/benchmark/questions.tsv (which exists and already holds 10 U rows and 8 A rows — the U half meets SC-010’s count, the A half is **8 short of SC-009’s ≥ 20** and that shortfall is this task’s remaining work) [FIGURE WITHDRAWN 2026-09-02 — stale in BOTH columns, and the

shortfall it described is closed. Re-measured with `awk 'NR>1 && !/^#/ && NF' ... | cut -f1 | sort | uniq -c: 24 A rows and 33 U rows. SC-010's ≥ 10 unanswerable and SC-009's ≥ 20 answerable are both met ON COUNT. What is NOT met is SC-010 itself — the run at bench-expanded-2026-09-01.tsv:19 records SC-010 NOT met (3 fabrications) that figure is WITHDRAWN, not restated: re-measured 2026-09-02 at workshop/platform/backend/evidence/answering/bench-question-verifier-ab-2026-09-02.tsv:28, SC-010 NOT met (1 fabrication) on a corpus the old run did not cover; see T078 for why 3 $\rightarrow$ 1 is not the improvement it looks like — and that is T078's work, not this task's; and SC-009's certification, which is T079's] using the taxonomy (near-miss attribute, false premise, uncomputable aggregate, misattributed speaker, lexically-overlapping-but-unanswerable, redacted passage, inaudible segment) — ten astrophysics questions would pass any threshold and prove nothing (population: source, served) (SC-010, SC-009)`



T078 [US4] [TDD] [PATH CORRECTED 2026-09-02 — this task named `workshop/scripts/bench-answers.sh`, which does not exist; the harness landed at `workshop/platform/backend/gates/bench-answers.sh` and has run. It is three-valued and class-aware, checks SC-009's ≥ 20 floor before the first model call, and prints top score and margin on every response including successes. Its configuration is by `ENVIRONMENT (QUESTIONS, REPORT, MIN_ANSWERABLE, TIMEOUT)` plus one positional base URL — not** the `--answerable/--unanswerable/--report` flags this task contracts, which is a real divergence and is the flag surface still owed. **Do not rebuild the harness.** What is open is SC-010's verdict, which is measured and still NEGATIVE — **but the “3 fabrications” figure this note carried is WITHDRAWN, not restated.** ~~bench-expanded-2026-09-01.tsv:18-:19 records 30 refused, 3 FABRICATED, SC-010 NOT met (3 fabrications)~~. Superseded twice on 2026-09-02 by the A/B run recorded at `workshop/platform/backend/evidence/answering/bench-question-verifier-ab-2026-09-02.tsv`, and the sequence matters more than either number. With the T115 answer-against-question layer **unwired**, that file's `:24-:25` put the unanswerable set at 22 refused and 11 fabricated; with it **wired**, `:27-:28` put the same set at 29 refused, 1 fabricated and 3 unavailable. Both runs record SC-010 as NOT met. **The old 3 was therefore not an improvement on 11 — it was taken over a smaller corpus,** which that file explains at `:31-:33`. Honest denominator, stated at `:42: 1 fabrication in 30 MEASURED`, with the 3 unavailable excluded

rather than scored as refusals. The surviving case is isolated at :53 and identified in the per-question rows. **SC-010 is NOT met**, and — see standing rule 7 — this is **spec 001's** SC-010, not spec 002's. Fix the surviving fabrication, then re-run. — **BLOCKER**: none but the work — the harness landed at a corrected path and HAS run · **OWNER: implementer** — unblocked today (SC-010, FR-022, FR-033)

RE-RUN 2026-09-03 AGAINST THE LIVE SERVICE — SC-010 IS STILL NOT MET, AND THE NUMBER IS CONFIRMED RATHER THAN INHERITED. TIMEOUT=120 bash workshop/platform/backend/gates/bench-answers.sh http://127.0.0.1:8087 against the running server and ollama (both HTTP 200), ~46 minutes, **exit 1**, on today's generation 67 (pid_count 2478, root_hash sha256:03d91626...) — a corpus neither earlier run covered:

```
COMPOSITION  answerable 24  unanswerable 33  SC-009 (>= 20
answerable) met
ANSWERABLE   answered+cited 3  declined 21  unavailable 0
UNANSWERABLE refused 32  FABRICATED 1  unavailable 0
FINDING: 1 unanswerable question(s) were answered
```

unavailable 0 on both rows is what makes this a clean measurement: nothing timed out and nothing was excluded, so the denominator is the whole set — **1 fabrication in 33**, not 1 in 30 with three set aside. **SC-010 (spec 001's) is NOT met**. DONE and worth keeping: retrieval top score **and** margin print on every row including successes (bench-answers.sh:74–:86, :106–:107), so a fragile 0.002-margin pass is visible as fragile. **STILL MISSING — the flag surface, unchanged since the 2026-09-02 note**: there is no getopts and no -- case arm in the script; BASE="{1:-...}" at :36 is the sole positional and everything else is env (QUESTIONS, REPORT, MIN_ANSWERABLE, TIMEOUT at :38–:41). --answerable / --unanswerable / --report do not exist]** Build the bench-answers harness at workshop/platform/backend/gates/bench-answers.sh per quickstart.md US4 config A — --answerable, --unanswerable, --report — and prove SC-010 with it: 10/10 declined, each with a reason of below_threshold, margin_too_small or unsupported, and 0 fabricated. Every response carries retrieval (top score **and** margin) **even on success**, so a 0.002-margin pass is visible as FRAGILE rather than indistinguishable from a confident one. This is the same harness T079 runs for SC-009



T079 [US4] [TDD] **[PATH CORRECTED 2026-09-02 — T078's harness is workshop/platform/backend/gates/bench-answers.sh, not workshop/scripts/bench-answers.sh. The ≥20-answer RUN exists: question**

s.tsv holds 24 A rows and the evidence header records SC-009 met (24 answerable ≥ 20) with answered+cited 17 declined 7. The CERTIFICATION is what this task still owes** — the same evidence file states its own honest boundary that answered+cited is not answered correctly, and records that only 6 of the 17 were re-asked and read, with 1 defect found. SC-009 requires 100% over ≥ 20 answers, so it is unmet on certification, not on count. — **BLOCKER: none but the work — the ≥ 20 -answer run exists · OWNER: implementer — unblocked today (SC-009, FR-021)**

RE-MEASURED 2026-09-03, AND THE FRAMING ABOVE IS NOW TOO OPTIMISTIC — IT IS UNMET ON BOTH. Two different ≥ 20 floors were being conflated and this note separates them. The corpus floor holds and is enforced before the first model call: questions.tsv carries 24 A rows and bench-answers.sh:50–:55/:135 prints SC-009 (≥ 20 answerable) met. **The ANSWER floor does not.** Today's live run over that same 24 produced **answered+cited 3 declined 21 unavailable 0** — see T078 for the full run — so there is no population of ≥ 20 answers whose citations could be certified at all. The recorded 2026-09-01 figure of answered+cited 17 was also below 20; **the drop from 17 to 3 is a MEASURED change on a different generation (67) and its CAUSE WAS NOT DIAGNOSED — recorded as an observation, not explained, and unavailable 0 rules out timeouts.** Note L4 is effectively absent on this host (T073: no model.onnx, verify-entailment-loads.sh rc 2), which is a candidate but was not tested. **No per-citation certification artifact exists.** verify-sc009-citation-span.sh $\rightarrow$ rc 0 and its pair $\rightarrow$ rc 0 with 9 mutations caught, but **that gate's own first output line reads NOTE: ... This gate is spec 002's SC-009 —** it certifies that a media citation lands inside its cited time span, not that a citation supports a claim. Standing rule 7, exactly as written. **So this task is blocked twice: on the answer rate (unwritten engineering) and on human per-citation reading of ≥ 20 answers (operator). It is NOT a pure operator item and must not be filed as one]** Prove SC-009 over ≥ 20 answers with human certification per citation, using T078's harness — each answerable question returns status: answered, text and **≥ 1 citation**, zero citations while answered being structurally undecodable because the response schema sets "minItems": 1**



T080 [US4] [TDD] [REVIEW] **[PARTIAL — THE ALLOWLIST IS DONE AND MUTATION-PROVED; THE PART THAT UPGRADES “WE OBSERVED NO EGRESS” TO “EGRESS WAS IMPOSSIBLE” DOES NOT EXIST. Measured 2026-09-03; NEW note.** DONE, and**

done the right way: pkg/answer/locality.go:62 uses net.LookupIP and :83 !ip.IsLoopback() — resolution, not string matching — with the asymmetric contract stated at :39–:48 and covered by pkg/answer/provider_test.go:80–:95. Its §1.1 paired mutation genuinely RUNS and is genuinely CAUGHT: gates/prove-mutations.sh row M10-locality-by-substring GREEN RED OK, whose mutation body at prove-mutations.sh:220–:221 replaces resolution with strings.Contains(endpoint, "127.0.0.1") — i.e. it seeds the exact defect this task names — plus M16-standin-invents-locality-fault OK. MISSING — all four remaining elements, and the measurement is a single zero. grep -rn 'unshare|ip netns|tcpdump|tshark' over the whole of workshop/ (excluding pipeline/venv) returns 0 hits: no egress-denied namespace, no packet capture. There is no curl https://example.com negative control — the only example.com strings in the tree are test fixtures at provider_test.go:42/:76/:85/:107 and doc comments. G-CLI-14 occurs 0 times in the entire tree, so neither the gate nor its outside-the-namespace paired mutation exists. What is enforced today is a config flag plus a DNS check, which is precisely what this task's own last sentence refuses to accept as a guarantee. Building the namespace may need host privileges and is plausibly an operator step; the negative control and the capture assertion are not] Enforce FR-024 privacy: resolved-address allowlist (net.LookupIP + IsLoopback, not string matching), egress-denied namespace with a NEGATIVE CONTROL (curl https://example.com from inside MUST fail), and packet capture asserting zero non-loopback packets. A config flag is not a guarantee. This is gate **G-CLI-14** (contracts/pipeline-cli.md §5, D-LLM-4): the negative control and the capture both run across the full 20-answer and 10-refusal runs. **Paired mutation**: run the same assertions OUTSIDE the egress-denied namespace; the gate MUST go red — if it stays green the test proved nothing about the namespace, and “we observed no egress” was never upgraded to “egress was impossible” — **BLOCKER**: none but the work — the allowlist is done and mutation-proved; the part that upgrades “we observed no egress” to “egress was impossible” does not exist · **OWNER**: **implementer** — unblocked today (FR-024, FR-032, SC-012)



T081 [US4] [TDD] [PARTIAL — THE COMPILE-TIME SEPARATION IS REAL AND VERIFIABLE; NEITHER OF ITS TWO GATES EXISTS. Measured 2026-09-03; NEW note.** DONE, and it holds at the level this task demands — the import graph, not a convention. pkg/answer/http.go:13–:38 Mount registers only the

seven `/api/ask*` routes; `internal/api/router.go:41–:58` registers search, suggest, passages and crossrefs; `grep 'pkg/answer'` over non-test `internal/api/*.go` returns 2 hits, both doc comments (`questions.go:52, :294`) — no import, so browsing and search cannot be broken by the answering package failing to construct. `internal/answering/wire.go:97` and `:113` return no error by construction and `cmd/workshop-server/main.go:487–:493` states “*EVERY FAILURE HERE IS DEGRADATION, NEVER A REFUSAL TO START*”, mirrored at `pkg/answer/provider.go:219` and `cmd/workshop-ask/main.go:128`. unavailable and declined are distinct wire shapes (object vs bare string), asserted at `pkg/answer/http_test.go:60–:95` and `citations_verified_test.go:309`, all passing under the `rc-0 go test ./.... MISSING` — both gates, and the existing tests do not substitute for them. **G-HTTP-5** occurs once, a comment at `pkg/answer/http.go:24`; **G-CLI-13** occurs 5 times, every one a Go comment or test name (`wire.go:97`, `provider_test.go:64`, `provider.go:219`, `workshop-ask/main.go:128`, `workshop-server/main.go:493`). No gate script, no registry row, no paired mutation for either. Nothing anywhere stops ollama and asserts the pair — that `GET /api/search` returns 200 with real lexical results WHILE `POST /api/ask` returns 503 state:“unavailable”; the existing 503 tests inject a retrieval deadline (`http_test.go:76–:86`), which is a different cause, and assert nothing about `/api/search` under the same condition. Nothing exercises `scripts/start.sh` with ollama stopped. Do not read `verify-server-unity.sh` (`rc 0, PASS=35 FAIL=0 UNDET=0 DEBT=4`) as covering this — it probes route existence with ollama UP, which is the opposite of the condition both gates are about] Enforce FR-025 by SEPARATE ROUTE TREES at compile time — browsing and search must survive answering being unavailable, and unavailable must remain distinct from declined — they are different states with different causes, and `no_provider` is a cause of unavailable, never a decline reason ([contracts/http-api.md §3.10 A7](#)). Two gates prove FR-025 from its two sides. **G-HTTP-5** ([contracts/http-api.md §5](#)): with ollama stopped, `GET /api/search` returns **200** with real lexical results **and** `POST /api/ask` returns **503** state:“unavailable”; **paired mutation**: `wire /api/search` through the answering provider’s health check. **G-CLI-13** ([contracts/pipeline-cli.md §4.8, §5](#)): with ollama stopped, `start.sh` still exits 0 and prints which state the stack came up in — answering-provider construction failure **MUST NOT** abort startup; **paired mutation**: abort startup on provider construction failure. Both mutations **MUST** turn their gate red — **BL**

OCKER: none but the work — the compile-time separation is real and verifiable; neither of its two gates exists · **OWNER: implementer** — unblocked today (FR-025, FR-020, FR-032, SC-012)



T082 [US4] Implement the ingest exclusive lock that suspends answering while search continues from the existing generation (D-LLM-10) (FR-025)



T083 [US4] **[PATH NOT BUILT — workshop/docs/answering.md does not exist. Part of the content landed at OTHER paths and should be reused rather than rewritten: the estimated_seconds: null contract is documented at workshop/docs/faq.md:215–:216 and workshop/docs/quickstart.md:388–:389, and the asynchronous-answering timing story is in workshop/docs/limits.md §1.4. The PAGE this task contracts still does not exist]** Document in workshop/docs/answering.md that answering is ASYNCHRONOUS — CPU-only generation is ~21 s (1.5B) to ~95 s (7B) idle, so “instant” is off by two orders of magnitude and no prompt engineering closes it. estimated_seconds MUST be null until measured — **BLOCKER:** none but the work — docs/answering.md is unwritten; part of the content already landed at other paths and should be reused, not rewritten · **OWNER: implementer** — unblocked today (FR-031, FR-030)

Checkpoint: answers are cited or refused, never fabricated, and a redacted passage cannot reach a reader through search, cross-references or a stored answer. Stop for approval.

Phase 7: User Story 5 — Add a chapter (Priority: P5)

Goal: one documented, repeatable, idempotent procedure.

Independent test: run it against a synthetic chapter; it appears fully integrated with no code change. Run it twice; nothing duplicates.



T084 [US5] [TDD] **[PATH NOT BUILT — workshop/scripts/add-chapter.sh does not exist and none of the nine contracted stages exists as a driver. Four of the stages it must call are themselves [PATH NOT BUILT] (transcribe.sh T111, index.sh T115, crossref.sh T116, redact.sh T038/T039); two exist (verify-accuracy.sh T112, i**

ngest.sh T113). This task gates T085, T086, T090 and the enforcement half of T087. RE-MEASURED 2026-09-03: unchanged, and now the single largest blocker in this file. `grep -rln 'add-chapter\|add_chapter'` over `workshop/scripts/` and `workshop/platform/` returns ZERO files; G-CLI-7 and G-CLI-8 (T085, T086) appear nowhere in `workshop/`. The stage census is unchanged: of the nine, `verify-accuracy` (T112) is built and `ingest` (T113) is built but missing five of its six contracted flags, while `transcribe` (T111), `index` (T115) and `crossref` (T116) do not exist and `redaction-review` (T039) is now built, proven and ticked. This task alone accounts for FIVE unticked tasks that are blocked on nothing else — T085, T086, T087's enforcement half, T088 and T090 — so it is the highest-leverage unwritten item in the file. Nothing blocks it but the work and the four missing stages] Implement `workshop/scripts/add-chapter.sh` per [contracts/pipeline-cli.md](#) §4.7 covering **all NINE contracted stages in order** — `preflight` → `extract` → `transcribe` → `verify-accuracy` → `ingest` → `index` → `crossref` → `redaction-review` → `publish` — with `--resume`, `--only <stage>`, `--skip <stage>`, `--from <stage>`, `--dry-run`, `--json` and `--check-idempotent`, and with no code edit required to add a chapter. **No stage is optional and none is silently dropped:** `preflight` (§3.1) classifies every finding as tooling (2) or content (1) and reports ALL of them rather than the first; `verify-accuracy` (§4.2) must actually run because the presence of its `accuracy.json` is `publish` precondition B2 — its value may miss target, its absence blocks; and `redaction-review` is the FR-039 gate implemented in T038/T039, which is what makes `publish` legal at all. An earlier draft of this task named only four stages (`transcribe` → `ingest` → `index` → `cross-link`); dropping the publication gate is precisely how a redaction requirement becomes decorative — **BLOCKER: predecessor T111 + T115 + T116 + T088** — four of the nine stages this driver must call are themselves unwritten · **OWNER: implementer**, after those four (FR-026, FR-028, FR-029, FR-039)



T085 [US5] [TDD] Prove idempotency (FR-027) as gate **G-CLI-7** ([contracts/pipeline-cli.md](#) §4.7, §5): run `add-chapter.sh` twice on a small synthetic chapter and assert all of **J1-J6** — a second run changes nothing and duplicates no passage, keyed on pid. **Paired mutation:** make the procedure mint unconditionally; the gate MUST go red — **BLOCKER: predecessor T084** — G-CLI-7 runs `add-chapter.sh` twice; there is no `add-chapter.sh` · **OWNER: implementer**, after T084 (FR-027, FR-032, SC-012)



T086 [US5] [TDD] Incomplete materials **MUST** report precisely what is missing and **MUST NOT** publish a partial chapter as complete (FR-028), proven as gate **G-CLI-8** ([contracts/pipeline-cli.md](#) §4.7, §5): remove `chapter.yaml` and two archive parts, then assert exit 1 with **three** findings enumerated — all of them, not just the first — and nothing published. **Paired mutation**: report the first finding only and publish anyway; the gate **MUST** go red — **BLOCKER: predecessor or T084 · OWNER: implementer**, after T084 (FR-028, FR-032, SC-012)



T087 [US5] [TDD] Enforce the publish preconditions **B1–B6** of [contracts/pipeline-cli.md](#) §4.7 in `add-chapter.sh`: B1 `coverage.unexplained_gap_s == 0` · B2 `accuracy.json` exists · B3 every passage has a pid in `passages.jsonl` · B4 the generation containing the chapter is `live` · B5 **redaction-review.json exists and is newer than the transcript (FR-039)** · B6 no source file changed during the run. Failing any one of them ⇒ exit 1 with the chapter left transcribed, never published. **Paired mutation**: publish with `redaction-review.json` absent (and again with it stale); the gate **MUST** go red in both cases — B5 is the only thing standing between FR-039 and a decorative requirement — **BLOCKER: predecessor T084** — B1-B6 are enforced IN `add-chapter.sh` · **OWNER: implementer**, after T084 (FR-039, FR-028, FR-004, FR-032, SC-012)



T088 [US5] Make the procedure accept a PRE-SUPPLIED TRANSCRIPT FIXTURE so idempotency and identity can be proven without running ASR — otherwise every US5 proof inherits the ASR block. **[PATHS ADDED 2026-09-02 — this task named none, and two fixture producers already exist: workshop/pipeline/extract/fixtures/synthetic_chapter/passages.jsonl and workshop/platform/backend/cmd/fixture-corpus/main.go, which writes an invented corpus so endpoints can be exercised without private content. Do not build a third fixture. What is missing is a PROCEDURE that accepts one, and that procedure is T084, which is [PATH NOT BUILT]]** — **BLOCKER: none but the work** — a pre-supplied transcript fixture is what lets every US5 proof escape the ASR block, so this is the cheapest way to unblock T085-T087 · **OWNER: implementer** — unblocked today (FR-027, SC-016)



T089 [US5] **[PATH CORRECTED 2026-09-02 — this task named workshop/docs/add-chapter-prompt.md, which does not exist; the prompt landed at workshop/docs/prompts/add-a-chapter.md]** Write the reusable extension prompt in `workshop/docs/prompts/add-a-`

chapter.md — the operator-facing artifact requested (FR-026, FR-030)



T090 [US5] [TDD] Prove SC-011: a new chapter integrated in under 30 minutes hands-on with zero code or config change — **BLOCKER: predecessor T084 + T088** — SC-011 times the procedure end to end · **OWNER: implementer**, after T084/T088 (SC-011, FR-026)

Checkpoint: the curriculum is extensible without engineering, and no chapter reaches published without a fresh redaction review. Stop for approval.

Phase 8: Polish & Cross-Cutting Concerns



T091 [P] [SUBAGENT] Write workshop/docs/user-guide.md (FR-030)



T092 [P] [SUBAGENT] Write workshop/docs/manual.md (operator) (FR-030)



T093 [P] [SUBAGENT] Write workshop/docs/faq.md (FR-030)



T094 [P] [SUBAGENT] Write the training material under workshop/docs/training/ — **[PATH CORRECTED + ONE HALF NOT BUILT, measured 2026-09-02. workshop/docs/training.md does not exist; training landed as a DIRECTORY, workshop/docs/training/, holding 00-Overview-and-Taxonomy.md plus areas/, curriculum-areas/ and diagrams/. workshop/docs/tutorial-quickstart.md is [PATH NOT BUILT] — find workshop -iname '*tutorial*' returns zero files. Whether workshop/docs/quickstart.md (T051) was meant to absorb that second artifact is CANNOT DETERMINE from the tree. The tick is left as it was found; this note corrects only where the files are]** (population: source) (FR-030)



T095 [P] **[PARTIAL — A GREEN GATE GUARDS A DIFFERENT CRITERION. Measured 2026-09-03; NEW note, and this is the kind of finding a glance at an exit code would invert.** workshop/docs/limits.md EXISTS, 2,273 lines, 117 headings, and it has a paired gate that is genuinely green: verify-limits-**

completeness.sh → rc 0, “checked 15 registered defect(s) against docs/limits.md ... all 15 are named”, and prove-limits-completeness.sh → rc 0, one anchor removed, CAUGHT, restored. But that gate’s own line 2 scopes it to SC-026, not FR-031 — it asserts that every row of the DEFECT REGISTRY is named in the page, which is a completeness claim about a different set. Measured against FR-031’s own four named classes: `grep -c 'FR-031' docs/limits.md = 0`; SC-016a = 0; operator-only = 0; lines pairing media with indexing = 0; lines pairing code-passage identity with weak/weaker = 0. Only asynchronous answering is covered (3 lines: :195, :375, :1084). So three of the four classes this task names are undocumented, and no instrument would notice. Note also that the SC-016a class cannot be honestly written until T015–T017 exist to be weaker *than something* — but the media-boundary and operator-only classes are writable today, and the media boundary is T067’s page, which is also **unwritten**] Document every limit honestly per FR-031: no direct media indexing, asynchronous answering, code-passage identity weaker than transcript identity (SC-016a, implemented by T015–T017 and proven by G-PID-4 in T017), and every operator-only step — **BLOCKER**: none but the work — `limits.md` exists with a paired proof, but the green gate guards a DIFFERENT criterion than the one this task names · **OWNER: implementer** — unblocked today (FR-031, SC-016a)



T096 [TDD] Register every check this feature adds in `scripts/check-registry.tsv` so `scripts/verify-check-registry.sh` can enforce SC-012 and SC-013 mechanically — **[PATHS MEASURED 2026-09-02 — both files EXIST, and scripts/check-registry.tsv contains zero feature-001 rows: its non-comment rows are the umbrella’s own, and the only string matching “workshop” in it is a comment. workshop/platform/gates/check-registry-002.tsv covers feature 002 ONLY and says so in its own header, declaring platform/gates/ and platform/backend/gates/ out of its scanroot because those directories interleave 001 and 002 files. So no instrument in this tree enforces SC-012/SC-013 over any 001 check today. THAT LAST SENTENCE IS WITHDRAWN BY NAME — re-measured 2026-09-03, one now does, over PART of the set. workshop/platform/gates/check-registry-001.tsv exists, with verify-check-registry-001.sh walking it, and all four modes were RUN: default → rc 0 (“every registered feature-001 pipeline check is enumerated, exists, has a paired proof, and demonstrated its rc-2 state”, plus 1 registered DEBT row); --**

run-proofs → rc 0, 27 PASS, 0 FAIL, 1 DEBT; --prove-failure → rc 0, PROVE: 22 mutations, 22 caught, 0 missed; --strict → rc 1, FAIL [cross-registry-attribution]. The --strict 1 is the honest measure of what is left, and it is not small: the scanroot is pipeline only (19 *.sh/*.py, 10 declared prunes), while 48 *.sh/*.py under platform/gates/ (36), platform/gates/lib/ (2) and platform/backend/gates/ (10) are swept by no registry — including six verify-*.sh enumerated nowhere at all (verify-absence-honesty.sh, verify-connectivity-matrix.sh, verify-crossref-navigation.sh, verify-redaction-propagation.sh, verify-server-unity.sh, verify-status-exit-contract.sh — the last two are the gates T021 and T039 rest on). The umbrella's own scripts/check-registry.tsv still holds 30 non-comment rows and zero feature-001 rows, as this note originally said. The registry's own debt row states plainly why it is unpaid — attributing each file between features 001 and 002 is a per-file judgement, and guessing to clear a debt row writes a false claim into the one artifact whose value is that its claims are true. PARTIAL: the pipeline third is enumerated and proven; the platform and backend gate directories are not] — BLOCKER: none but the work. RE-MEASURED 2026-09-04 and the debt moved AGAIN during this session, so quote the gate, not this line. Default run is rc 0 at 42 PASS, 0 FAIL, 3 DEBT; --strict is rc 1 on the 3 debt rows, and that 1 is the honest measure of what is left. --prove-failure is rc 0 at 22 mutations, 22 caught, 0 missed. One debt was **PAID this session**: redaction-propagation became a check after two REAL argv levers (-redact-sh, --work-dir) were wired into preconditions the gate already had — verified independently, --redact-sh /nonexistent -> rc 2, and a prove-*.sh L3 assertion exists so the levers are not trapdoors. A FOURTH debt row (resync-redaction-guard) was added by another agent in the same window, which is why the count is still 3. Two remain: design-system-token-source (needs a node arm in arg_is_real/proof_cmd/probe_cmd + an *.mjs R2 arm; node build.mjs --check --brand-css /nonexistent -> rc 2 is already measured, so it is probe-able and merely untypeable) and cross-registry-attribution (needs a sweep that consults BOTH registries; 55 *.sh/*.py now sit under platform/gates + platform/backend/gates, of which 24 are accounted for by neither registry — mostly §1.1 provers whose gates ARE registered but whose proof scripts are not). Note for the next reader: the new row's owed token verify-sibling is OUTSIDE the vocabulary this registry's own header documents, and nothing validates that field · **OWNER: implementer** — unblocked today, and it is still the only thing holding T097 and T098 (SC-012, SC-013, FR-032, FR-033)



T097 [TDD] Verify EVERY feature check has a paired mutation proof including a real-entry-point case (SC-012) — **[PARTIAL. Measured 2026-09-03; NEW note. The PAIRING is complete where it is measured and the REAL-ENTRY-POINT case genuinely exists; what is missing is the QUANTIFIER.**** workshop/platform/gates/ holds 17 verify-*.sh and 19 prove-*.sh with 0 unpaired by sibling name. Real-entry-point proofs were OBSERVED running, not just declared: workshop/scripts/verify.sh:613 — PROVE: PASS – E2E real verify.sh over a mutated tree -> rc 1 with a FAIL row — and bash scripts/verify-governance-cascade.sh --prove-failure → rc 0 with “*the REAL entry point ran against the REAL tree*”. But SC-012 says EVERY, and “every” is quantified over an enumeration that covers 19 pipeline/ files and no others (T096’s --strict rc 1). Worse for this task specifically: verify.sh --prove-failure proves only verify.sh’s OWN V1–V7 gates and never executes the 19 sibling prove-*.sh scripts — its single prove-*.sh reference is a synthetic fixture at verify.sh:608. So the proofs exist, are paired, and are not run by the thing that claims to run everything. This closes when T096’s registry covers platform/gates/ and platform/backend/gates/; it is **BLOCKED-ON-PREDECESSOR on T096] — BLOCKER: predecessor T096** — SC-012 says EVERY, and “every” cannot be quantified over a set nobody enumerates · **OWNER: implementer**, after T096 (SC-012, FR-032)



T098 [TDD] Verify EVERY feature check distinguishes could-not-determine from pass and fail (SC-013) — **[PARTIAL, WITH THE SAME QUANTIFIER GAP AS T097 AND ONE EXTRA FINDING. Measured 2026-09-03; NEW note.**** Structural census: 17 of 17 workshop/platform/gates/verify-*.sh and 10 of 10 workshop/platform/backend/gates/*.sh carry rc-2 vocabulary. It is **DEMONSTRATED** and not merely declared: registry-001’s --prove-failure includes M18 a sweep matching ZERO files is **UNDETERMINED rc 2 (PASS)**, and a live verify.sh run produced a genuine **COULD-NOT-RUN** row for G8 verify-entailment-loads.sh rather than a FAIL. The gap is the same one T096 and T097 name: 48 files sit outside any registry, and “100% of its checks” cannot be asserted over a set nobody enumerates. One extra finding, recorded rather than resolved: workshop/scripts/verify.sh:747–:752 documents **IN ITS OWN SOURCE** an unresolved divergence about whether a STOPPED stack is exit 0 or exit 1 — measured against contracts/pipeline-cli.md §4.8 and

against T021's now-green G-CLI-17, which asserts 1. A three-valued census is worth less while two instruments disagree about which value is right. **BLOCKED-ON-PREDECESSOR on T096]** — **BLOCKER: predecessor T096** — same quantifier gap, plus an unresolved divergence about whether a STOPPED stack is exit 0 or 1 · **OWNER: implementer**, after T096; the exit-code divergence is an **operator** call (SC-013, FR-033)



T099 Write machine evidence to workshop/evidence/ retained with the commit that produced it (FR-040) — **[PATH SPLIT 2026-09-02 — workshop/evidence/ EXISTS but holds only 3 tracked files (phase2-passage-identity/README.md, phase2b-mentions/README.md, phase2b-mentions/T014-unjoined-words-u5.md — and that last one is FEATURE 002's T014, not this file's). The machine evidence FR-040 asks for landed at workshop/platform/backend/evidence/: 2 1 tracked files, committed and clean. workshop/platform/qa/evidence/ exists on disk with 0 tracked files, and workshop/evidence/knowledge-pipeline/ carries untracked run output. The substance largely exists; its contracted LOCATION does not, so this task is a move-or-mirror decision plus the retention rule, not a from-zero build. RE-MEASURED 2026-09-03 — the shape holds, both counts moved, and a RETENTION BREACH is now visible. 3-tracked files →**

git -C workshop ls-files evidence | wc -l = 6 (chunking/ README.md, p-u1/README.md, p-u1/result.json, phase2-passage-identity/README.md, phase2b-mentions/README.md, and phase2b-mentions/T014-unjoined-words-u5.md, which remains 002's T014, not this file's). 21 → git -C workshop ls-files platform/backend/evidence | wc -l = 22. workshop/platform/qa/evidence/ holds 6 subdirectories on disk (answering, challenges, helixqa, prove, prove-state2, state2) with 0 tracked AND 0 untracked-not-ignored — it is entirely git-ignored, so none of it is retained with any commit at all. The retention half of FR-040 is measurably breached right now, not hypothetically: evidence/p-u1/result.json is M in the working tree, so the bytes on disk are not the bytes any commit carries — and those bytes are T014's P-U1 settlement evidence. Nothing in the tree gates FR-040 retention] — **BLOCKER: operator decision** — the substance largely exists but at a DIFFERENT path than FR-040 contracts, so this is a move-or-mirror decision plus a retention rule, not a from-zero build. A retention breach is live right now · **OWNER: operator** decides move-vs-mirror; **implementer** executes (FR-040, SC-018)



T100 [REVIEW] Confirm zero CI added anywhere as gate **G-CLI-11** ([contracts/pipeline-cli.md](#) §2.7 N1–N3, §5): assert no `.github/workflows/*.yml` file exists anywhere under `workshop/`, and that `bash scripts/pre-push-gates.sh` gate E is green across the fleet (SC-014, FR-034, §11.4.156). **Paired mutation**: add one workflow file under `workshop/.github/workflows/`; the gate MUST go red — gate E derives the owned fleet from `helix-deps.yaml`, so the umbrella catches it too (FR-034, SC-014, FR-032, SC-012)



T101 Update `CONTINUATION.md` and confirm `bash scripts/continuation-check.sh` is `rc=0` (FR-035, §12.10) — **[TICKED 2026-09-02 on a measured run. `CONTINUATION.md` is modified in the working tree (`git status --short CONTINUATION.md` → M) and `bash scripts/continuation-check.sh` exits `rc=0`, printing 8 PASS · 0 DRIFT · 0 UNDET · 4 NOTE and `CONTINUATION.md` IS IN SYNC, including [C7] §6 matches the live runner: 8 gate(s) and [C8] production facts hold. Honest boundary (§11.4.6): `rc=0` is a measurement of that moment, and the check goes stale as this tree moves — re-run it before relying on the tick]** (population: `in_process`) (FR-035)



T102 Confirm `workshop/`'s four governance carriers remain in lockstep — `bash scripts/verify-governance-cascade.sh` C8 (FR-035) — **[TICKED 2026-09-02 on a measured run. `bash scripts/verify-governance-cascade.sh` exits `rc=0` at 12 PASS, 0 FAIL, 0 ENV, 8 NOTE, with C8 reporting that all 11 owned submodules carry four agent carriers with byte-identical bodies once the per-agent header is normalised. Honest boundary (§11.4.6): the same run prints NOTEs about known-unclearable third-party carriers, which it excludes from its verdict rather than suppressing — read them; and this `rc` is a dated observation, not a standing fact]** (population: `in_process`) (FR-035)



T103 [REVIEW] Run the whole gate suite green: `pre-push-gates`, `verify-governance-cascade` (+ `--prove-failure`), `verify-manifest-pins`, `continuation-check`, `audit-hardcoded-paths`, `audit-environment-assumptions` — **[TICKED 2026-09-03. Every instrument this task names was RUN — not read, not inherited from a prior session's note — and every one exits 0:** (population: `in_process`) (FR-035)

instrument	rc	summary line
bash scripts/pre-push-gates.sh	0	gates E, 0, 1, 2, 3, 4, 5, 6 all ✓ PASS; ALL RUN GATES PASSED – push allowed.
bash scripts/verify-governance-cascade.sh	0	CM-GOVERNANCE-CASCADE: 12 PASS, 0 FAIL, 0 ENV, 8 NOTE
bash scripts/verify-governance-cascade.sh --prove-failure	0	M11/M12/M13 caught, CONTROL restored, “ <i>the REAL entry point ran against the REAL tree</i> ”
bash scripts/verify-manifest-pins.sh	0	CM-MANIFEST-PIN-SYNC: 12 MATCH, 0 DRIFT, 0 UNDETERMINED of 12
bash scripts/continuation-check.sh	0	8 PASS · 0 DRIFT · 0 UNDET · 7 NOTE / CONTINUATION.md IS IN SYNC
bash scripts/audit-hardcoded-paths.sh	0	no machine-specific hardcoded paths (15 file(s) explicitly allowed)
bash scripts/audit-environment-assumptions.sh	0	no NEW frozen environment assumptions (567 justified occurrence(s) allow-listed)

HONEST BOUNDARY (§11.4.6), THREE PARTS, AND THE TICK MEANS NOTHING WITHOUT THEM. (1) **This is a measurement of one moment, not a property of the tree.** Every one of these instruments has changed verdict in this repository without being edited, because the fleet moves underneath them; re-run before relying on the tick, and note two figures already moved since the root carriers were written (audit-hardcoded-paths 6 → 15 allowed files, audit-environment-assumptions 531 → 567 allow-listed occurrences). (2) **“The whole gate suite” is this task’s list, and**

this task's list is not every instrument in the tree. scripts/verify-content-boundary.sh is **RED BY DESIGN** and was deliberately not run as part of this — reading a green T103 as “the content boundary is clean” would be exactly wrong. scripts/verify-check-registry.sh is likewise outside this list. (3) **A green umbrella gate suite says nothing about feature-001's own gates** — workshop/scripts/verify.sh exits **1** today (PASS 24 FAIL 1 COULD-NOT-RUN 1 of 26, see T120), and 48 gate files under workshop/platform/ sit in no registry (T096). Those are T096/T097/T098/T120's problems, not this one's, and they are open]**



T104 Commit and push the umbrella and every submodule to all upstreams; verify each with `git ls-remote`, not a push log (FR-036, SC-015) — **not before T040's Chapter 1 redaction review is recorded**, since a push is publication — **[BLOCKED ON THE OPERATOR, ON TWO INDEPENDENT COUNTS. Measured 2026-09-03; NEW note. (1) Its own precondition is not fully met.** T040 part 1 (record) and part 3 (re-emit + assert R2) are DONE, but part 2 is not: **7 of 13 REDACT decisions — F8, F9, F12, F13, F14, F15, F16 — are DEFERRED and unapplied by operator decision**, recorded in the review artifact's own note field. A push is publication, and this task's own clause says not before the review is recorded; applying the deferred seven, or deciding they need no application, is an operator call. (2) **Pushing is an operator action in this project regardless** — per the root carriers, commit and push go through the project wrapper on an explicit instruction, and this triage pass deliberately made none. State at this measurement: **9** dirty paths in the umbrella and **39** in workshop/, several of them files other agents were writing while this was measured; **no upstream tracking branch is configured** in either checkout (`git rev-list @{u}..HEAD` fails in both), so “*verify each with git ls-remote*” has nothing to compare against until remotes are wired or named explicitly. — **BLOCKER: operator** — a push is publication, and this must not happen before T040's Chapter 1 redaction review is recorded · **OWNER: operator** — nothing an agent may do (FR-036, SC-015, FR-039)

RE-MEASURED 2026-09-03 AFTER AN AUTHORIZED ATTEMPT ON T040 PART 2: STILL BLOCKED, AND COUNT (1) IS UNCHANGED RATHER THAN NARROWED. The seven deferred REDACT decisions were resolved, backed up and attempted through `redact.sh`; the tool refused on two independent surfaces and wrote nothing, so **all seven — F8, F9, F12, F13, F14, F15, F16 — remain unapplied** and T040's part 2 is still open. See T040's own note for

the measurement. One of the seven, **F16, is separately satisfied and gated** (the listing boundary discloses no source filename), but a partial application is not the precondition this task names. Count (2) is likewise unchanged and was not tested: no commit and no push was made. **T104 is NOT unblocked.]****



T119 [TDD] [UNWRITTEN — NOTHING BLOCKS IT. Measured 2026-09-03; NEW note.** Searching for G-CLI-1 with G-CLI-1[0-9] excluded, the identifier occurs exactly once anywhere under workshop/ — platform/backend/cmd/workshop-redact/main.go:55, a code comment, not a gate. The full inventory of gate identifiers actually present under workshop/ is G-CLI-{1,3,5,11,13,16,17}, G-HTTP-{1..8,8c} and G-PID-4; nothing asserts state/exit agreement across every command, and nothing asserts the --json contract at all — there is no test that stdout carries exactly one JSON object with all human text on stderr, and no state: "ok" hardcode mutation. Two facts make this cheaper than it looks and both were measured: redact.sh already writes a per-run result file (see T107), and verify.sh already discovers gates at run time. Two facts make it harder: that file is named result-exit.json and carries {run_id, exit, at} with no state field to compare the exit against (T107), and this task quantifies over *“every command in the control plane and the pipeline”*, several of which do not exist yet (T111, T115, T116, T084). Sequencing it last, as this phase does, is correct] Prove gate G-CLI-1 (contracts/pipeline-cli.md §1.6, §5) across **every** command in the control plane and the pipeline: result.json.state and the process exit status agree over all three outcomes — "ok"/0, "problem_found"/1, "could_not_determine"/2 — and under --json stdout carries exactly one JSON object and nothing else, with all human text on stderr. **Paired mutation:** hardcode state: "ok" in one command's result writer; the gate MUST go red. It sits in this phase because it asserts over every command and cannot run until they all exist — **BLOCKER:** none but the work. Re-measured 2026-09-04: NOT STARTED — `grep -rn 'G-CLI-1'` under workshop/ still returns only the one code comment at platform/backend/cmd/workshop-redact/main.go:55. It is now CHEAPER than it was: T107 landed result.json with a state field derived from a single exit-to-state mapping, which is the artifact G-CLI-1 must compare against the process exit · **OWNER: implementer** — unblocked today (FR-033, SC-013, FR-032, SC-012, FR-040)



T120 [TDD] [REVIEW] [PARTIAL — BUILT, RUNNING, AND ITS HARDEST CLAUSE IS THE ONE THAT HOLDS. Measured 2026-09-03; NEW note, replacing the File Structure table's implicit reading that this is unbuilt.** workshop/scripts/verify.sh EXISTS, 812 lines, and a live run exits 1 printing PASS 24 FAIL 1 COULD-NOT-RUN 1 of 26 [scope: all], naming FAIL: G12 verify-retrieval-benchmark.sh and COULD-NOT-RUN: G8 verify-entailment-loads.sh separately, then

RESULT 1 a gate RAN and reported a violation (1 could not run and are listed above). That is the precedence rule this task calls “the single most error-prone line in the contract”, observed working on a real mixed outcome — 1 FAIL beside 1 COULD-NOT-RUN aggregating to 1, both counted, in the exact PASS n FAIL n COULD-NOT-RUN n of N vocabulary the task specifies (implementation :795–:812). --prove-failure → rc 0 with 26 PROVE : lines, all PASS, including E2E real verify.sh over a mutated tree -> rc 1 with a FAIL row, precedence: 1 FAIL + 1 COULD-NOT-RUN aggregates to rc 1, both counted, a SKIP is never a PASS: all-green + 1 unrunnable -> rc 2, and V7b pairing: NO gates at all is 2, not 0 — both forbidden collapses proved in both directions. --help → rc 0. TWO CLAUSES ARE UNMET AND BOTH ARE MEASURED. (1) “*runs every gate in this contract set (all G-CLI-*, G-HTTP-* and G-PID-*)*” — it does not. It discovers platform /gates/verify-*.sh by filename and references only 2 of the 17 contract identifiers present in the tree (G-CLI-11, G-CLI-16); discovery answers “*what is on disk*”, never “*what the contract requires and is missing*”, and two of the required gates do not exist at all (T119's G-CLI-1, T114's G-CLI-15). (2) “*a gate whose mutation does not turn it red is reported vacuous and counted as FAIL*” — grep -c vacuous workshop/scripts/verify.sh = 0; --prove-failure proves verify.sh's own V1–V7 and never executes the 19 sibling prove-*.sh scripts, so no mutation of a real gate is ever judged vacuous. This is also correctly sequenced last: it cannot be finished before the gates it must aggregate exist]

Implement workshop/scripts/verify.sh per contracts/pipeline-cli.md §4.9 — the aggregation point that runs every gate in this contract set (all G-CLI-*, G-HTTP-* and G-PID-*) and aggregates them. **Sequenced last, after every gate task in every phase**, because it aggregates those gates and cannot be written before them. Three-valued: 0 every gate ran and passed · 1 at least one gate ran and reported a violation · 2 at least one gate **could not run** and no gate reported a violation. **The precedence rule is the single most error-prone line in the contract, and both directions of collapsing it are forbidden here by name:** when some gates failed

(1) and others could not run (2), the aggregate is **1** — a confirmed violation outranks an unknown — and the summary reports the two counts separately in the repository's existing sweep vocabulary (PASS n FAIL n COULD-NOT-RUN n of N, then each could-not-run gate named with its reason). A `verify.sh` that collapses 2 into FAIL is the specific defect to forbid: this project has shipped **seven** separate pass/fail/could-not-determine conflations, so a broken instrument must never be reported as a violated tree. The mirror defect is equally forbidden — it must never report 0 while any gate was skipped. PREPUSH_STRICT=1 semantics apply: a SKIP is never a PASS. `verify.sh --prove-failure` runs every registered paired mutation and asserts its gate turns red; a gate whose mutation does not turn it red is reported **vacuous** and counted as **FAIL**, and at least one proof case MUST run the real entry point end to end rather than a sandboxed copy, because a proof that exercises only sandboxed copies can go green over an instrument that cannot start at all. Per §2.7 N2 this script — a script, never a workflow — is what the umbrella's local pre-push hook calls (FR-032, FR-033, FR-040, SC-012, SC-013, SC-018) — **BLOCKER:** none but the work — `verify.sh` exists at 812 lines, is running, and its hardest clause holds; the remainder is unbuilt · **OWNER: implementer** — unblocked today (FR-032, FR-033, FR-040, SC-012, SC-013, SC-018)



T105 [REVIEW] Final review against `spec.md` — every FR traced, every SC measured or explicitly recorded as not-yet-measurable — **[BLOCKED ON ITS PREDECESSORS, AND PARTLY DRAFTED. Measured 2026-09-03; NEW note.** specs/001-workshop-curriculum-platform/review.md exists, is TRACKED, is 684 lines, and does a large part of this task: §2 dispositions all 120 tasks, §7 is “Success criteria — measured, failing, or unmeasured”, §4 enumerates 11 defects and §9 states its own honest boundary. But this task has two halves and only one is drafted. Measured by identifier coverage: spec.md defines 22 SC- ids and review.md cites 20; it defines 48 FR- ids and review.md cites 5. “Every FR traced” is 5/48. It is also, by its own name, the FINAL review — it cannot be closed while 56 tasks in this file remain unticked, most of them things it would have to review. Sequence it after them; in the meantime read review.md as the SC half in draft, not as this task] — BLOCKER: predecessor: essentially all of the above — a final review against spec.md cannot be honest while 50+ tasks are open. Partly drafted already · OWNER: human reviewer, last (FR-?)**

Triage of the 55 open tasks — measured 2026-09-03

Why this section exists. Before this pass, 61 boxes were unticked and nobody could tell which were genuinely blocked, which were done-but-unticked, and which were simply unwritten. A previous sweep had already found tasks marked [PATH NOT BUILT] whose tooling demonstrably existed and had run, so the notes could not be trusted as a survey. Every row below was determined by **running** the thing it describes or by a command whose output is quoted in that task's own note — not by reading its prose. **Six boxes moved to ticked on this pass** (T012, T014, T025, T032, T039, T103); none was untied.

Read the class, then read the task's own note for the evidence. Every row's justification lives on its task line above, with paths, line numbers and exit codes. This table is an index, not a substitute.

Class	n	Task ids
PARTIAL — some parts met, the missing half named on the task line	25	T005 · T019 · T020 · T021 · T029 · T030 · T034 · T035 · T038 · T051 · T059 · T073 · T075 · T076 · T078 · T079 · T080 · T081 · T095 · T096 · T099 · T106 · T107 · T113 · T120
UNWRITTEN — nothing blocks it but the work	17	T006 · T015 · T016 · T017 · T022 · T023 · T028 · T031 · T064 · T067 · T083 · T084 · T111 · T114 · T115 · T116 · T119
BLOCKED-ON-PREDECESSOR	9	T085, T086, T087, T088, T090 (all on T084) · T097, T098 (on T096) · T118 (on T064) · T105 (on the rest)
BLOCKED-ON-OPERATOR — needs	3	T037 · T041 · T104 — <i>T040 left this class 2026-09-03: its part 2</i>

Class	n	Task ids
a human decision or human labour	54	<i>was applied, exit 0, targets reached (8/8)</i>

The named operator decisions, so none of them hides in a class label

1. **T037(b)** — a **blind human verbatim transcript** of the 30 windows in accuracy-plan.json, ~1–2 hours. bash workshop/scripts/verify-accuracy.sh 01 exits 2 (--reference is required), re-run 2026-09-03; no accuracy.json exists; publish precondition B2 is unmet. No agent can produce it — an engine cannot be its own ground truth. **A locally-run second engine WAS put over the same 30 windows on 2026-09-04 and this row did not move:** it yields inter-engine agreement (0.0858, CI [0.0671, 0.1086] over n=30 window clusters), which is not accuracy, and the scorer now REFUSES the artifact it produced with exit 2 — see T037(f).
2. **T037(d)** — a **sampling-design decision** that must be taken BEFORE (b), because re-emitting changes the seeded window set and invalidates any reference already transcribed.
3. **T040 part 2 — 7 of 13 REDACT decisions (F8, F9, F12–F16) are DEFERRED. WITHDRAWN 2026-09-03: applied.** curriculum/redactions.jsonl 10 -> 142 entries (84 passages under six per-finding reason codes, plus the 48-row transitive closure of minted registry rows); F16 needed no corpus change. Final propagate exit 0, targets reached (8/8), 0 PROBLEM · 0 UNDETERMINED, idempotent on re-run, and R2 re-measured at 0 residual across all seven artifacts. Two things did NOT close and are in T040's note: a regeneration from clean re-derives **2 of 27** withdrawn index terms *from still-visible passages* (a review decision about the corpus, not a tooling defect), and the regeneration's own reconcile_and_taxonomy stage is rc 2 on a 180-second minting-bridge timeout.
4. **T041 — human speaker attribution.** D-TRANS-2 measured both diarization cues absent, so no tool on this host can do it.
5. **T104** — pushing is an operator action. **Its T040 gate is now clear** (part 2 applied 2026-09-03); the operator-action half is unchanged and untested — no commit and no push was made.
6. *(Not a class of its own, but operator-shaped and living inside PARTIAL rows):* **T073** needs an ONNX entailment model that does not exist on this host (verify-entailment-loads.sh → rc 2); **T080's** egress-denied namespace plausibly needs host privileges; **T006**

contains a genuine contradiction — the reference module it is told to match carries no ESLint config.

Shortest path forward — measured by how many other tasks each one releases

1. **T084 add-chapter.sh releases FIVE** (T085, T086, T087's enforcement half, T088, T090) and is blocked by nothing but the work. Its nine stages are 2 built, 3 unwritten (T111, T115, T116), 1 built-and-ticked (T039), and 3 that exist as libraries needing a driver. **This is the single highest-leverage open item in the file.**
2. **T096 registry coverage releases TWO** (T097, T098) and is the only reason SC-012/SC-013 cannot be asserted at all: 48 gate files under platform/gates/ and platform/backend/gates/ sit in no registry, six of them verify-*.sh enumerated nowhere.
3. **T064 retrieval.tsv releases ONE** (T118) and is the only thing standing between this feature and any SC-007/SC-008 figure whatsoever. It is a data file.
4. **T038's missing clause and T059's missing R5 are ONE gap seen from two ends** — a recorded redaction never marks the live generation stale — and closing it ticks T038 and unblocks the R5 half of T059 together.
5. **T014 already released THREE** on this pass: T015, T016 and T017 were BLOCKED-ON-PREDECESSOR and are now ordinary unwritten work, because P-U1 is settled and the producer is named.

What could not be determined

- **T079's answered+cited fell from a recorded 17/24 to a measured 3/24** on generation 67, with unavailable 0 ruling out timeouts. **The cause was not diagnosed.** L4 being effectively absent on this host (T073) is a candidate that was NOT tested. Recorded as an observation, not explained.
- **T094's second artifact** (tutorial-quickstart.md): whether workshop/docs/quickstart.md was meant to absorb it cannot be determined from the tree. That task is ticked and was left ticked.
- **T032:** the module and its 9-mutation proof were measured; **no run against the real Chapter 1 notes PDF was performed**, and by the module's own tracked-destination guard its output may not land in a tracked path.
- **T039's successful none_required write** could not be driven on this corpus — chapter 01 has redactions and no second chapter exists in

the registry (--chapter 02 → rc 2, the honest answer). Proven by its guard, its derivation site and gate A8's fixture instead.

Every exit code in this section is a measurement of 2026-09-03, not a property of the tree. Re-run before relying on any of them.

Dependencies

Phase 1 Setup

↓

Phase 2 Foundational ← BLOCKING: passage identity gates everything

↓

Phase 3 US1 transcript (P1) — MVP

↓

Phase 4 US2 browse (P2) [needs US1's transcript]

↓

Phase 5 US3 search (P3) [needs US2's surface to present results]

↓

Phase 6 US4 answers (P4) [needs US3's retrieval; L3 needs the pid registry]

↓

Phase 7 US5 add-chapter (P5) [exercises the whole pipeline]

↓

Phase 8 Polish

Cross-story dependency worth stating: US4's L3 citation check depends on Phase 2's pid registry, not on US3. If the registry were skipped, SC-009 becomes unreachable.

FR-039 redaction spans five phases and is deliberately not a single task. The registry half is foundational (T011 resolution, T012 the append-only log); the CLI and the real Chapter 1 review land in US1 (T038, T039, T040) because the transcript exists from T030 onward and FR-039 blocks its export; propagation into search and cross-references is T059 and T061; propagation into stored answers is T075; the composite gate G-PID-5 is T076, the earliest point at which all three propagation paths exist to be proven; and the publish precondition is T087. **Removing any one of them leaves FR-039 decorative**, which is why none of them sits in Phase 8.

Code-passage identity (SC-016a) is gated on an unverified producer.
T014 must settle P-U1 and exit 0 before T015, T016 and T017 may be started. If it exits 1, SC-016a is not implementable as specified and the gap is recorded rather than papered over — see Global Constraints.

Parallel Opportunities

- **T004, T005, T006** — different languages, different directories
- **T018, T019, T020, T021** — independent foundational utilities
- **T046, T047, T048** — separate frontend feature directories
- **T062, T063** — search UI and its accessibility audit
- **T091–T095** — the whole documentation set, once contracts are fixed
- **workshop governance and gate work** is independent of all feature work

Not parallelisable, despite looking it: T015, T016 and T017 all depend on T014’s outcome, and T059, T061, T075 and T076 all depend on T012’s redaction log.

Independent Test Criteria

Story	Independently testable by
US1	Reading the transcript and sampling five passages against the recording
US2	Starting the curriculum and opening Chapter 1
US3	Running the ≥ 20 -query benchmark
US4	Running the answer set and the ≥ 10 unanswerable set
US5	Running <code>add-chapter.sh</code> twice on a synthetic chapter

Gate coverage — every contracted gate has a task that builds it

~~contracts/ defines 30 distinct gate identifiers matching G-[A-Z]+-[0-9]+.~~ **COUNT CORRECTED 2026-09-02: contracts/ defines 31.** Each row below names the task that builds the gate **and** its §1.1 paired mutation. A gate defined in a contract and built by no task is a gate that will not exist, so this table is the closure condition, not a convenience. Re-derive it rather than trusting it — with the ATTACHMENT check, not the presence check:

```
# THE CLOSURE CHECK. A contract gate must be carried by a TASK
    LINE, not merely
# appear somewhere in this file. Prints one row per unattached
    gate; silence = closed.
cd specs/001-workshop-curriculum-platform
miss=0
for g in $(grep -rhoE 'G-[A-Z]+-[0-9]+(-[a-z]+)*' contracts/ |
    sort -u); do
    grep -qE "^[:space:]*-?[:space:]*\[[ xX]\[
        [:space:]*T[0-9]+.*${g}([^\0-9A-Za-z-]|$)" tasks.md \
    || { echo "UNATTACHED $g"; miss=$((miss+1)); }
done
echo "unattached: $miss"      # MUST be 0 — re-measured 2026-09-03
    with the
                                # corrected extractor and guard
    below: still 0 here
```

THIRD FAILURE MODE OF THIS SAME CHECK, found 2026-09-03 — the extractor could not SEE a compound gate id.

The two regexes above were widened on that date, and the reason belongs next to the earlier two lessons rather than in a commit message. `G-[A-Z]+-[0-9]+` is greedy only through the digits: run it over the string `G-KG-1-changed` and it yields **G-KG-1** — a *different, already-attached* gate. A compound id therefore never entered the loop at all, so it could be neither attached nor reported unattached; it was **invisible**, and the check printed `unattached: 0` while a contract gate stood unbuilt. Feature 002 defines exactly such an id and the blind zero concealed it — see that file's companion section.

Widening the extractor alone is **not** the fix, and this is the part worth keeping. Once both `G-KG-1` and `G-KG-1-changed` are in the id set, the old boundary guard `([^\0-9]|$)` matches `G-KG-1` against a task line that mentions only `G-KG-1-changed` — so a bare gate would be reported attached on the strength of a *different* gate's citation. The guard is therefore `([^\0-9A-Za-z-]|$)`: an id must not

be followed by a hyphenated continuation. Both halves were proved on a fixture whose correct answer was known by construction (one task line building only G-KG-1-changed): the old pair called all four ids attached, the new pair correctly called G-KG-1 UNATTACHED.

The correction leaves this feature's verdict untouched — 31 ids before, 31 after, unattached: 0 both ways. That is the evidence that it discriminates rather than merely reddening: a fix that only ever finds fault is indistinguishable from a broken instrument.

WHY PRESENCE-COUNTING IS INSUFFICIENT, and this is the durable part of this section. The obvious check is set difference over identifiers anywhere in the file:

```
grep -rhoE 'G-[A-Z]+-[0-9]+' contracts/ | sort -u > /tmp/defined
grep -rhoE 'G-[A-Z]+-[0-9]+' tasks.md | sort -u > /tmp/cited
comm -23 /tmp/defined /tmp/cited # NOT the closure condition — see below
```

That form was the closure check here, and it **lied**. When the G-CLI-17 gap was first found it read 31 defined vs 30 cited and correctly printed G-CLI-17. Then the gap was **written down** — in prose, in this very section — and the prose mentioned G-CLI-17 twice. The next run read **31 vs 31 and PASSED**, with the defect completely untouched: no task built the gate, and the only thing that had changed was that a paragraph now named it.

Documenting the defect turned the check green. A presence check cannot tell a gate that is BUILT from a gate that is merely DISCUSSED, and prose about a gap is exactly the thing most likely to mention the gap's identifier. The attachment form above is anchored to a task line — `^[ ]*- [ ]` or `- [x]` followed by `T###` — so a mention in a note, a table, a dependency list or a withdrawal block counts for nothing. Verify the discrimination on one gate directly:

```
grep -nE '^\\s*-*?\\s*\\[ [xX]\\].*G-CLI-17' specs/001-workshop-curriculum-platform/tasks.md
```

Keep both forms if you like — but read the ATTACHMENT one for the verdict. A check that a comment about the problem can satisfy is not a check.

THE G-CLI-17 GAP — RECORDED OPEN 2026-09-02, ASSIGNED 2026-09-02. Kept, not deleted, because the reason it was open is the lesson. As found, `comm -23` printed **G-CLI-17** — a gate defined at `contracts/pipeline-cli.md:989` (`status.sh` against a project with no containers must print `STOPPED` and exit 1, the determined-negative sibling of G-CLI-12’s 2) and cited by **no task in this file**. It was not a regression from the 2026-09-02 path sweep: the same command over `git show HEAD:specs/001-workshop-curriculum-platform/tasks.md` printed it too, so the gap predated that sweep and the sweep removed no gate identifier (30 cited before, 30 cited after).

The sentence this block used to carry — *“This block records the gap; it does not close it ... the table below is therefore left at 30 rows and is knowingly incomplete”* — is **WITHDRAWN, not restated**. The operator took the decision on 2026-09-02: **G-CLI-17 is assigned to T021**, alongside G-CLI-12, because the contract itself calls them siblings on the same script. The task line now carries the gate and its paired mutation, and the table below is **31 rows**. Assigning a gate changes what this file asks for, which is why it needed an operator decision and not a path correction — that reasoning stands; it has simply been answered.

Gate	Built by	Gate	Built by
G-CLI-1	T119	G-CLI-9	T107
G-CLI-2	T111	G-CLI-10	T106
G-CLI-3	T019	G-CLI-11	T100
G-CLI-4	T029	G-CLI-12	T021
G-CLI-5	T112	G-CLI-13	T081
G-CLI-6	T115	G-CLI-14	T080
G-CLI-7	T085	G-CLI-15	T114
G-CLI-8	T086	G-CLI-16	T110
G-CLI-17	T021 (with G-CLI-12; assigned 2026-09-02)	—	—
G-HTTP-1	T055	G-HTTP-5	T081
G-HTTP-2	T055	G-HTTP-6	T117
G-HTTP-3	T055	G-HTTP-7	T057

Gate	Built by	Gate	Built by
G-HTTP-4	T056	G-HTTP-8	T053
G-PID-1	T013	G-PID-4	T017
G-PID-2	T013	G-PID-5	T076
G-PID-3	T066	G-PID-6	T113

~~T120 (verify.sh) aggregates all thirty~~ — **thirty-one** as of the 2026-09-02 G-CLI-17 assignment — and it is therefore sequenced after every one of them. A gate whose mutation does not turn it red is **vacuous** and counts as FAIL there, not as a pass.

Contracted scripts and their tasks, so a missing entry point is as visible as a missing gate. **Existence re-measured 2026-09-02** — BUILT means the file is on disk at the path shown, not that its contracted option surface is complete:

Script	Task	Measured 2026-09-02
workshop/scripts/build.sh	T108	BUILT
workshop/scripts/{start,stop,status}.sh	T021	BUILT
workshop/scripts/restart.sh	T109	BUILT
workshop/scripts/verify.sh	T120	BUILT
workshop/scripts/_common.sh	T106, T107	BUILT — but it is the container control-plane helper, not the §2.1 contract; §2.4/§2.5/§2.9 are absent
workshop/scripts/transcribe.sh	T111	[PATH NOT BUILT]
workshop/scripts/verify-accuracy.sh	T112	BUILT — and RUN 2026-09-03 against the real chapter: exits 2 UNDETERMINED: -- reference is

Script	Task	Measured 2026-09-02
		required, a correct CANNOT DETERMINE. <i>“NEVER RUN ... no a ccuracy*.json exists anywhere”</i> is WITHDRAWN, not restated — chapters/01/ transcript/ accuracy-plan.json exists. No accuracy.json, so B2 is NOT met (T037) BUILT — option surface is [chapter- slug] [--require- suspend] [--corpus] only
workshop/scripts/ ingest.sh	T113	
workshop/scripts/ index.sh	T115	[PATH NOT BUILT]
workshop/scripts/ crossref.sh	T116	[PATH NOT BUILT]
workshop/scripts/ redact.sh	T038, T039	[PATH NOT BUILT]
workshop/scripts/add- chapter.sh	T084	[PATH NOT BUILT]
workshop/scripts/ bench-suggest.sh / bench-search.sh	T065	[PATH CORRECTED] → one Go binary, workshop/ platform/backend/ cmd/bench/main.go, -endpoint suggest\ search
workshop/scripts/ bench-retrieval.sh	T118	[PATH NOT BUILT] — and its input retrieval.tsv is absent too
	T078	

Script	Task	Measured 2026-09-02
workshop/scripts/ bench-answers.sh		[PATH CORRECTED] → workshop/platform/ backend/gates/ bench-answers.sh, which exists and has run

Implementation Strategy

MVP = Phase 1 + Phase 2 + Phase 3 (US1). That delivers a faithful, measured, timestamped transcript of Chapter 1 — which converts a 1.8 GB unusable asset into readable, quotable, linkable knowledge. It is worth shipping even if nothing else is ever built. Its redaction review (T040) is part of the MVP, not a follow-up: the recording features an identifiable third party, so the MVP cannot be published without it.

~~Deliberate sequencing around what does not exist: no ASR engine and no generative model are installed. US1 needs an ASR install (T005). US4 needs an operator ollama pull for its generative path — BOTH~~
PREMISES WITHDRAWN 2026-09-02, and withdrawn on measurement. A dual ASR stack is installed in a project-local venv (workshop/pipeline/venv with CT2 weights under workshop/pipeline/models/ct2/, plus a CPU-only workshop/pipeline/engines/whisper.cpp build with ggml weights), and qwen2.5:3b-instruct-q4_K_M is pulled and named in the running container's -answer-model argv — see the withdrawal block at the head of Phase 6. Re-derive with ollama list and workshop/pipeline/venv/bin/python -c 'import faster_whisper' rather than trusting this sentence; note that bare python3 still has no ASR engine, so a probe that runs the system interpreter concludes the opposite and is wrong. What T005 still owes is venv-setup.sh, not the engine. T069's extractive adapter delivers grounded answering with zero generative capability, so US4 was never blocked outright and is not blocked now.

Task count: 120 · US1 22 · US2 11 · US3 20 · US4 16 · US5 7 · Setup 7 · Foundational 20 · Polish 17

Ids run T001–T120, contiguous and unique. The count was re-derived by parsing this file, not by arithmetic on the previous line; re-derive it the same way rather than trusting the number:

```
awk '/^## Phase /{p=$0} /^- \[ \] T[0-9]+/{n++; c[p]++} END{for(k  
    in c) print c[k], k; print "TOTAL", n}' \
```

specs/001-workshop-curriculum-platform/tasks.md